UNIVERSITATEA BABEŞ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ

LUCRARE DE LICENŢĂ

Optimizarea prin preprocesare adaptivă a sistemelor de recunoaștere facială bazate pe rețele neuronale cu spiking

Conducător științific

Drd. Marc Daria
Prof. Dr. Pop Horia Florin

Absolvent
Bubuianu Adrian-Florin

2026

ABSTRACT

This thesis investigates the optimization of facial emotion recognition (FER) systems based on Spiking Neural Networks (SNN) through adaptive preprocessing techniques. SNNs represent the third generation of neural network models, processing information via sparse discrete spikes rather than continuous values, offering significant advantages in biological plausibility and energy efficiency. The theoretical foundation covers the mathematical modeling of spiking neurons (Leaky Integrate-and-Fire model), neural coding schemes (Rate Coding, Temporal/Latency Coding, Rank Order Coding), synaptic plasticity mechanisms (STDP and its variants), homeostasis through adaptive threshold adaptation, and 3D Convolutional Spiking Neural Network (CSNN) architectures. The preprocessing pipeline integrates the Viola-Jones face detection algorithm, KLT optical flow tracking, 68-point facial landmark extraction, and three sampling strategies (random, bounding-box-based, and landmark-centred) to isolate the Region of Interest before spike conversion. Experiments are conducted on the CK+ dataset (309 sequences, 7 emotion classes) using 10-fold cross-validation. The proposed 3D CSNN with unsupervised STDP learning achieves a mean accuracy of 48.02%, outperforming the raw-pixel SVM baseline (41.61%) and demonstrating that bio-inspired feature extraction encodes temporally discriminative information. An equivalent ANN with supervised backpropagation achieves 73.23%, while a high-performance hybrid CNN-LSTM model (ResNet18 backbone) reaches 88.37% ± 5.65%, providing a state-of-the-art reference. Feature map and Grad-CAM analyses offer qualitative validation of the regions each model attends to for each emotion class. The practical application, Neuro-Guardian, is a real-time clinical monitoring platform that integrates the CSNN inference engine into a hospital workflow. The system comprises a Node.js/Express backend, a React/TypeScript frontend, PostgreSQL for relational data, InfluxDB for emotion time-series, and a WebSocket–TCP bridge for live inference streaming. The alert subsystem implements a hysteresis mechanism (thresholds 70/55) and computes Distress and Pain scores from the inferred emotion probabilities to notify the attending physician of sustained negative emotional states.

Cuprins

Introducere **1**

Structura lucrării **3**

1 Motivația și formularea problemei **4**
1.1 Contextul general și relevanța problemei 4
1.2 Formularea problemei 5
1.3 Dificultatea recunoașterii emoțiilor 6

Încadrarea în literatura de specialitate **7**

2 Fundamente teoretice despre inteligența artificială și rețele neuromorfice **11**
2.1 Modele neuronale artificiale de a doua generație 11
2.1.1 Învățarea automată și rețele neuronale artificiale (ANN) . . . 11
2.1.2 Rețele Neuronale Convoluționale (CNN) 12
2.1.3 Rețele Recurente și LSTM 14
2.1.4 Mașini cu Vectori Suport (SVM) 15
2.2 Trecerea de la ANN la Spiking Neural Networks (SNN) 16
2.3 Modelarea matematică a neuronului cu impulsuri 16
2.4 Codificarea informației neurale (Neural Coding) 19
2.5 Mecanisme de învățare și plasticitate sinaptică 21
2.6 Arhitecturi Spiking Convolutional Neural Networks (CSNN) 25
2.7 Algoritmi de preprocesare și detecție facială 28
2.8 Metode de eșantionare a datelor de intrare (Sampling) 29
2.9 Metode de Decodificare a Semnalelor Neuronale 33

3 Experimente și analiza rezultatelor **35**
3.1 Setul de date 35
3.2 Metodologie de antrenare și evaluare 36
3.3 Metrici de evaluare 37
3.4 Rezultate obținute și analiză 39
3.5 Analiză Comparativă și Validarea Modelului 45

3.6 Modelul de Înaltă Performanță: Arhitectura Hibridă CNN-LSTM . . 46
3.7 Sinteza Rezultatelor și Concluzii Experimentale 47
3.8 Modelul ANN Echivalent: Izolarea Impactului Paradigmei de Spiking 51

4 Neuro-Guardian **53**
4.1 Colectarea cerințelor 53
4.2 Modelul structural 57
4.3 Modelul dinamic al sistemului 61
4.4 Sistemul de alerte și notificări 69
4.5 Testarea aplicației 72
4.6 Testarea de integrare (Top-down, Depth-first) 73
4.7 Manual de utilizare al aplicației 77

5 Concluzii **82**
5.1 Sinteza concluziilor experimentale 82
5.2 Impactul preprocesării adaptive 83
5.3 Sistemul Neuro-Guardian 83
5.4 Limitări identificate 83
5.5 Direcții viitoare de cercetare 84

Bibliografie **85**

Introducere

Recunoașterea automată a emoțiilor faciale reprezintă una dintre cele mai active direcții de cercetare la intersecția dintre inteligența artificială, neuroștiință și viziunea computerizată. Aplicațiile sale acoperă un spectru larg, de la sisteme de asistență pentru persoane cu dizabilități de comunicare și detectarea stărilor afective în contexte clinice, până la interfețe om-calculator adaptive și sisteme de supraveghere a siguranței la volan.

Odată cu creșterea exponențială a volumelor de date vizuale și cu necesitatea procesării în timp real pe dispozitive cu resurse limitate, limitele energetice ale arhitecturilor clasice de tip ANN devin din ce în ce mai vizibile, deschizând calea pentru paradigme computaționale alternative.

Rețelele neuronale cu impulsuri (*Spiking Neural Networks* – SNN) constituie a treia generație a modelelor neuronale artificiale și se aseamănă mult mai fidel cu mecanismele biologice ale creierului uman. În aceste modele, informația este transmisă prin trenuri de impulsuri discrete, iar neuronii se activează rar, doar atunci când potențialul acumulat depășește un prag critic. Această caracteristică de activare rară (*sparse activation*) reduce drastic numărul operațiilor de calcul față de rețelele clasice și face SNN-urile ideal potrivite pentru implementare pe hardware neuromorfic dedicat, cum ar fi Intel Loihi sau IBM NorthPole, unde consumul energetic poate fi redus cu până la 75% față de echivalentele ANN [WDZ+22].

Obiectivele lucrării

Prezenta lucrare de licență are ca obiectiv central investigarea potențialului rețelelor neuronale convoluționale de tip spiking (CSNN) în sarcina recunoașterii emoțiilor faciale, cu accent pe impactul preprocesării adaptive asupra calității trăsăturilor extrase. Concret, obiectivele urmărite sunt:

- **Implementarea unui simulator CSNN 3D** cu învățare nesupervizată prin regula STDP (*Spike Timing-Dependent Plasticity*), capabil să proceseze secvențe video ale expresiilor faciale.

- **Proiectarea unui pipeline de preprocesare adaptivă** care permite eșantionarea locală, în jurul celor 68 de repere faciale, facilitând analiza dinamică a musculaturii feței pe parcursul cadrelor videoclipului.
- **Evaluarea comparativă** a modelului SNN propus față de un clasificator SVM pe pixeli bruți, un ANN echivalent cu *backpropagation* supervizat și o arhitectură hibridă CNN-LSTM.
- **Dezvoltarea aplicației Neuro-Guardian**, un sistem clinic de monitorizare în timp real pentru urmărirea stărilor emoționale ale pacienților internați.

Contribuții proprii

Contribuțiile originale ale prezentei lucrări includ:

1. Proiectarea unui pipeline de preprocesare adaptivă bazat pe repere faciale, cu trei strategii de eșantionare (aleatorie, *bounding-box* și *landmark-centrată*);
2. Studiul comparativ sistematic între patru paradigme de clasificare pe același set de date;
3. Proiectarea și implementarea completă a aplicației clinice **Neuro-Guardian**, incluzând canalul de inferență în timp real prin WebSocket–TCP și baza de date time-series InfluxDB.

Structura lucrării

Lucrarea este organizată în opt capitole, după cum urmează:

Capitolul 1 – Formularea problemei prezintă grupul țintă vizat de aplicație și datele de intrare-ieșire.

Încadrarea în literatura de specialitate trece în revistă cele mai relevante rezultate: capacitatea rețelelor bazate pe spike-uri neuronale de a integra date multimodale, de a utiliza dinamica mișcărilor faciale pentru a detecta cu succes persoane deghizate, robustețea și puterea de generalizare când antrenarea și testarea au loc pe seturi diferite de date, respectiv consumul redus de energie.

Capitolul 2 – Fundamente teoretice prezintă baza matematică: tranziția de la ANN la SNN, modelele de neuron (Hodgkin-Huxley, Integrate-and-Fire), schemele de codificare (Rate, Latency, Rank Order) și mecanismele de plasticitate sinaptică (STDP).

Capitolul 3 – Experimente și analiza rezultatelor constituie nucleul experimental. Include detaliile setului de date CK+, metodologia 10-fold cross-validation și rezultatele comparative (SNN: 48.02% vs. CNN-LSTM: 88.37%).

Capitolul 4 - Neuro-Guardian prezintă aplicația construită.

Capitolul 5 – Concluzii sintetizează contribuțiile originale și direcțiile de cercetare viitoare.

Capitolul 1

Motivația și formularea problemei

1.1 Contextul general și relevanța problemei

Emoțiile reprezintă un canal fundamental de comunicare umană, dinamica lor fiind intens studiată în cadrul psihologiei sociale și a comunicării. Cercetările de pionierat realizate de Mehrabian et al. [MW67, MF67] demonstrează că, în contextul transmiterii mesajelor afective (atitudini și sentimente), în special atunci când apar neconcordanțe între canalele de comunicare, informația este decodificată asimetric de către interlocutor. Mai exact, semnificația totală a mesajului este determinată în proporție de 55% de expresiile faciale, 38% de indicii vocali (tonul, ritmul și volumul vocii) și doar 7% de cuvintele propriu-zise [Meh71]. Astfel, indicii nonverbali cumulează 93% din impactul total în evaluarea atitudinii emoționale. Cu toate acestea, sistemele informatice convenționale tratează interacțiunea cu utilizatorul aproape exclusiv prin conținut verbal sau textual, ignorând această dimensiune esențială a comunicării.

Deficitul devine critic în contextele în care persoana nu poate sau nu dorește să verbalizeze starea afectivă:

- Pacienții cu boli neurodegenerative (demență, Alzheimer);
- Persoanele cu tulburări din spectrul autist;
- Pacienții sedați sau postoperator;
- Copiii mici sau persoanele care nu vorbesc limba personalului medical.

În toate aceste situații, un sistem capabil să detecteze automat și în timp real stările emoționale exclusiv pe baza expresiei faciale ar constitui un instrument clinic de valoare considerabilă, permițând personalului medical să reacționeze prompt la semnale de disconfort, durere sau anxietate care altfel ar trece neobservate.

Dincolo de contextul clinic, aplicațiile recunoașterii automate a emoțiilor faciale (*Facial Emotion Recognition* - FER) acoperă un spectru larg: monitorizarea atenției șoferilor, sisteme de tutoriat inteligent, analize de marketing și interfețe om-calculator adaptive pentru realitate virtuală.

1.2 Formularea problemei

Problema abordată în prezenta lucrare poate fi enunțată formal astfel: dat fiind un flux video al feței unui subiect uman, să se construiască un sistem automat care mapează fiecare secvență de cadre la una dintre cele șapte categorii emoționale fundamentale definite în modelul extins al lui Ekman [EF86, Ekm92]: *Anger, Contempt, Disgust, Fear, Happiness, Sadness* și *Surprise*.

Intrarea sistemului: O secvență de cadre video (în cazul de față, câte 10 cadre extrase din setul de date CK+), care surprind evoluția temporală a expresiei faciale de la starea neutră până la apexul emoției.

Ieșirea sistemului: Un vector de probabilități al emoțiilor. În cadrul aplicației **NeuroGuardian**, aceste date sunt agregate temporal pentru calculul scorurilor de *suferință* și *durere*.

Procesul intern investigat în cazul rețelelor neuronale bazate pe spike-uri este un pipeline în trei etape, ilustrat schematic în Figura 1.1:

1. **Preprocesarea adaptivă:** eșantionarea centrată pe repere faciale;
2. **Codificarea neurală:** Conversia în trenuri de impulsuri prin *latency coding* și filtrare On/Off;
3. **Procesarea SNN:** Extragerea de trăsături prin straturi convoluționale spiking (CSNN) antrenate prin STDP și clasificarea finală.

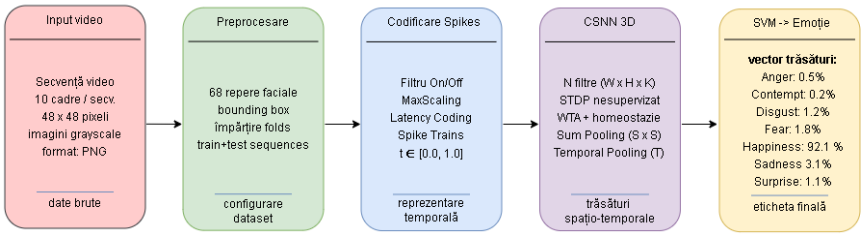


Figura 1.1: Schema bloc a sistemului propus: de la fluxul video la clasificarea emoțiilor prin SNN.

1.3 Dificultatea recunoașterii emoțiilor

Spre deosebire de recunoașterea obiectelor statice, FER se confruntă cu provocări unice:

- **Variabilitatea intra-clasă ridicată:** Același individ poate exprima fericirea în moduri vizual diferite (zâmbet larg vs. ridicarea subtilă a pomeților).
- **Similitudinea inter-clasă:** Perechi precum *Fear* și *Surprise* partajează elemente vizuale comune (ochi larg deschiși), diferențierea fiind posibilă doar prin dinamica temporală fină.
- **Dezechilibrul de clase:** Seturile de date academice (precum CK+) prezintă adesea o distribuție neuniformă (ex: 77 secvențe de *Surprise* vs. 12 de *Contempt*), ceea ce îngreunează antrenarea nesupervizată prin STDP.
- **Eficiența computațională:** Necesitatea procesării în timp real pentru aplicația Neuro-Guardian impune o latență redusă, limitând utilizarea modelelor *deep learning* extrem de masive.

Deși arhitecturile clasice (CNN-LSTM) ating acuratețe ridicată (de 88.37% în cadrul experimentelor realizate), ele implică un cost energetic substanțial prin înmulțiri matriciale continue.

Rețelele neuronale cu impulsuri adresează această ineficiență prin:

1. **Activare bazată pe evenimente:** Neuronii consumă energie doar atunci când generează un *spike*. În perioadele de imobilitate facială, activitatea rețelei este minimă.
2. **Filtrarea naturală a zgomotului:** Prin filtrarea On/Off, pixelii statici din fundal nu generează impulsuri, concentrând calculul exclusiv pe zonele cu dinamică relevantă.
3. **Codificarea temporală (*Latency Coding*):** Trăsăturile proeminente produc impulsuri timpurii, creând o ierarhie temporală pe care regula STDP o exploatează pentru a învăța trăsături spațio-temporale fără a avea nevoie de etichete în faza de antrenare.

Argumentul central al lucrării este că un pipeline adaptiv care izolează *Regiunea de Interes* (ROI) permite unui CSNN să extragă trăsături suficient de puternice pentru a depăși metodele liniare clasice, oferind o alternativă eficientă energetic pentru sistemele de *Edge AI* clinice.

Încadrarea în literatura de specialitate

Recunoașterea emoțiilor faciale folosind rețele SNN bio-inspirate

Lucrarea elaborată de Mansouri-Benssassi și Ye [MBY21] investighează aplicabilitatea rețelelor neuronale cu impulsuri (SNN) bio-inspirate în sarcini de recunoaștere a emoțiilor faciale (FER), punând un accent deosebit pe robustețea la zgomot și capacitatea de generalizare. Arhitectura propusă adaptează modelul Diehl și Cook [DC15a], utilizând învățarea nesupervizată prin plasticitate dependentă de timpul impulsurilor (STDP). Procesarea imaginilor implică utilizarea filtrului Laplacian of Gaussian (LoG) pentru extracția eficientă a marginilor, urmată de o codificare de tip *rate coding* prin trenuri de impulsuri Poisson, unde intensitatea pixelilor dictează rata de declanșare a neuronilor de tip Leaky Integrate-and-Fire (LIF). Modelul este structurat pe trei straturi: unul de intrare, un strat excitator (cu funcții de convoluție) și un strat inhibitor pentru selecția neuronului activ. Evaluările realizate pe seturile de date JAFFE și CK+ relevă faptul că SNN-ul obține performanțe comparabile cu metodele HOG+SVM și superioare arhitecturilor CNN pe seturi de date mici. Un aspect critic evidențiat este superioritatea SNN în experimentele de generalizare *cross-dataset* (atingând până la 92%) și reziliența remarcabilă la zgomot de tip *salt and pepper*; la un nivel de degradare de 0,5, SNN menține o acuratețe de 56,2%, depășind net modelele CNN (22,6%) și SVM (14,1%). Totuși, se identifică limitări privind utilizarea codificării pe bază de rată în detrimentul celei temporale și necesitatea rulării pe hardware neuromorfic dedicat pentru a maximiza eficiența energetică și computațională față de implementările actuale pe CPU/GPU.

Abordări bazate pe evenimente: Arhitectura Spiking-FER

O contribuție majoră în domeniu este adusă de Barchid et al. [BAA+23], care introduc conceptul de „Event-based FER" și propun primul benchmark de acest tip prin conversia seturilor de date video clasice (CK+, ADFES, CASIA și MMI) în fluxuri de evenimente folosind simulatorul V2E [HLD21]. Autorii propun *Spiking-FER*, o ar-

hitectură SNN convoluțională profundă bazată pe SEW-ResNet-18, care utilizează neuroni de tip *Integrate-and-Fire* (IF). Pentru a permite antrenarea de tip *end-to-end* și a depăși problema non-diferențiabilității funcției de prag, se utilizează *Surrogate Gradient Learning* cu o aproximare bazată pe funcția arctan. Spre deosebire de metodele modulare, abordarea *end-to-end* permite optimizarea simultană a extractorului de trăsături și a clasificatorului. Rezultatele experimentale indică faptul că *Spiking-FER* atinge performanțe comparabile cu modelele ANN, însă cu o eficiență energetică net superioară, fiind de până la 65,39 ori mai eficient din punct de vedere computațional. Studiul analizează, de asemenea, impactul tehnicilor de augmentare a datelor specifice evenimentelor (*Event Data Augmentation*), concluzionând că metodele *EventDrop* și *Mirror* sunt esențiale pentru reglarea antrenării pe seturi de date limitate. Totuși, o limitare importantă a studiului rămâne dependența de date simulate în locul senzorilor DVS (*Dynamic Vision Sensor*) reali și utilizarea bazelor de date de laborator, care pot să nu reflecte complexitatea emoțiilor spontane din mediul real.

Fuziunea multimodală și arhitecturi evolutive: Modelul FusionSense

Lucrarea prezentată de Tan et al. [TCKPS20] introduce *FusionSense*, primul sistem de recunoaștere a emoțiilor bazat pe rețele neuronale cu impulsuri care utilizează fuziunea datelor multimodale (expresii faciale corelate cu semnale fiziologice precum ECG, conductanța pielii și respirația). Autorii utilizează cadrul *NeuCube* [Kas14], o arhitectură SNN evolutivă (eSNN) care încorporează o structură spațială 3D de tip *reservoir* (3D-SNNr) cu conectivitate *small-world*. Pipeline-ul vizual utilizează algoritmul Viola-Jones pentru detecție și metoda Kanade-Lucas-Tomasi (KLT) pentru urmărirea punctelor de interes, extrăgând distanțe relative între 68 de repere faciale (*landmarks*) raportate la o stare neutră de referință. Un aspect metodologic distinct este utilizarea codificării de tip *population coding* bazată pe câmpuri receptive Gaussiene, urmată de o învățare nesupervizată prin STDP și o etapă supervizată ce folosește reguli de tip *Rank Order* (deSNN). Experimentele pe setul de date MAHNOB-HCI demonstrează că fuziunea la nivel de trăsături (*feature-level fusion*) este superioară celei la nivel de decizie, atingând o acuratețe de 73,15% pentru valența binară. Sistemul se remarcă prin capacitatea de antrenare incrementală (*one-pass*), fiind optim pentru aplicații *real-time*. Totuși, studiul prezintă limitări legate de excluderea semnalelor EEG și de dependența de o fereastră temporală fixă de 30 de secunde, care poate omite dinamica emoțională pe termen lung.

Recunoașterea identității prin dinamica mișcărilor faciale

Lucrarea propusă de Liu et al. [LBY20] introduce o paradigmă inovatoare de recunoaștere facială bazată pe dinamica mișcărilor musculare, metodă eficientă chiar și în scenarii cu fețe deghizate sau parțial ocultate. Spre deosebire de sistemele convenționale care se bazează pe trăsături spațiale statice, acest model utilizează cadre diferențiale pentru a captura micro-mișcările unice induse de vorbire. Arhitectura este de tip SNN profund, structurată pe cinci straturi care simulează calea ventrală a sistemului vizual biologic, incluzând module de extracție a trăsăturilor de nivel înalt prin filtre Gabor și codificare temporală bazată pe *Rank Order Coding* (ROC). Un element central al modelului este stratul de învățare care utilizează mecanismul *Soft Winner-Take-All* (WTA) cu inhibiție laterală și reguli STDP *event-driven*, asigurând selectivitatea neuronilor pentru tipare motorii specifice. Evaluările pe setul de date MakeFaceDB au demonstrat rate de succes între 95% și 100%, depășind modelele CNN echivalente, în special în scenariile cu deghizări complexe (măști, barbă falsă). Autorii subliniază eficiența procesului de convergență, care necesită doar două clipuri video pentru antrenare. Totuși, sunt identificate limitări practice importante: dependența de mișcarea activă a subiectului (necesitatea de a vorbi), sensibilitatea ridicată la mișcările de fundal sau ale camerei și necesitatea unei reglări manuale riguroase a hiperparametrilor pentru a menține stabilitatea rețelei.

Recunoașterea emoțiilor prin senzori de viziune dinamică simulați

Lucrarea propusă de Wang et al. [WDZ+22] abordează deficitul de seturi de date pentru senzori de viziune dinamică (DVS) în recunoașterea emoțiilor prin propunerea unei metode de generare a datelor simulate din segmente video convenționale (setul CMU-MOSI [ZZPM16]). Autorii introduc un cadru SNN bazat pe modelul de neuron *Parametric Leaky Integrate-and-Fire* (PLIF), care permite optimizarea automată a parametrilor de scurgere (*leak*) în timpul antrenării. O contribuție metodologică importantă este utilizarea unei prime rame de tip *float-value* pentru a stabiliza potențialul de membrană, urmată de trenuri de impulsuri binare, strategie care accelerează scăderea funcției de pierdere și îmbunătățește acuratețea față de modelele care folosesc exclusiv *spikes*. Arhitectura este compusă dintr-un modul de extracție a trăsăturilor cu straturi convoluționale și *average pooling*, un grup de neuroni de votare și un modul de mapare emoțională care traduce ratele de declanșare în etichete de polaritate. Antrenarea se realizează prin gradient surogat și *backpropagation thro-*

ugh time (BPTT). Rezultatele experimentale demonstrează că modelul SNN reduce consumul de energie cu 75% față de echivalentul ANN, atingând performanțe optime la un prag de tensiune de 0,4. Totuși, datele simulate nu pot reproduce fidel rezoluția temporală fină a unui senzor DVS real, iar constrângerea ferestrei temporale la 68 de cadre poate duce la pierderea informațiilor în cazul secvențelor video de lungimi inegale.

Capitolul 2

Fundamente teoretice despre inteligența artificială și rețele neuromorfice

2.1 Modele neuronale artificiale de a doua generație

Prima parte a capitolului este dedicată analizei fundamentelor teoretice care stau la baza modelelor computaționale de a doua generație utilizate în prezenta lucrare. Se va realiza o trecere în revistă a evoluției de la algoritmii clasici de învățare automată până la arhitecturile complexe de învățare profundă, evidențiind principiile lor de funcționare, avantajele structurale și limitările matematice care au condus la necesitatea dezvoltării rețelelor neuromorfice.

2.1.1 Învățarea automată și rețele neuronale artificiale (ANN)

Învățarea automată (*Machine Learning*) constituie pilonul central al inteligenței artificiale contemporane, oferind sistemelor computaționale capacitatea de a identifica tipare complexe în seturi de date fără a necesita o programare algoritmică explicită pentru fiecare regulă de decizie. În centrul acestei paradigme se află rețelele neuronale artificiale (ANN), arhitecturi proiectate ca o abstractizare a rețelelor neuronale biologice, însă simplificate structural pentru a fi compatibile cu hardware-ul convențional.

Funcționarea unei rețele neuronale standard se bazează pe transmiterea informației printr-o ierarhie de straturi compuse din unități de procesare elementare. Fiecare neuron artificial realizează o transformare a datelor de intrare în două etape succesive. În prima fază, se calculează o combinație liniară a semnalelor de intrare, unde fiecare contribuție este ajustată prin intermediul unor coeficienți numiți ponderi (sau greutăți). Rolul acestora este de a amplifica sau atenua importanța fiecărui semnal

în procesul decizional. Matematic, această sumă ponderată, la care se adaugă un termen de polarizare (*bias*), este exprimată prin relația (2.1):

$$z = \sum_{i=1}^{n} w_i x_i + b, \tag{2.1}$$

unde x_i reprezintă intrările sistemului, w_i sunt ponderile asociate, iar b este termenul de *bias*. În a doua etapă, pentru a introduce neliniaritatea necesară modelării unor fenomene complexe, rezultatul z este procesat de o funcție de activare σ. Valoarea finală de ieșire a neuronului, y, este definită de ecuația (2.2), procesul fiind ilustrat schematic în Figura 2.1:

$$y = \sigma(z) = \sigma\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{2.2}$$

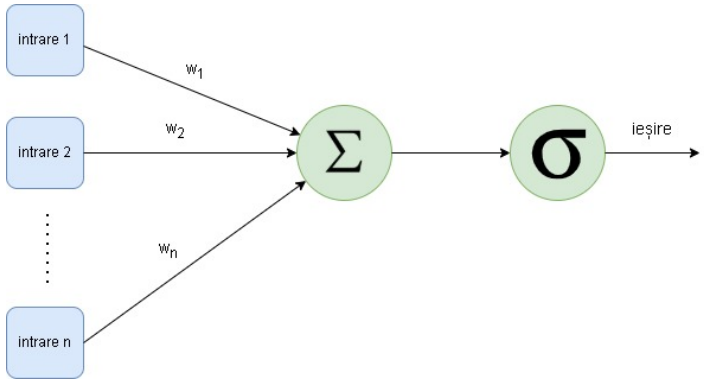


Figura 2.1: Reprezentarea schematică a unui neuron artificial: procesarea intrărilor prin ponderare și aplicarea funcției de activare.

Deși aceste rețele, integrate în ceea ce comunitatea științifică numește „a doua generație" de modele neuronale, au demonstrat performanțe remarcabile, ele prezintă limitări inerente. Utilizarea valorilor numerice continue, conform modelului descris în ecuația (2.2), forțează sistemul să realizeze calcule intensive în mod constant. Acest aspect conduce la un cost computațional ridicat și la un consum energetic substanțial, care rămâne invariant indiferent de relevanța sau densitatea informației procesate.

2.1.2 Rețele Neuronale Convoluționale (CNN)

Rețelele neuronale convoluționale (*Convolutional Neural Networks* – CNN) reprezintă o categorie specializată de arhitecturi ANN, proiectate explicit pentru procesarea datelor cu structură spațială, cum ar fi imaginile sau secvențele video. Spre deosebire de rețelele complet conectate (*fully connected*), care tratează fiecare pixel ca o intrare independentă, CNN-urile exploatează corelațiile locale dintre pixelii vecini

prin intermediul operației de convoluție, reducând drastic numărul de parametri și conferind invarianță la translație.

Operația de convoluție constituie nucleul funcțional al acestor arhitecturi. Un filtru (*kernel*) de dimensiune redusă $k \times k$ este aplicat glisant pe întreaga suprafață a imaginii de intrare, calculând la fiecare poziție (x, y) produsul scalar dintre valorile locale și ponderile filtrului, după cum reiese din ecuația 2.3:

$$\text{FeatureMap}(x, y) = \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} I(x+i, y+j) \cdot W(i, j) + b \tag{2.3}$$

unde I reprezintă imaginea de intrare, W este matricea de ponderi a filtrului, iar b este termenul de *bias*. Rezultatul acestei operații este o **hartă de trăsături** (*feature map*), care codifică prezența unui anumit tipar vizual (de exemplu, o margine orizontală sau un colț) în diverse regiuni ale imaginii.

O arhitectură CNN tipică este compusă din mai multe tipuri de straturi:

- **Straturi convoluționale**: Aplică un set de filtre antrenabile, generând hărți de trăsături pentru tipare de complexitate crescândă pe măsura adâncirii rețelei.
- **Straturi de pooling**: Reduc rezoluția spațială a hărților de trăsături prin agregarea valorilor dintr-o vecinătate locală (ex. *MaxPooling*, *AvgPooling*), conferind robustețe la mici variații de poziție.
- **Straturi de activare**: Introduc neliniaritatea necesară, funcția *ReLU* (*Rectified Linear Unit*), $\sigma(z) = \max(0, z)$, fiind cea mai frecvent utilizată în arhitecturile moderne datorită eficienței sale computaționale și comportamentului stabil la antrenare.
- **Straturi complet conectate (Fully Connected)**: Situate la finalul rețelei, realizează maparea vectorului de trăsături extrase către spațiul claselor de ieșire.

Antrenarea CNN-urilor se realizează prin algoritmul de *backpropagation*, care propagă eroarea de la stratul de ieșire către straturile anterioare și actualizează ponderile filtrelor prin metoda gradientului descendent. Gradul de expresivitate al acestor rețele crește exponențial cu adâncimea lor, arhitecturi profunde precum ResNet [HZRS16] sau VGG [SZ15] reușind să extragă trăsături semantice de nivel înalt.

ResNet18, utilizat ca extractor de trăsături spațiale în arhitectura hibridă CNN-LSTM din cadrul acestei lucrări, introduce mecanismul *residual connections* (conexiuni reziduale), prin care ieșirea unui bloc de straturi este adunată cu intrarea sa originală în ecuația 2.4:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \tag{2.4}$$

Această abordare rezolvă problema dispariției gradientului (*vanishing gradient*) în rețelele adânci, permițând antrenarea eficientă a arhitecturilor cu zeci sau sute de

straturi.

2.1.3 Rețele Recurente și LSTM

Rețelele neuronale complet conectate și CNN-urile tratează fiecare intrare ca independentă, ignorând orice dependență temporală sau secvențială. Pentru sarcinile în care ordinea elementelor este esențială – recunoașterea vorbirii, analiza seriilor de timp sau recunoașterea expresiilor faciale dintr-o secvență video – această limitare devine critică.

Rețelele neuronale recurente (*Recurrent Neural Networks* – RNN) adresează această problemă prin introducerea unei bucle de feedback: ieșirea neuronului la pasul temporal t este transmisă ca intrare suplimentară la pasul $t + 1$, permițând rețelei să mențină o **stare ascunsă** (*hidden state*) $\mathbf{h}_t$ care codifică informația acumulată din pașii anteriori. după cum se poate observa în ecuația 2.5:

$$\mathbf{h}_t = \sigma\left(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + b\right) \tag{2.5}$$

Cu toate acestea, RNN-urile clasice suferă de problema dispariției sau explozia gradientului în secvențe lungi: gradientul propagat înapoi prin multe etape temporale tinde fie să se anuleze (împiedicând învățarea dependențelor pe termen lung), fie să crească exponențial (destabilizând antrenamentul).

Rețelele LSTM (*Long Short-Term Memory*), introduse de Hochreiter și Schmidhuber [HS97], rezolvă această problemă printr-un mecanism explicit de **memorie gestionată prin porți** (*gated memory*). Unitatea LSTM menține două stări: starea celulei $\mathbf{c}_t$ (memoria pe termen lung) și starea ascunsă $\mathbf{h}_t$ (memoria pe termen scurt), actualizate conform componentelor descrise în Tabelul 2.1.

Tabela 2.1: Componentele structurale ale unei unități LSTM și ecuațiile lor matematice.

Componentă	Rol funcțional	Ecuație de calcul
Poarta de uitare	Decide ce informație din starea anterioară a celulei trebuie eliminată.	$\mathbf{f}_t = \sigma\left(W_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + b_f\right)$
Poarta de intrare	Determină ce proporție din informația nouă va fi transmisă mai departe.	$\mathbf{i}_t = \sigma\left(W_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + b_i\right)$
Starea candidată	Creează un vector cu noile valori potențiale ce pot fi adăugate în starea celulei.	$\tilde{\mathbf{c}}_t = \tanh\left(W_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + b_c\right)$
Poarta de ieșire	Determină ce parte din starea celulei este propagată ca ieșire curentă.	$\mathbf{o}_t = \sigma\left(W_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + b_o\right)$

Actualizarea stărilor se realizează conform relațiilor 2.6:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \tag{2.6}$$

unde $\odot$ denotă produsul element-cu-element (Hadamard). Prin această arhitectură, LSTM-ul poate reține în mod selectiv informații relevante pe durate arbitrar de lungi, ignorând zgomotul temporal.

În contextul recunoașterii expresiilor faciale, LSTM-ul este deosebit de util pentru modelarea dinamicii temporale a expresiei: trecerea de la o față neutră la apexul emoției implică o secvență specifică de mișcări musculare care nu poate fi surprinsă dintr-un singur cadru izolat.

2.1.4 Mașini cu Vectori Suport (SVM)

Mașinile cu vectori suport (*Support Vector Machines* – SVM) reprezintă o clasă de algoritmi de clasificare supervizată bazată pe principii ale teoriei statistice a învățării (*Statistical Learning Theory*), formalizată de Vapnik și Chervonenkis [Vap95]. Spre deosebire de rețelele neuronale, SVM-ul nu necesită o arhitectură adâncă și se comportă eficient inclusiv în spații de trăsături de dimensionalitate ridicată, cu seturi de date relativ mici.

Principiul fundamental al SVM constă în identificarea unui **hiperplan de separare optim** în spațiul trăsăturilor, definit ca hiperplanul care maximizează **marginea** (*margin*) dintre clasele de date. Formal, pentru un set de date $\{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, cu $y_i \in \{-1, +1\}$, se caută vectorul de ponderi $\mathbf{w}$ și pragul b care satisfac 2.7:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{a.î.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall i = 1, \ldots, N. \tag{2.7}$$

Vectorii de antrenare situați pe granița marginii (la distanța minimă față de hiperplan) poartă denumirea de vectori suport (*support vectors*) și sunt singurii care influențează poziția hiperplanului de decizie, conferind modelului robustețe la *outlieri*.

Extensia pentru date neseparabile liniar se realizează prin introducerea variabilelor de relaxare (*slack variables*) $\xi_i \geq 0$ și a parametrului de regularizare C, care controlează în ecuația 2.8 compromisul dintre maximizarea marginii și toleranța la erori de clasificare (*soft-margin SVM*):

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N} \xi_i \tag{2.8}$$

Trucul kernel (*kernel trick*) permite SVM-ului să opereze implicit în spații de trăsături de dimensionalitate foarte mare (sau infinită), fără a calcula explicit transformarea. Prin înlocuirea produsului scalar cu o funcție kernel $K(\mathbf{x}_i, \mathbf{x}_j)$, se pot separa date care nu sunt liniar separabile în spațiul original. Cele mai frecvent utilizate funcții kernel includ:

- Kernel liniar: $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$
- **Kernel RBF** (*Radial Basis Function*): $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)$

Extensia la clasificare multiclasă (necesară pentru cele 7 emoții din setul CK+) se realizează prin strategii de tip *One-vs-One* (un clasificator binar pentru fiecare pereche de clase) sau *One-vs-Rest* (un clasificator per clasă față de restul).

În cadrul lucrării de față, SVM-ul este utilizat în două ipostaze distincte: ca model de referință *baseline* aplicat direct pe pixeli bruți (fără extracție de trăsături), și ca clasificator final în pipeline-ul propus, primind ca intrare vectorii de trăsături spațio-temporale extrași de stratul convoluțional SNN prin operațiile de *SumPooling* și *TemporalPooling*. Această separare între etapa de extracție a trăsăturilor (nesupervizată, prin STDP) și etapa de clasificare (supervizată, prin SVM) este o abordare consacrată în literatura de specialitate, care permite evaluarea izolată a calității reprezentărilor învățate de rețeaua neuronală cu impulsuri.

2.2 Trecerea de la ANN la Spiking Neural Networks (SNN)

Limitările energetice menționate anterior au impus tranziția către o nouă paradigmă: rețelele neuronale cu impulsuri (*Spiking Neural Networks* - SNN). Considerate a treia generație a rețelelor neuronale, SNN-urile se apropie mult mai mult de mecanismele fiziologice ale creierului, înlocuind valorile continue cu impulsuri discrete de joasă energie, numite *spikes* [TGK+19].

Spre deosebire de modelele clasice, SNN-urile adoptă o abordare bazată pe evenimente (*event-driven*), caracterizată printr-o activare rară (*sparse activation*), așa cum se poate observa în Figura 2.2. În acest context, neuronii procesează evenimente distribuite temporal, activându-se doar atunci când stimulii recepționați ating un anumit prag critic. Această proprietate elimină necesitatea calculelor redundante în perioadele de inactivitate, optimizând consumul de resurse [FTB+19]. Mai mult, natura asincronă a acestor rețele facilitează paralelizarea proceselor, oferind avantaje clare de viteză pe hardware neuromorfic dedicat [BBJ+19].

2.3 Modelarea matematică a neuronului cu impulsuri

Spre deosebire de modelele generațiilor anterioare, rețelele neuronale cu impulsuri (SNN) utilizează ca unitate fundamentală de comunicare fluxuri de impulsuri discrete, cunoscute sub denumirea de *spike trains*. Fiecare impuls reprezintă un

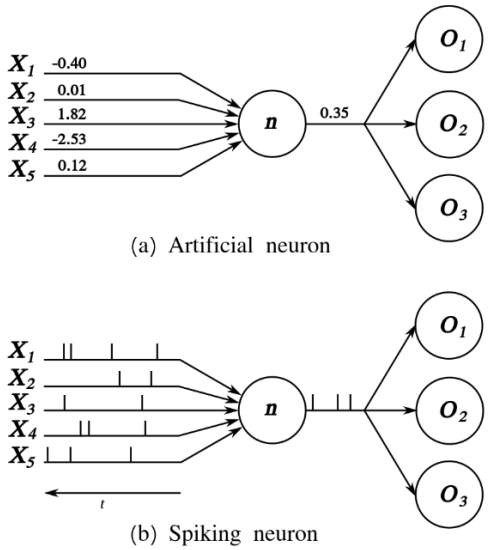


Figura 2.2: Comparație între un neuron artificial și un neuron cu impulsuri (*spiking neuron*). Deși structura de bază este comună în cadrul ANN-urilor și SNN-urilor, funcționarea diferă: (a) un neuron artificial primește valori numerice continue pentru a calcula o valoare numerică de ieșire, în timp ce (b) un neuron cu impulsuri primește evenimente discrete și parsimonioase (numite impulsuri sau *spikes*) și generează o secvență de impulsuri de ieșire. Sursă: [EA24].

potențial de acțiune, o descărcare electrică de scurtă durată care transportă informația prin rețea, imitând fidel procesele electro-chimice din creierul uman.

Mecanismul de funcționare al unui neuron de tip *spiking* este definit prin interacțiunea dintre potențialul de membrană și un prag de activare (*firing threshold*). Procesul poate fi sintetizat în patru etape critice:

- **Integrarea și acumularea:** Neuronul recepționează semnale de intrare prin sinapse și le cumulează în timp, ceea ce determină o creștere incrementală a potențialului de membrană.
- **Declanșarea (*Firing*):** În momentul în care potențialul acumulat atinge sau depășește valoarea pragului critic stabilit, neuronul generează un impuls de ieșire (*spike*).
- **Resetarea:** Imediat după generarea impulsului, potențialul de membrană revine la o valoare de bază, numită stare de repaus (*resting state*).
- **Perioada refractară:** Urmează un scurt interval de timp în care neuronul devine insensibil la noile impulsuri primite, indiferent de intensitatea acestora, permițând refacerea echilibrului ionic.

Evoluția temporală a acestor etape și modul în care potențialul de membrană reacționează la stimulii externi sunt ilustrate în Figura 2.3.

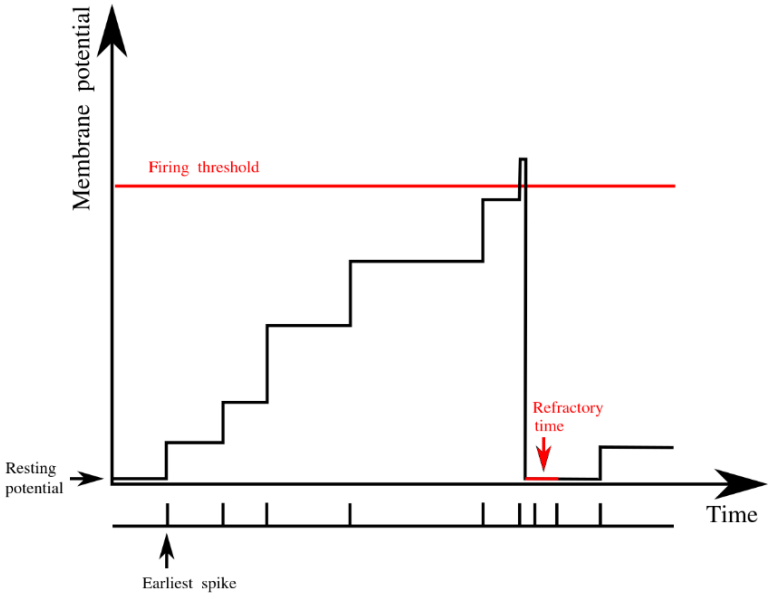


Figura 2.3: Evoluția potențialului de membrană al unui neuron de tip *spiking* ca răspuns la un tren de impulsuri de intrare. Se observă fazele de acumulare, atingerea pragului critic, generarea impulsului și perioada de resetare. Sursă: [EA24].

Modele de neuroni: de la biologie la eficiență hardware

În literatura de specialitate, există o varietate de modele matematice care descriu acest comportament, prezentând un compromis direct între plauzibilitatea biologică și complexitatea computațională [Izh04].

1. **Modelul Hodgkin-Huxley [HH52]:** Este unul dintre cele mai detaliate modele biofizice, utilizând un sistem de ecuații diferențiale neliniare pentru a descrie dinamica canalelor ionice (sodiu și potasiu). Deși oferă o acuratețe biologică ridicată, costul computațional necesar pentru simularea acestuia îl face extrem de dificil de implementat în sisteme hardware neuromorfice la scară largă.

2. **Modelul Integrate-and-Fire (IF) [Bur06]:** Reprezintă varianta cea mai simplificată și eficientă din punct de vedere algoritmic. În ciuda arhitecturii sale elementare, modelul IF s-a dovedit a fi un instrument robust în aplicațiile de învățare vizuală și recunoaștere a formelor, fiind preferat în designul sistemelor de tip *edge AI*.

Modelul IF descrie evoluția potențialului de membrană $V_m(t)$ ca o acumulare liniară a ponderilor sinaptice asociate impulsurilor recepționate. Dinamica sistemului, guvernată de capacitatea membranei c_m, este exprimată prin relația (2.9):

$$c_m V_m(t) = V_r + \sum_{i \in E} w_i f_s(t - t_i) \tag{2.9}$$

unde:

- V_r reprezintă potențialul de repaus al membranei;
- E constituie setul de impulsuri de intrare;
- w_i este ponderea sinaptică (greutatea) asociată celui de-al i-lea impuls;
- t_i reprezintă coordonata temporală a recepționării impulsului;
- f_s este funcția nucleu (*spike kernel*), definită conform ecuației (2.10):

$$f_s(x) = \begin{cases} 1, & \text{dacă } x \geq 0 \\ 0, & \text{în caz contrar} \end{cases} \tag{2.10}$$

Regula de actualizare a stării neuronului impune ca atunci când potențialul de membrană depășește pragul V_{th}, să aibă loc descărcarea, urmată de resetarea potențialului, așa cum reiese din ecuația 2.11:

$$V_m(t) \leftarrow V_r \quad \text{pentru} \quad V_m(t) \geq V_{th}(t) \tag{2.11}$$

Această abordare matematică permite procesarea eficientă a datelor asincrone, fiind ideală pentru sarcini care implică o dinamică temporală complexă, cum este identificarea emoțiilor în secvențe video.

2.4 Codificarea informației neurale (Neural Coding)

Întrucât rețelele neuronale cu impulsuri procesează informația sub formă de evenimente discrete, transformarea datelor de intrare (semnale analogice sau valori digitale continue) în trenuri de impulsuri (*spike trains*) reprezintă o etapă fundamentală. În contextul prezentei lucrări, datele de intrare sunt prezentate neuronilor secvențial, eșantion cu eșantion. Fiecare stimul este menținut pentru un interval temporal definit, cunoscut sub numele de perioadă de expunere sau $t_{exposition}$ [Fal19].

Deși există numeroase scheme de codificare, acestea pot fi clasificate în două categorii majore, în funcție de modul în care informația este reprezentată în raport cu timpul.

Rate Coding

Codificarea prin rată corelează intensitatea stimulului de intrare $x \in [0, 1]$ cu frecvența de apariție a impulsurilor. Din punct de vedere computațional, acest proces este modelat printr-un proces stochastic Poisson [GK02]. Probabilitatea ca un neuron

i să emită un impuls ($S_i = 1$) într-un interval de timp discret foarte mic Δt este proporțională cu valoarea semnalului de intrare, aspect ilustrat în ecuația 2.12:

$$P(S_i[t] = 1) = x \cdot \Delta t \tag{2.12}$$

unde x reprezintă intensitatea stimulului (rata de descărcare dorită), Δt este pasul de simulare temporală, $S_i[t] \in \{0, 1\}$ este variabila Bernoulli care indică prezența unui impuls la momentul t.

Deși este robustă, codificarea prin rată prezintă limitări semnificative în ceea ce privește viteza de procesare. Pentru a obține o reprezentare precisă a valorii codificate, neuronii post-sinaptici trebuie să integreze un număr ridicat de impulsuri pe o durată extinsă [GFES21]. Acest fenomen introduce o latență inerentă în rețea, deoarece decizia este condiționată de acumularea unei statistici suficiente a impulsurilor.

Temporal Coding

Spre deosebire de codificarea prin rată, metodele temporale utilizează momentul precis al apariției impulsurilor (*timestamps*) pentru a reprezenta informația, oferind o eficiență energetică și temporală superioară.

Rank Order Coding

Codificarea prin ordinea rangului [TG98] se bazează pe premisa că ordinea în care neuronii dintr-o populație declanșează impulsuri poartă informația esențială [DPT01]. Neuronii care primesc stimuli de intensitate ridicată tind să se activeze înaintea celor stimulați slab. O formă simplificată de reprezentare a acestui impact, independentă de modelul de neuron utilizat, este redată de ecuația (2.13):

$$f_{in}(x_i) = r(x_i) \times \frac{1}{n}, \tag{2.13}$$

unde x_i este valoarea de intrare, n reprezintă numărul total de conexiuni, iar $r(x_i)$ reprezintă rangul (poziția în ordine descrescătoare) al valorii x_i în setul de intrări.

Latency Coding

Codificarea prin latență [GM08] reprezintă intensitatea stimulului prin intervalul de timp scurs până la primul impuls. Un stimul puternic va genera un răspuns rapid (latență mică), în timp ce un stimul slab va întârzia activarea neuronului. Relația de inversă proporționalitate între intensitatea x și momentul declanșării este formalizată prin ecuația (2.14):

$$f_{in}(x) = (1.0 - x) \times t_{exposition}, \tag{2.14}$$

Această abordare este deosebit de eficientă pentru procesarea video (HAR), deoarece permite rețelei să reacționeze extrem de rapid la schimbările semnificative din câmpul vizual, procesând prioritar cele mai relevante informații.

2.5 Mecanisme de învățare și plasticitate sinaptică

Capacitatea de procesare a informației în cadrul rețelelor SNN rezidă în conexiunile sinaptice dintre neuroni. **Plasticitatea sinaptică** reprezintă proprietatea fundamentală a acestor conexiuni de a-și modifica intensitatea (ponderea sinaptică) în funcție de tiparele activității neuronale și de regulile de învățare aplicate. Din punct de vedere funcțional, ponderile sinaptice modulează probabilitatea de declanșare a neuronului post-sinaptic: o pondere ridicată amplifică contribuția unui impuls recepționat, în timp ce o pondere scăzută atenuează impactul acestuia asupra potențialului de membrană.

Antrenarea acestor rețele presupune ajustarea sistematică a ponderilor și a pragurilor de activare pentru a permite sistemului să identifice trăsături relevante în datele de intrare. În literatura de specialitate, metodele de antrenare a SNN-urilor pot fi clasificate în trei paradigme majore.

Conversia ANN în SNN

Această abordare presupune antrenarea prealabilă a unei rețele neuronale artificiale (ANN) convenționale, utilizând algoritmi standard precum *Backpropagation*, urmată de transferul și adaptarea ponderilor învățate către o arhitectură SNN echivalentă [KPPC20].

Procesul de conversie urmărește conservarea funcționalității și a performanței rețelei originale prin scalarea ponderilor și ajustarea acestora la intervalul dinamic specific modelului de neuron cu impulsuri ales. Deși această metodă oferă rezultate competitive și permite utilizarea infrastructurii software existente pentru *Deep Learning*, ea prezintă limitări critice:

- Necesită resurse computaționale ridicate pentru etapa inițială de antrenare a ANN-ului.
- Implică procese post-conversie complexe, precum normalizarea ponderilor și echilibrarea pragurilor (*threshold balancing*).

- Nu exploatează pe deplin avantajele temporale native ale SNN-urilor în timpul procesului de învățare.

Spiking Back-propagation

O altă metodă supervizată constă în adaptarea directă a algoritmului *Backpropagation* pentru domeniul impulsurilor [Gru23]. Această tehnică optimizează ponderile prin calcularea unei funcții de cost bazată pe eroarea dintre ieșirea dorită și cea prezisă, eroarea fiind ulterior propagată de la stratul de ieșire către straturile anterioare.

Principala provocare a acestei metode rezidă în faptul că funcția de activare a unui neuron *spiking* (funcția Heaviside) este discontinuă și, prin urmare, nederivabilă [LDP16]. Acest aspect împiedică calculul direct al gradientului necesar optimizării. Pentru a depăși acest obstacol, cercetările recente propun:

- Utilizarea unor **gradienți surogat** (*surrogate gradients*) care aproximează derivata impulsului în timpul etapei de *backward pass* [TGK+19].
- Tratarea potențialului de membrană ca pe un semnal continuu și derivabil [LDP16].

Deși metoda este eficientă pentru obținerea unei precizii ridicate, ea rămâne costisitoare din punct de vedere computațional și necesită volume mari de date etichetate, ceea ce limitează aplicabilitatea sa în sistemele de tip *on-chip learning* cu resurse reduse [LPSR18].

Învățarea de tip Hebbian

Plasticitatea de tip Hebbian constituie un principiu fundamental în neuroștiințe, stipulând că intensitatea unei sinapse între doi neuroni este determinată de corelația activităților acestora. Conform celebrului postulat al lui Donald Hebb: „neuronii care se activează împreună, se conectează împreună" (*neurons that fire together, wire together*) [Heb02]. În domeniul SNN, învățarea Hebbiană este utilizată pe scară largă pentru antrenarea rețelelor în regim supervizat [LXJL22], nesupervizat [KGTM18] sau auto-supervizat [HZ23].

Mecanismul de bază presupune modificarea greutății sinaptice în funcție de corelația dintre activitatea pre-sinaptică (intrarea) și cea post-sinaptică (ieșirea). Deși există mai multe reguli de plasticitate, precum *Spike-Driven Synaptic Plasticity* (SDSP), majoritatea aplicațiilor contemporane se bazează pe **Spike Timing-Dependent Plasticity (STDP)**, datorită robusteții sale în procesarea semnalelor temporale.

Spike Timing-Dependent Plasticity (STDP)

STDP este o regulă de învățare cu înaltă plauzibilitate biologică [SMA00] ce ajustează ponderile sinaptice pe baza diferenței temporale relative dintre impulsurile pre-sinaptice și cele post-sinaptice. Această metodă este esențială pentru implementările pe hardware neuromorfic cu consum redus, fiind capabilă să extragă trăsături complexe fără necesitatea unei supervizări externe.

Principiul de funcționare al STDP se bazează pe două fenomene opuse, așa cum este observat experimental în mod sistematic în [BP98]:

- **Potențarea pe termen lung (Long-Term Potentiation - LTP):** [BL73] Dacă impulsul pre-sinaptic are loc cu puțin timp înaintea impulsului post-sinaptic, sinapsa este întărită. Acest lucru sugerează că intrarea a contribuit direct la activarea neuronului.
- **Depresia pe termen lung (Long-Term Depression - LTD):** [DB92] Dacă impulsul post-sinaptic are loc înaintea celui pre-sinaptic, sinapsa este slăbită. Aceasta indică faptul că semnalul de intrare nu a avut un rol cauzal în declanșarea impulsului de ieșire.

Această dinamică permite rețelei să prioritizeze conexiunile care transportă informație relevantă pentru modelul identificat, ignorând în același timp zgomotul sau semnalele necorelate temporal.

În funcție de complexitatea dorită și de constrângerile hardware, STDP poate fi implementată prin mai multe modele matematice:

1. STDP Biologic: Este forma de bază care utilizează o descreștere exponențială a actualizării ponderii în funcție de distanța temporală dintre impulsuri [BP98]. Formalizarea matematică este dată de ecuația (2.15):

$$\Delta w = \begin{cases} \eta_w e^{-\frac{t_{post}-t_{pre}}{\tau_{STDP}}}, & \text{dacă } t_{pre} \leq t_{post} \\ -\eta_w e^{-\frac{t_{pre}-t_{post}}{\tau_{STDP}}}, & \text{în caz contrar} \end{cases} \tag{2.15}$$

unde η_w reprezintă rata de învățare, iar τ_{STDP} este constanta de timp ce definește fereastra de învățare.

2. STDP Aditiv (Add-STDP): O variantă simplificată unde modificarea ponderii depinde doar de semnul diferenței temporale, nu și de magnitudine. Deoarece poate conduce la instabilitate, aceasta necesită limite stricte de control (*hard boundaries*), rezultând adesea în ponderi binare (0 sau 1) [SMA00], așa cum este ilustrat în ecuația 2.16:

$$\Delta w = \begin{cases} \eta_{w+}, & \text{dacă } \delta t \leq 0 \\ -\eta_{w-}, & \text{în caz contrar} \end{cases} \tag{2.16}$$

3. STDP Multiplicativ (Mult-STDP): Această variantă introduce dependența de valoarea curentă a ponderii, prevenind saturarea și oferind o distribuție mai fină a greutăților sinaptice, comparativ cu modelul aditiv [vRBT00]. Actualizarea surprinsă în ecuația 2.17 este reglată astfel încât ponderile mici să poată crește mai rapid, în timp ce ponderile mari sunt limitate superior:

$$\Delta w = \begin{cases} \eta_{w+} e^{-\beta \frac{w - w_{min}}{w_{max} - w_{min}}}, & \text{dacă } \delta t \leq 0 \\ -\eta_{w-} e^{-\beta \frac{w_{max} - w}{w_{max} - w_{min}}}, & \text{în caz contrar} \end{cases} \tag{2.17}$$

Inhibiția

Inhibiția reprezintă procesul de diminuare sau suprimare a activității neuronale, fiind un mecanism fundamental pentru reglarea dinamicii corticale [SM07]. Într-o rețea neuronală cu impulsuri (SNN), inhibiția joacă un rol vital în menținerea echilibrului dintre mecanismele excitatorii și cele inhibitorii, permițând un control precis asupra fluxului de informație și a momentelor de declanșare a impulsurilor [ZZL22].

Cea mai utilizată metodă de inhibiție în faza de antrenare este mecanismul *Winner-Take-All* (WTA) [QBG11, Che17]. Conform acestei reguli, primul neuron dintr-un strat care generează un impuls este declarat „câștigător" și transmite imediat un semnal inhibitor către ceilalți neuroni din același strat pentru a le bloca activitatea. Această strategie este deosebit de eficientă în învățarea nesupervizată bazată pe STDP, deoarece introduce o competiție între neuroni [FMT18]. Prin împiedicarea mai multor neuroni să răspundă simultan la același stimul, rețeaua este forțată să învețe trăsături distincte și mai puțin corelate, optimizând astfel capacitatea de reprezentare a sistemului.

Homeostazia

Deși mecanismul *Winner-Take-All* (WTA) este esențial pentru diversificarea procesului de învățare, acesta poate introduce un dezechilibru în care un număr restrâns de neuroni devin dominanți. Acești neuroni „câștigători" tind să se declanșeze repetat, monopolizând procesul de adaptare a ponderilor, în timp ce restul unităților rămân inactive. Pentru a evita această saturație și a menține stabilitatea sistemului, este necesară implementarea unor mecanisme de homeostazie prin adaptarea pragului de declanșare (*threshold adaptation*).

În cadrul acestei arhitecturi, se utilizează două metode complementare de adaptare a pragului. Prima și cea mai importantă este **Leaky Adaptive Threshold (LAT)** [DC15b], utilizată pentru a asigura o competiție echitabilă între neuroni. Cea de-a doua este metoda **adaptării pragului bazată pe un timp țintă** (*target timestamp*) [FTB+19], utilizată în conjuncție cu LAT în modelele de tip CSNN de ultimă generație [Fal19]. Aceasta din urmă forțează neuronii să tindă spre un moment de declanșare predefinit $\hat{t}$, permițând modelului să se specializeze pe anumite tipare temporale, contribuind în același timp la homeostazia rețelei.

Pragurile tuturor neuronilor dintr-un strat (atât pentru cel câștigător, cât și pentru cei care au primit semnal de inhibiție) sunt actualizate la fiecare declanșare a unui impuls sau la recepția unui semnal inhibitor. Din punct de vedere matematic, modificarea pragului se bazează pe două componente, Δ_{th}^1 și Δ_{th}^2, după cum reiese din ecuația 2.18:

$$\Delta_{th}^1 = -\eta_{th}(t - \hat{t}), \tag{2.18}$$

unde t este momentul declanșării impulsului, $\hat{t}$ este timpul țintă dorit, iar η_{th} este rata de învățare a pragului. Componenta LAT este definită în ecuația (2.19):

$$\Delta_{th}^2 = \begin{cases} \eta_{th}, & \text{dacă } t_i = \min\{t_0, \ldots, t_N\} \\ -\frac{\eta_{th}}{ld(n)}, & \text{în caz contrar} \end{cases}, \tag{2.19}$$

unde t_i reprezintă timpul de declanșare al neuronului i, $\min\{t_0, \ldots, t_N\}$ corespunde neuronului care a generat primul impuls (câștigătorul), iar $ld(n)$ reprezintă numărul de neuroni aflați în competiție în stratul respectiv. În final, actualizarea potențialului pragului de membrană V_{th} se realizează conform Equation (2.20):

$$V_{th}(t) = \max(th_{min}, V_{th}(t-1) + \Delta_{th}^1 + \Delta_{th}^2) \tag{2.20}$$

unde th_{min} reprezintă valoarea minimă admisă a pragului [Fal19], asigurând astfel că sensibilitatea neuronului nu crește peste o limită care ar putea genera activitate spontană cauzată de zgomot. Această abordare combinată asigură faptul că latența impulsurilor converge către valoarea țintă, menținând în același timp un echilibru între activitatea tuturor celor N neuroni din strat.

2.6 Arhitecturi Spiking Convolutional Neural Networks (CSNN)

Arhitecturile Spiking Convolutional Neural Networks (CSNN) combină capacitatea de extragere a trăsăturilor spațiale cu eficiența energetică a neuronilor de tip spiking.

Implementarea unui strat de convoluție în acest context presupune o metodologie structurată în două etape distincte: etapa de învățare a filtrelor și etapa de generare a hărților de trăsături (**feature maps**), după cum urmează:

1. **Etapa de învățare (Learning Phase):** În această fază, fiecare nucleu de convoluție (**kernel**) este asociat unui neuron specific (sau unui grup restrâns de neuroni). Scopul acestei etape este antrenarea ponderilor sinaptice W, adesea utilizând regula STDP, pentru a specializa nucleul în detectarea unui anumit tipar vizual.

2. **Etapa de testare/inferență (Inference Phase):** Odată ce ponderile au fost învățate, acestea sunt „înghețate" și replicate pe întreaga suprafață a intrării. În această etapă, se asociază neuroni virtuali pentru fiecare element din hărțile de trăsături rezultate. Esențial este faptul că toate unitățile dintr-o hartă de trăsături partajează același nucleu învățat anterior, asigurând invarianța la translație.

Convoluția în domeniul impulsurilor (2D-CSNN)

Generarea hărților de trăsături în etapa de inferență se realizează prin aplicarea glisantă a filtrului, unde dinamica potențialului de membrană pentru un neuron de tip *Integrate-and-Fire* (IF) de la coordonatele (x, y) este calculată conform ecuației (2.21):

$$V_{m_{x,y,k}}(t) = \sum_{i \in E} W_{n(x_i),j(y_i),k_i,k} \cdot f_s(t - t_i) \tag{2.21}$$

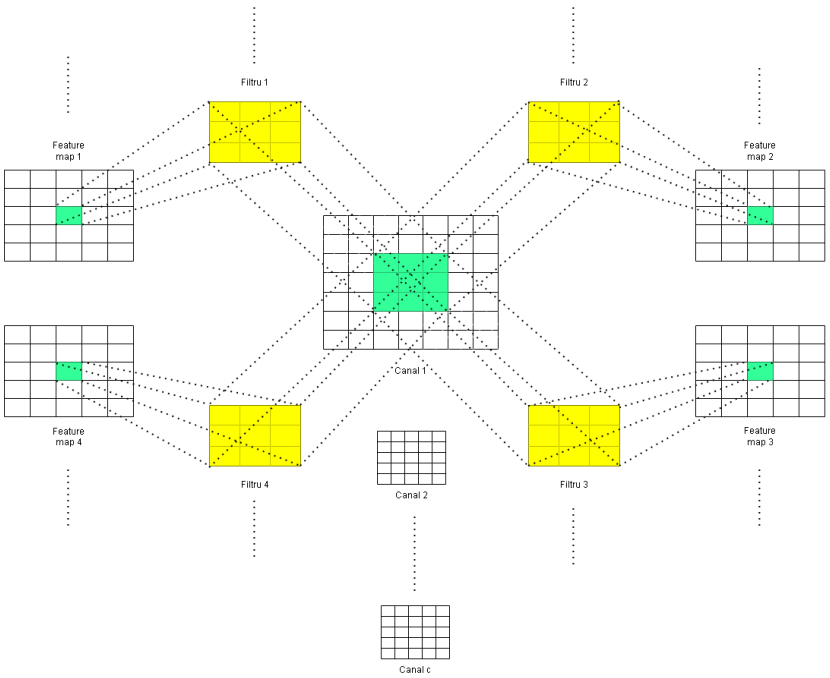


Figura 2.4: Reprezentarea procesului de convoluție 2D. În faza de testare, nucleul învățat este aplicat pe toată imaginea pentru a genera harta de trăsături.

3D-CSNN pentru procesare video

Extensia către 3D-CSNN urmează aceeași logică bimodală (învățare vs. testare), dar extinde domeniul de aplicare în dimensiunea temporală. Această arhitectură permite extragerea directă a trăsăturilor spatio-temporale, eliminând necesitatea unor algoritmi externi de calcul al fluxului optic [HS81].

În etapa de inferență 3D, nucleul glisant integrează informația dintr-un volum de intrare (lățime, înălțime și adâncime temporală). Toți neuronii dintr-un volum de ieșire k utilizează același set de ponderi 3D învățate, calculând potențialul de membrană la momentul t conform formulei 2.22:

$$V_{m_{x,y,z,k}}(t) = \sum_{i \in E} W_{n(x_i),j(y_i),h(z_i),k_i,k} \cdot f_s(t - t_i) \tag{2.22}$$

unde $h(z_i)$ mapează coordonata temporală a impulsului în fereastra nucleului 3D. Acest mecanism permite neuronilor să devină selectivi la schimbări dinamice, simulând comportamentul neuronilor din cortexul vizual primar care răspund la direcții specifice de mișcare în câmpul receptiv (Figura 2.5).

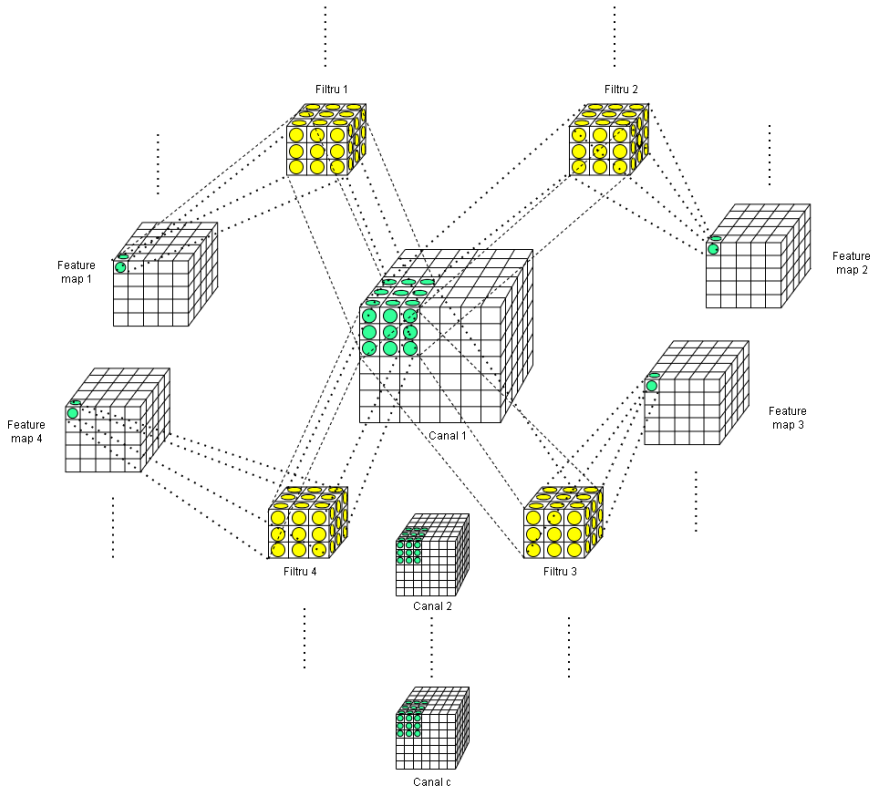


Figura 2.5: Operația de convoluție 3D. Aceasta procesează „cuburi" de evenimente, unde aceeași matrice de ponderi W este utilizată pentru a scana întregul flux video.

2.7 Algoritmi de preprocesare și detecție facială

Pentru a asigura o rată ridicată de recunoaștere a emoțiilor în cazul în care se dorește antrenarea pe un set de date pentru care nu sunt disponiible cele 68 de puncte faciale de referință asociate cadrelor, sistemul utilizează o etapă riguroasă de preprocesare. Aceasta are rolul de a extrage doar informația relevantă (fața subiectului) și de a menține alinierea acesteia pe parcursul întregii secvențe video, proces adaptat conform metodologiei descrise în [TCKPS20].

În primul cadru al secvenței video, sistemul nu deține informații despre locația subiectului. Pentru inițializare, se utilizează algoritmul **Viola-Jones** [VJ01], un standard în domeniu datorită eficienței sale computaționale.

Algoritmul funcționează prin scanarea imaginii cu ferestre glisante de dimensiuni variabile, căutând trăsături simple numite *Haar-like features*. Acestea se bazează pe diferențele de intensitate dintre regiuni adiacente (de exemplu, zona ochilor este de regulă mai întunecată decât pometii, iar puntea nasului este mai luminoasă decât zonele laterale). Rezultatul acestei etape este definirea unei **Regiuni de Interes (ROI)** sub forma unui dreptunghi care încadrează fața detectată.

Odată ce ROI a fost stabilit, procesarea întregii suprafețe a pielii în fiecare cadru ar fi ineficientă și predispusă la erori din cauza texturii uniforme. Din acest motiv, se trece la identificarea unor puncte de „ancoră".

Pentru a selecta cele mai robuste puncte pentru urmărire, se utilizează metoda *Min-Eigenvalue* (cunoscută și sub numele de detectorul Shi-Tomasi [ST94]). Acest algoritm identifică colțuri — puncte în care intensitatea pixelilor se schimbă brusc în două direcții ortogonale. Sunt selectate astfel puncte unice precum colțurile ochilor, vârful nasului sau extremitățile gurii, acestea fiind ușor de „agățat" și urmărit în cadrele subsecvente.

În loc să ruleze detecția Viola-Jones în fiecare cadru (proces costisitor), sistemul utilizează algoritmul KLT [LK81]. Acesta reprezintă „motorul" de mișcare al preprocesării, analizând fluxul optic (*optical flow*) pentru a determina unde s-au mutat punctele ancoră din cadrul curent în cadrul următor. KLT este optimizat pentru „micro-mișcări", fiind mult mai rapid decât o detecție de la zero.

Deoarece subiectul se poate mișca liber (rotații ale capului, aplecări sau schimbări de scară), simpla urmărire a punctelor nu este suficientă pentru a menține un cadru stabil pentru SNN.

Dacă punctele ancoră detectate la pasul anterior și-au schimbat coordonatele, dreptunghiul inițial trebuie actualizat, acest lucru având loc cu ajutorul matricii de transformare M (translație, rotație și scalare) care descrie mișcarea medie a tuturor punctelor. Această matrice este aplicată întregului poligon ROI, permițând „ramei" feței să sară exact pe noua poziție.

2.8 Metode de eșantionare a datelor de intrare (Sampling)

În cadrul arhitecturilor de tip Convolutional Spiking Neural Networks (CSNN), procesul de eșantionare (sampling) joacă un rol critic în eficiența procesării informației vizuale. Spre deosebire de rețelele neurale convoluționale clasice, unde întreaga imagine este procesată uniform, simulatoarele de SNN pot utiliza strategii de eșantionare pentru a selecta ferestre locale (patch-uri) care urmează să fie convertite în trenuri de impulsuri (spikes).

Această abordare este inspirată din mecanismele biologice de atenție vizuală, permițând modelului să se concentreze pe regiunile cele mai informative ale imaginii, reducând astfel costul computațional și zgomotul de fundal. În cadrul acestui simulator, au fost implementate trei strategii principale de eșantionare, atât 2D, cât și 3D.

Eșantionarea Spațială (2D)

Trei strategii distincte de eșantionare spațială au fost analizate și implementate, fiecare prezentând un grad diferit de complexitate și specificitate raportat la geometria feței. Acestea evoluează de la o abordare stohastică globală, complet independentă de conținutul imaginii, până la tehnici ghidate de structura anatomică a subiectului, capabile să localizeze trăsăturile cu densitate informațională maximă. În continuare, sunt detaliate principiile matematice și logica algoritmică a fiecărei metode în parte.

Eșantionarea Aleatorie

Metoda de eșantionare aleatorie reprezintă abordarea de bază, în care locația ferestrei de filtrare este determinată în mod stohastic pe întreaga suprafață a imaginii de intrare. Coordonatele colțului stânga-sus al ferestrei, (x, y), sunt extrase dintr-o distribuție uniformă, respectând constrângerile dimensionale ale imaginii, conform ecuației 2.23:

$$x \sim \mathcal{U}(0, W - w_f), \quad y \sim \mathcal{U}(0, H - h_f), \tag{2.23}$$

unde W, H reprezintă lățimea și înălțimea imaginii, iar w_f, h_f dimensiunile filtrului. Deși această metodă asigură o acoperire imparțială a spațiului de intrare, ea este ineficientă în sarcini de recunoaștere a emoțiilor faciale, deoarece o mare parte din eșantioane pot proveni din zone irelevante (fundal).

Eșantionarea bazată pe Regiunea Facială

Pentru a restrânge spațiul de căutare la zona de interes (Region of Interest - ROI), se utilizează casete de delimitare (bounding boxes) detectate în prealabil. Această metodă asigură faptul că ferestrele eșantionate se suprapun cu zona feței.

Procesul de selecție a coordonatelor este limitat de limitele minime și maxime ale casetei de delimitare, asigurând astfel că eșantionul rezultat va conține informație relevantă pentru sarcina de clasificare a emoțiilor.

Eșantionarea bazată pe Puncte de Reper

Cea mai avansată metodă implementată este eșantionarea bazată pe puncte de reper (landmarks). Această strategie se fundamentează pe ipoteza că emoțiile faciale sunt cel mai bine reprezentate de mișcările micro-expresiilor din zonele ochilor, nasului și gurii.

Logica de funcționare presupune parcurgerea următoarelor etape:

1. **Selecția punctului cheie:** Se alege în mod aleatoriu un punct de reper dintr-un set predefinit (corespunzător indicilor 17–67 în formatul standard de 68 de puncte, după cum este ilustrat în Figura 2.6), care acoperă trăsăturile interne ale feței.

2. **Centrarea ferestrei:** Coordonatele ferestrei sunt calculate astfel încât punctul de reper selectat să fie poziționat în centrul patch-ului de $w_f \times h_f$.

3. **Introducerea zgomotului (Jitter):** Pentru a crește robustețea modelului și a evita supra-învățarea pe poziții fixe, se adaugă un factor de variație aleatorie (jitter) în intervalul $[-2, 2]$ pixeli prin aplicarea distribuției uniforme.

4. **Constrângere (Clamping):** Se aplică o funcție de *clamping* pentru a asigura că fereastra rezultată nu depășește limitele fizice ale imaginii.

Această selecție a indicilor nu este arbitrară, ci urmează schema de adnotare propusă în setul de date 300-W [STZP13], utilizată pe scară largă în bibliotecile de procesare a feței (precum Dlib sau MediaPipe), detalierea fiecărei regiuni faciale fiind prezentată în Tabelul 2.2. Intervalul $[17, 67]$ a fost ales pentru a exclude conturul feței (jawline) și a maximiza probabilitatea de a eșantiona trăsături care definesc starea emoțională (ochi, sprâncene, gură).

Prin utilizarea acestor repere, procesul de eșantionare imită modul în care sistemul vizual uman prioritizează punctele de înaltă frecvență informațională pentru a recunoaște expresiile faciale complexe.

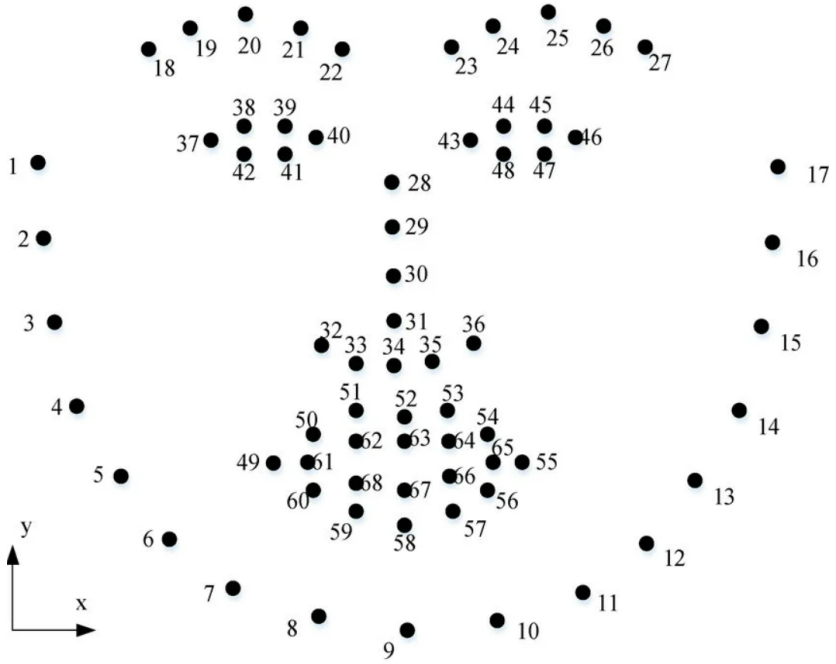


Figura 2.6: Reprezentarea celor 68 de puncte de reper faciale (landmarks) conform standardului 300-W [LWG+19].

Eșantionarea Spatio-Temporală (3D)

În cazul procesării secvențelor video pentru recunoașterea emoțiilor faciale, modelul trebuie să capteze nu doar trăsăturile statice, ci și dinamica temporală a expresiilor. Astfel, eșantionarea 3D extinde conceptele anterioare prin adăugarea unei a treia dimensiuni: timpul (t). Rezultatul procesului de eșantionare este un volum de date definit de coordonatele (x, y, t), unde t reprezintă punctul de start al ferestrei temporale.

Eșantionarea Aleatorie

Această metodă generează în mod stohastic toate cele trei coordonate. Fereastra temporală este selectată dintr-o distribuție uniformă 2.24, asigurând că volumul extras se încadrează în durata totală a secvenței video (T).

$$t \sim \mathcal{U}(0, T - t_f), \tag{2.24}$$

unde t_f reprezintă dimensiunea filtrului pe axa temporală.

Strategii Faciale bazate pe Bounding Box

Pentru a gestiona mișcarea feței de-a lungul timpului, au fost implementate două strategii de restrângere a spațiului de căutare:

Tabela 2.2: Distribuția punctelor de reper faciale în formatul standard de 68 de puncte.

Regiune Facială	**Indici (0-indexed)**
Contur Mandibulă	0 – 16
Sprânceana Dreaptă	17 – 21
Sprânceana Stângă	22 – 26
Puntea Nazală	27 – 30
Baza Nasului	31 – 35
Ochiul Drept	36 – 41
Ochiul Stâng	42 – 47
Contur Exterior Gură	48 – 59
Contur Interior Gură	60 – 67

- **Per Frame:** Această metodă este dinamică. Pentru fiecare fereastră temporală selectată, se calculează o regiune de reuniune (*Union BBox*) care cuprinde toate casetele de delimitare ale cadrelor din intervalul $[t, t + t_f]$. Aceasta garantează că, indiferent de mișcarea capului în acea fracțiune de secundă, eșantionul va conține fața subiectului.
- **Per Video:** Reprezintă o abordare mai conservatoare, unde se calculează un singur *Union BBox* pentru întreaga durată a videoclipului. Este utilă în scenarii cu mișcare redusă, oferind o stabilitate spațială mai mare eșantioanelor extrase.

Eșantionarea bazată pe Landmark-uri

Logica acestei metode combină precizia punctelor de reper cu dimensiunea temporală. Procesul decurge astfel:

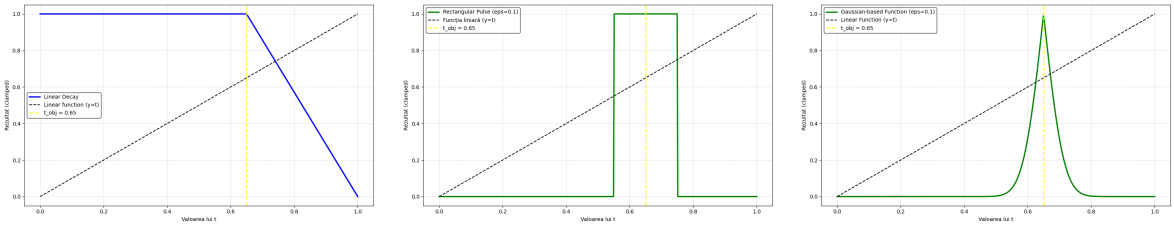
1. Se alege aleatoriu momentul de start t.
2. Se accesează datele despre punctele de reper specifice cadrului t.
3. Se selectează un punct de reper facial (din intervalul de indici $17 - 67$) și se centrează fereastra spațială (x, y) în jurul acestuia.

Această eșantionare permite rețelei CSNN să analizeze evoluția temporală a unei trăsături faciale specifice (de exemplu, curbura gurii în timpul unui zâmbet), eliminând necesitatea procesării întregului cadru și focalizând activitatea neuronală (spikes) pe zonele cu relevanță semantică maximă.

2.9 Metode de Decodificare a Semnalelor Neuronale

În cadrul arhitecturilor de tip Convolutional Spiking Neural Networks (CSNN), etapa de decodificare reprezintă puntea de legătură între activitatea neuronală discretă (spike-uri) și interpretarea acestora ca probabilități pentru expresiile faciale. Deoarece informația în CSNN este codificată temporal, am implementat trei strategii distincte de ponderare a activității neuronale pentru a evalua relevanța spike-urilor în raport cu un moment de referință t_{obj} setat la antrenare, ce reprezintă timpul obiectiv la care să aibă loc spike-urile.

Comportamentul vizual al acestor funcții de decodificare este prezentat în Figura 2.7.



(a) Linear Decay (b) Rectangular Pulse (c) Gaussian-based

Figura 2.7: Vizualizarea comparativă a funcțiilor de decodificare utilizate.

Decodificarea prin Declin Liniar (Linear Decay)

Această metodă, ilustrată în Figura 2.7a, aplică o descreștere constantă a importanței semnalului pe măsură ce timpul t avansează dincolo de pragul t_{obj}. Ea a fost selectată pentru eficiența sa computațională și pentru capacitatea de a modela degradarea simplă a memoriei pe termen scurt a neuronilor, oferind o penalizare proporțională pentru spike-urile care apar târziu în fereastra de procesare a expresiei faciale.

Decodificarea prin Impuls Dreptunghiular (Rectangular Pulse)

Metoda impulsului dreptunghiular (Figura 2.7b) acționează ca un filtru binar de tip ”fereastră”. Aceasta atribuie o importanță maximă (1.0) tuturor spike-urilor care apar într-un interval strict de ± 0.1 față de centrul t_{obj}, ignorând complet restul activității. Această abordare este utilă pentru detectarea coincidențelor temporale rigide, eliminând zgomotul de fond și spike-urile izolate care nu se încadrează în dinamica temporală specifică unei anumite micro-expresii.

Decodificarea Gaussiană (Gaussian-based Importance)

Cea mai sofisticată metodă, bazată pe funcția de eroare (erf), generează o curbă de tip clopot centrată pe t_{obj} (Figura 2.7c). Spre deosebire de metodele anterioare, aceasta aproximează cel mai bine variabilitatea biologică a neuronilor și oferă o tranziție lină (smooth transition), fiind robustă la mici variații temporale.

Capitolul 3

Experimente și analiza rezultatelor

3.1 Setul de date

În cadrul acestui proiect, pentru evaluarea performanței modelului propus, a fost utilizat setul de date **CK+ (Cohn-Kanade Extended Dataset)**, unul dintre cele mai utilizate seturi de date publice pentru recunoașterea expresiilor faciale în mediul controlat.

Descrierea datelor

Setul de date CK+ conține înregistrări faciale reale, fiind compus din secvențe video ce surprind trecerea de la o expresie neutră la vârful (apexul) unei emoții specifice. Link-ul oficial pentru accesarea setului de date este: `http://www.pitt.edu/~emotion/ck-spread.htm`.

Caracteristicile principale ale subsetului utilizat sunt:

- **Categorii:** Există 7 clase de emoții etichetate: 1. Furie (Anger), 2. Dispreț (Contempt), 3. Dezgust (Disgust), 4. Frică (Fear), 5. Fericire (Happiness), 6. Tristețe (Sadness), 7. Surpriză (Surprise).
- **Structura secvențelor:** Pentru fiecare subiect și fiecare ipostază, au fost extrase câte 10 cadre (frames), care acoperă evoluția temporală a expresiei.
- **Dimensiune imagini:** Imaginile au fost redimensionate la o rezoluție de 48×48 pixeli pentru a optimiza procesul de calcul.

Distribuția datelor pe fold-uri

Pentru a asigura o evaluare robustă, s-a utilizat o strategie de **10-fold cross-validation**, folosind seed-ul 42 pentru reproductibilitate. Distribuția celor 309 secvențe pe cele 10 fold-uri este prezentată în Tabelul 3.1.

Tabela 3.1: Distribuția emoțiilor pe cele 10 fold-uri (CK+)

Fold	Anger	Contempt	Disgust	Fear	Happiness	Sadness	Surprise	Total
1	5	2	6	3	7	3	8	34
2	5	2	6	3	7	3	8	34
3	5	1	6	3	7	3	8	33
4	5	1	6	3	7	3	8	33
5	5	1	5	2	7	3	8	31
6	4	1	5	2	7	3	8	30
7	4	1	5	2	7	3	8	30
8	4	1	5	2	7	3	7	29
9	4	1	5	2	7	2	7	28
10	4	1	5	2	6	2	7	27
Total	**45**	**12**	**54**	**24**	**69**	**28**	**77**	**309**

3.2 Metodologie de antrenare și evaluare

Arhitectura modelului și paradigma de învățare

Modelul implementat se bazează pe o rețea neuronală cu impulsuri (*Spiking Neural Network* - SNN) cu o arhitectură convoluțională 3D, capabilă să proceseze atât informația spațială, cât și pe cea temporală din secvențele video.

Paradigma de învățare utilizată este una de tip **STDP (Spike-Timing-Dependent Plasticity)**, specific regula *Biological STDP*. Aceasta permite ajustarea greutăților sinaptice în funcție de corelația temporală a impulsurilor dintre neuronii pre-sinaptici și post-sinaptici.

Preprocesarea datelor

Înainte de a fi introduse în rețea, datele trec prin următoarele etape:

1. **Filtrare On/Off:** Utilizarea unui `DefaultOnOffFilter` pentru evidențierea contrastelor și a marginilor (mărime filtru 7, parametri 0.1 și 1.0).
2. **MaxScaling:** Normalizarea intensității pixelilor.
3. **Latency Coding:** Conversia pixelilor în impulsuri (spikes), unde intensitatea este codificată prin latența temporală a primului impuls.

Configurarea stratului convoluțional și hiper-parametri

Nucleul modelului este reprezentat de un strat de convoluție 3D (`Convolution3D`). Parametrii utilizați pentru antrenare sunt detaliați în Tabelul 3.2.

Tabela 3.2: Hiper-parametrii rețelei SNN

Parametru	**Valoare**
Număr de filtre	64
Dimensiune filtru (WxHxD)	$5 \times 5 \times 3$
Număr de epoci	800
Threshold inițial (θ)	4.0
t_{obj} (Time Objective)	0.80
Learning rate Threshold (μ_{th})	0.09
Learning rate Greutăți (μ_w)	0.009
Spatial Pooling (output size)	4×4
Temporal Pooling (output size)	2

Evaluarea și Clasificarea

După extragerea trăsăturilor prin stratul convoluțional SNN și aplicarea metodelor de *SumPooling* și *TemporalPooling*, reprezentările finale sunt clasificate folosind un algoritm **SVM (Support Vector Machine)**. Evaluarea se face prin cross-validation pe cele 10 fold-uri menționate anterior.

3.3 Metrici de evaluare

- **Metrici Globale de Clasificare:**
  - **Acuratețea (Accuracy):** Proporția predicțiilor corecte din totalul eșantioanelor:

$$\text{Accuracy} = \frac{N_{correct}}{N_{total}} \times 100\%$$

    Matricea de Confuzie: Reprezentare tabelară pentru identificarea erorilor sistematice între emoții similare (ex. *fear* vs *surprise*).

- **Metrici Specifice Straturilor SNN (Per Layer Analysis):**
  - **Activitatea Neuronală (Activity):** Reprezintă rata de declanșare a spike-urilor în cadrul stratului.
    * *Importanță:* Un neuron care emite un spike tinde să își reducă la tăcere vecinii (inhibiție laterală). Analiza activității este esențială pentru menținerea stării de **homeostazie** a rețelei.
  - **Coerența (Coherence):** Măsoară gradul de redundanță al trăsăturilor învățate.
    * *Interpretare:* O coerență scăzută indică faptul că SNN-ul a învățat o varietate mare de trăsături distincte. Antrenamentul eficient trebuie

să tindă spre o anumită **incoerență** pentru a maximiza capacitatea de reprezentare.

Activitatea Neuronală (Activity)

Reprezintă dinamica de descărcare a spike-urilor în raport cu topologia stratului.

- **Sparsity (Raritatea):** Se calculează pe baza raportului dintre norma L_1 și norma L_2 a activărilor.
  - *Interval optim:* O valoare cuprinsă între **0.7 și 0.9** este ideală. O raritate prea mare (spre 1.0) indică o rețea "moartă", în timp ce o valoare mică indică o activitate densă, care anulează avantajul energetic al SNN.
- **Active Units (%):** Procentul de neuroni care emit cel puțin un spike per eșantion.
  - *Interval optim:* Pentru straturi convoluționale, o activitate de **1% - 5%** este considerată biologic plauzibilă și eficientă computațional.
- **Quiet (%):** Procentul de eșantioane pentru care stratul nu a generat niciun spike.
  - *Interpretare:* Această valoare trebuie să fie cât mai aproape de **0%**. O valoare ridicată (ex. ¿10%) sugerează că pragul de activare (*threshold*) este prea înalt sau că datele de intrare nu sunt normalizate corect.

Coerența Trăsăturilor (Coherence)

Măsoară similitudinea spectrală între nucleele (kernel-urile) de convoluție prin calcularea similitudinii cosinus între vectorii de ponderi w_i și w_j.

$$\text{Coherence}(w_i, w_j) = \frac{w_i \cdot w_j}{\|w_i\|\|w_j\| + \epsilon} \tag{3.1}$$

- **Interpretarea Cuartilelor (Q1, Q2, Q3):**
  - *Interval optim (Mediana/Q2):* O valoare sub **0.3 - 0.5** indică o rețea sănătoasă care a învățat trăsături diverse (margini, texturi diferite).
  - *Riscuri:* Dacă *Max* și *Q3* tind spre **0.9 - 1.0**, rețeaua suferă de redundanță extremă (mai mulți neuroni au învățat exact aceeași trăsătură), ceea ce limitează capacitatea de discriminare a emoțiilor.
- **Importanța Incoerenței:** Un grad ridicat de decorelare între filtre este esențial pentru ca algoritmul de clasificare ulterior (SVM sau FC) să poată extrage informație utilă din spațiul trăsăturilor.

3.4 Rezultate obținute și analiză

În această secțiune sunt prezentate performanțele obținute în urma procesului de cross-validation, precum și o analiză detaliată a comportamentului rețelei la nivel de distribuție a datelor și a impulsurilor (*spikes*).

Performanța pe fold-uri

Rezultatele clasificării pentru fiecare dintre cele 10 fold-uri sunt centralizate în Tabelul 3.3. Rata medie de clasificare obținută este de aproximativ **48.02%**.

Tabela 3.3: Acuratețea clasificării pe fiecare fold

Fold	Rata de clasificare (%)	Corecte / Total
Fold 1	58.82%	20/34
Fold 2	47.05%	16/34
Fold 3	51.51%	17/33
Fold 4	45.45%	15/33
Fold 5	41.93%	13/31
Fold 6	53.33%	16/30
Fold 7	50.00%	15/30
Fold 8	48.27%	14/29
Fold 9	35.71%	10/28
Fold 10	48.14%	13/27
Medie	**48.02%**	**159 / 309**

Analiza detaliată pe Fold 1

Fold-ul 1 a înregistrat cea mai ridicată performanță (58.82%). Pentru a înțelege punctele forte și limitările modelului, matricea de confuzie pentru Fold 1 (Tabelul 3.4), surprinde corelarea etichetelor reale (*True Label*) cu probabilitățile maxime generate de clasificatorul SVM.

Interpretarea erorilor de clasificare

Analiza probabilităților individuale relevă câteva fenomene tipice pentru procesarea imaginilor faciale prin SNN:

- **Suprapunerea caracteristicilor Furie-Dezgust:** Toate secvențele de *Anger* (clasa 1) au primit probabilitatea maximă pentru *Disgust* (clasa 3). Acest lucru se explică prin similitudinea morfologică a acestor expresii (încrețirea nasului și coborârea sprâncenelor), pe care filtrele convoluționale actuale le extrag ca fiind identice.

Tabela 3.4: Matricea de confuzie pentru Fold 1 (34 secvențe)

Real \Prezis	1	2	3	4	5	6	7	Recall %
1 (Anger)	0	0	5	0	0	0	0	0.0%
2 (Contempt)	1	0	1	0	0	0	0	0.0%
3 (Disgust)	0	0	**6**	0	0	0	0	100.0%
4 (Fear)	0	0	0	0	0	0	3	0.0%
5 (Happiness)	0	0	0	0	**7**	0	0	100.0%
6 (Sadness)	1	0	2	0	0	0	0	0.0%
7 (Surprise)	0	0	1	0	0	0	**7**	87.5%
Precision %	0.0%	0.0%	40.0%	0.0%	100.0%	0.0%	70.0%	**Acc: 58.8%**

- **Dificultatea detectării Fricii:** Emoția *Fear* (clasa 4) a fost clasificată sistematic drept *Surprise* (clasa 7). Din punct de vedere vizual, ambele emoții implică deschiderea largă a ochilor și ridicarea sprâncenelor. Rezultatele sugerează că modelul reușește să detecteze activarea musculară, dar nu și subtilitatea temporală care diferențiază frica de surpriză.
- **Performanță Robustă la Emoții Extreme:** Clasele *Happiness* și *Disgust* au un *recall* de 100%, indicând faptul că arhitectura SNN propusă este extrem de eficientă în izolarea trăsăturilor de intensitate mare (zâmbet larg, respectiv contracție nazală pronunțată).
- **Clasele sub-reprezentate:** *Contempt* și *Sadness* au înregistrat performanțe slabe, fiind confundate cu emoții negative mai „puternice". Aceasta este o limitare cunoscută a setului de date CK+, unde aceste clase au mult mai puține exemple de antrenament.

Analiza distribuției valorilor și a impulsurilor

Pentru a înțelege modul în care stratul convoluțional procesează informația, am extras distribuția globală a valorilor înainte și după aplicarea stratului de convoluție 3D.

Distribuția după Latency Encoding (Input)

În faza de preprocesare, valorile sunt codificate temporal. Distribuția prezentată în Tabelul 3.5 arată o concentrație masivă a valorilor în intervalul $[0.9 - 1.0]$ (peste 72%), ceea ce indică faptul că majoritatea neuronilor de intrare generează impulsuri cu latență mare.

Distribuția după Stratul Convoluțional (Spikes)

După trecerea prin stratul de convoluție, se observă o menținere a densității în zona superioară, dar cu o ușoară redistribuire. Această activitate intensă în intervalul $[0.8 - 1.0]$ sugerează că filtrele convoluționale au identificat numeroase corelații spațio-temporale în secvențele video de intrare.

Tabela 3.5: Analiza comparativă a distribuției valorilor (Fold 1)

Interval	Înainte de Conv (Input) %	După Conv (Spikes) %
$[0.0 - 0.5]$	1.66%	0.01%
$[0.5 - 0.6]$	1.71%	0.14%
$[0.6 - 0.7]$	3.37%	1.24%
$[0.7 - 0.8]$	6.57%	6.34%
$[0.8 - 0.9]$	14.01%	22.49%
$[0.9 - 1.0]$	72.68%	69.78%

Interpretarea rezultatelor

Analiza distribuției arată că rețeaua nu suferă de *vanishing spikes*, având o activitate susținută. Totuși, rata de clasificare medie sugerează că, deși anumite emoții cu trăsături faciale distincte (precum fericirea sau surpriza) sunt ușor de separat, distincția între emoțiile subtile (furie vs. dezgust) necesită o rafinare a hiper-parametrilor sau o creștere a numărului de filtre pentru a surprinde diferențe mai fine de textură și mișcare. Deși rata medie de 48.02% indică un potențial ridicat pentru recunoașterea emoțiilor de intensitate mare (Happiness, Disgust), se observă o dificultate în discriminarea stărilor afective subtile. Această limitare este pusă pe seama numărului redus de eșantioane pentru clase precum Contempt, dar și pe natura nesupervizată a algoritmului STDP, care prioritizează frecvența trăsăturilor în detrimentul discriminabilității lor. Optimizarea viitoare a pragului de activare (θ) și a ratei de învățare a pragului (μ_{th}) ar putea conduce la o mai bună specializare a filtrelor.

Vizualizarea și analiza hărților de trăsături (Feature Maps)

Pentru a investiga modul în care modelul *Convolutional SNN* extrage informația din secvențele video, au fost vizualizate hărțile de trăsături (*feature maps*) generate de stratul convoluțional. Această analiză este esențială pentru a valida capacitatea regulii de învățare STDP de a codifica tipare spațio-temporale relevante fără supraveghere.

Impactul mecanismului TimeObjectiveOutput

O componentă critică în arhitectura propusă este etapa de conversie a output-ului. În Figura 3.1 este ilustrat efectul aplicării transformării `OutputConverter.TimeObjectiveOut` asupra unei secvențe de *Anger*.

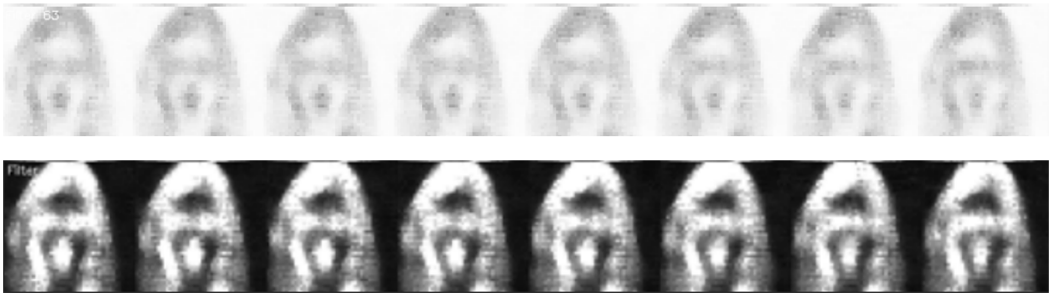


Figura 3.1: Vizualizarea hărților de trăsături pentru clasa Anger: Sus - activări brute (pre-procesare); Jos - activări după aplicarea TimeObjectiveOutput.

Analiza vizuală relevă următoarele aspecte:

- **Normalizarea Latenței:** În varianta brută, activările sunt difuze, având un raport semnal-zgomot scăzut. Aplicarea `TimeObjectiveOutput` forțează latențele spre valoarea țintă $t_{obj} = 0.8$, rezultând într-o reprezentare mult mai clară a contururilor faciale.
- **Evidențierea Dinamicii:** După procesare, se observă o focalizare a impulsurilor în zonele cu mișcare musculară intensă (frunte, ochi, gură). Această „curățare" a semnalului permite clasificatorului SVM să lucreze cu trăsături mai bine definite, minimizând redundanța spațială.

Analiza calitativă a hărților de trăsături per emoție

În Figurile 3.2 și 3.3 sunt prezentate seturile de hărți de trăsături extrase pentru cele 7 categorii de emoții. Fiecare eșantion este reprezentat printr-o succesiune de cadre ce reflectă intensitatea activării neuronale. În paradigma *Latency Coding*, zonele albe (pixeli cu intensitate mare) corespund unor latențe mici, indicând o declanșare rapidă a impulsurilor ca răspuns la trăsături faciale proeminente.

Corelația dintre trăsături și performanța clasificării

Observarea hărților de trăsături permite explicarea rezultatelor obținute în matricea de confuzie:

- **Happiness și Disgust:** Aceste clase prezintă zone de activare extrem de bine localizate (comisurile bucale pentru fericire și zona nazală pentru dezgust).

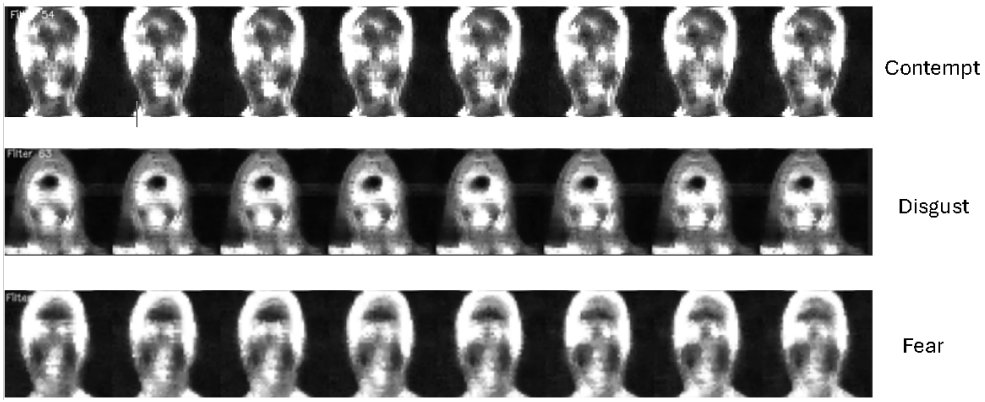


Figura 3.2: Hărți de trăsături pentru clasele Contempt, Disgust și Fear.

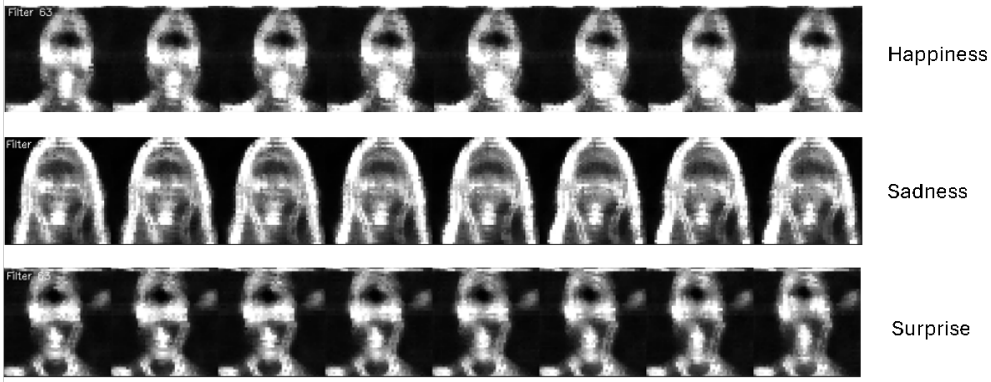


Figura 3.3: Hărți de trăsături pentru clasele Happiness, Sadness și Surprise.

Caracterul distinctiv al acestor impulsuri timpurii explică rata de *recall* de 100% pentru aceste clase.

- **Surprise vs. Fear:** Hărțile de trăsături confirmă similitudinea vizuală între cele două emoții. În ambele cazuri, filtrele SNN detectează deschiderea largă a ochilor, însă distribuția spațială a impulsurilor este aproape identică, ceea ce conduce la clasificarea sistematică a fricii drept surpriză.
- **Sadness și Contempt:** Aceste emoții generează hărți cu o densitate mai mică a impulsurilor („pete" mai întunecate), indicând o mișcare facială de intensitate redusă. Această activitate rarefiată face ca trăsăturile extrase să fie mai puțin robuste în fața zgomotului, explicând performanța scăzută pe aceste clase.

Această analiză vizuală demonstrează că modelul SNN propus a învățat să acționeze ca un extractor de trăsături fiziologice, reușind să mapeze corelările spațio-temporale din clipurile video în latențe neuronale discriminative.

Vizualizarea nucleelor convoluționale (Greutăți Sinaptice)

Pentru o înțelegere aprofundată a modului în care s-a realizat adaptarea sinaptică prin procesul STDP nesupervizat, au fost extrase greutățile sinaptice ale celor 64 de

filtre după cele 800 de epoci de antrenament pe Fold 1. Vizualizarea acestor nuclee (*kernels*) de dimensiune $5 \times 5 \times 3$ este prezentată în Figura 3.4. Deoarece un nucleu este tridimensional (spatial 5×5 pe o adâncime temporală 3), vizualizarea este o proiecție color a volumului de greutăți.

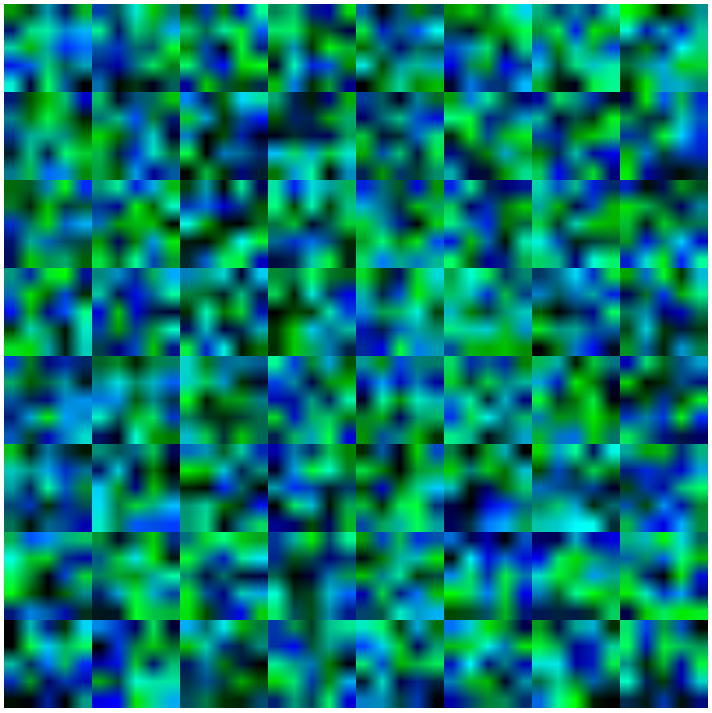


Figura 3.4: Vizualizarea celor 64 de nuclee convoluționale (8×8 filtre) învățate de SNN pe Fold 1. Culoarea reprezintă magnitudinea și polaritatea greutăților sinaptice (nivele de potențiere și depresie).

Analiza calitativă a Figura 3.4 relevă tipare caracteristice învățării STDP nesupervizate pe secvențe video:

- **High-Frequency Spatial Variance:** Nucleele prezintă o variație spațială ridicată, aspectul lor fiind aparent zgomotos. Acest fenomen este caracteristic etapelor inițiale de specializare a filtrelor în SNN-uri nesupervizate. În loc să învețe margini orientate generice (cum ar fi filtrele Gabor), STDP-ul tinde să specializeze nucleele pe tipare foarte locale și specifice de corelații temporale de impulsuri, prezente în datele de intrare.

- **Validarea Incoerenței:** Aspectul divers și aparent aleatoriu al filtrelor este, paradoxal, un rezultat dorit. Așa cum s-a menționat în secțiunea **??** (definiția coerenței), antrenamentul eficient trebuie să tindă spre o incoerență ridicată. Faptul că filtrele nu sunt identice și nu prezintă trăsături repetitive validează succesul regulii de învățare STDP în a evita redundanța și în a acoperi un spectru larg de variații ale datelor.

- **Limitările învățării trăsăturilor globale:** Absența unor structuri globale clar definite în kernels explică performanța scăzută pe clasele subtile. Filtrele sunt suficient de discriminate pentru a distinge între schimbări majore de intensitate (zâmbet vs. contracție nazală), dar nu au învățat încă să reprezinte deformări fine ale feței pe care clasificatorul liniar ulterior (SVM) să le poată separa eficient.

3.5 Analiză Comparativă și Validarea Modelului

Pentru a evalua relevanța trăsăturilor extrase de stratul convoluțional SNN, a fost realizat un experiment de control (baseline). În cadrul acestuia, clasificatorul SVM a fost antrenat direct pe pixelii imaginilor brute (*raw pixels*), eliminând complet componenta de spiking și procesarea STDP.

Configurația modelului de referință (Baseline)

Modelul de referință utilizează un strat de tip `layer::Stream`, care acționează ca o funcție identitate, pasând datele de intrare direct către etapele de pooling, fără nicio preprocesare sau filtrare temporală. Configurația utilizată este următoarea:

- **Input:** Pixeli raw (fără *Latency Coding*).
- **Pooling:** `SumPooling` (4×4) și `TemporalPooling` (2), identic cu modelul SNN pentru a asigura o comparație echitabilă a spațiului de trăsături.
- **Clasificator:** SVM liniar.

Rezultate comparative

Rezultatele obținute prin cross-validation pentru ambele abordări sunt centralizate în Tabelul 3.6.

Interpretarea superiorității modelului SNN

Compararea performanțelor evidențiază un câștig net de **6.41%** în favoarea arhitecturii bazate pe SNN. Această diferență, deși aparent modestă în cifre absolute, este semnificativă din punct de vedere al procesării bio-inspirate:

1. **Extracția de trăsături (Feature Extraction):** Diferența de performanță confirmă faptul că nucleele învățate prin STDP nu sunt aleatorii; ele reușesc să distileze informația vizuală în „puncte de interes" temporale care sunt mai ușor de separat liniar de către un SVM decât pixelii bruti.

Tabela 3.6: Comparație între SVM pe pixeli raw și arhitectura propusă (SNN + SVM)

Fold	SVM Pixeli Raw (%)	SNN + SVM (Propus) (%)
Fold 1	44.11%	58.82%
Fold 2	38.23%	47.05%
Fold 3	54.54%	51.51%
Fold 4	39.39%	45.45%
Fold 5	38.70%	41.93%
Fold 6	36.66%	53.33%
Fold 7	43.33%	50.00%
Fold 8	44.82%	48.27%
Fold 9	39.28%	35.71%
Fold 10	37.03%	48.14%
Medie	**41.61%**	**48.02%**

2. **Reducerea zgomotului:** În timp ce pixelii raw conțin multă informație redundantă (fundal, variații de iluminare), SNN-ul, prin filtrarea On/Off și Latency Coding, se concentrează pe marginile dinamice (schimbările de expresie), oferind un input mai „curat" pentru clasificator.

3. **Analiza cazurilor particulare:** Se observă că în aproape toate fold-urile (cu excepția fold-ului 3 și 9), SNN-ul depășește baseline-ul. În Fold 1, saltul este de la 44.11% la 58.82% (+14.71%), ceea ce indică faptul că pentru anumite secvențe video, reprezentarea prin impulsuri este net superioară celei spațiale clasice.

În concluzie, rezultatele demonstrează validitatea ipotezei de lucru: rețelele neuronale cu impulsuri, antrenate nesupervizat, sunt capabile să extragă trăsături temporale relevante pentru recunoașterea expresiilor faciale, îmbunătățind capacitatea de generalizare a sistemului față de procesarea directă a imaginilor.

3.6 Modelul de Înaltă Performanță: Arhitectura Hibridă CNN-LSTM

Pentru a oferi un termen de comparație raportat la standardele actuale de *Deep Learning* și pentru a echipa aplicația finală cu un motor de predicție robust, a fost implementată o arhitectură hibridă de tip **CNN-LSTM**. Acest model combină puterea de extracție a trăsăturilor spațiale ale unei rețele convoluționale pre-antrenate cu capacitatea rețelelor recurente de a modela evoluția temporală a emoțiilor.

Descrierea Arhitecturii

Modelul propus utilizează un mecanism de *Feature Extraction* urmat de un modul de memorie temporală:

1. **Extractorul Spațial (Backbone):** S-a utilizat modelul **ResNet18**, pre-antrenat pe setul de date ImageNet. S-a păstrat structura până la stratul de *Adaptive Average Pooling*, rezultând un vector de trăsături de dimensiune 512 pentru fiecare cadru video.

2. **Modulul Temporal (LSTM):** Trăsăturile extrase sunt introduse într-un strat **LSTM** (*Long Short-Term Memory*) cu o dimensiune ascunsă de 256. Acesta analizează succesiunea celor 10 cadre pentru a identifica „apexul" emoției.

3. **Capul de Clasificare:** Un modul de tip *Fully Connected* cu *Dropout* (0.5) și activare ReLU realizează maparea finală către cele 7 clase de emoții.

Strategia de Antrenament în Două Faze

Pentru a maximiza performanța pe un set de date de dimensiuni medii precum CK+, s-a adoptat o strategie de *Transfer Learning* structurată în două etape:

- **Faza 1 (Warm-up):** Înghețarea parametrilor CNN și antrenarea exclusivă a modulelor LSTM și FC timp de 10 epoci (LR=10^{-3}). Acest lucru permite modelului temporal să se adapteze la trăsăturile extrase de ResNet.

- **Faza 2 (Fine-tuning):** Dezghețarea selectivă a ultimului bloc convoluțional (*Layer 4*) din ResNet și antrenarea întregului ansamblu timp de 30 de epoci cu un *Learning Rate* scăzut (10^{-4}) și *Cosine Annealing*.

Rezultate Obținute

Performanța modelului CNN-LSTM, evaluată prin 10-fold cross-validation, este prezentată în Tabelul 3.7.

3.7 Sinteza Rezultatelor și Concluzii Experimentale

În Tabelul 3.8 este prezentată comparația finală între cele trei abordări investigate în cadrul acestei lucrări.

Tabela 3.7: Acuratețea modelului CNN-LSTM pe cele 10 fold-uri

Fold	Acuratețe (%)	Fold	Acuratețe (%)
Fold 1	87.88%	Fold 6	85.71%
Fold 2	85.71%	Fold 7	93.75%
Fold 3	81.82%	Fold 8	89.66%
Fold 4	87.50%	Fold 9	78.57%
Fold 5	96.97%	Fold 10	96.15%
Medie	**88.37% ± 5.65%**		

Tabela 3.8: Comparație centralizată a modelelor evaluate

Model	Paradigmă	Acuratețe Medie	Complexitate
SVM (Raw Pixels)	Baseline Statistic	41.61%	Minimă
3D Conv SNN	Bio-inspirat (STDP)	**48.02%**	Medie
CNN-LSTM (ResNet)	Deep Learning	**88.37%**	Ridicată

Discuții

Deși modelul CNN-LSTM oferă cea mai înaltă precizie, fiind soluția optimă pentru aplicația practică de monitorizare în timp real, rezultatele obținute cu modelul **SNN** sunt promițătoare.

Diferența de performanță este explicată prin natura învățării: în timp ce CNN-LSTM beneficiază de pre-antrenare pe milioane de imagini și învățare supervizată prin *backpropagation*, modelul SNN propus utilizează o regulă de învățare locală, nesupervizată (STDP), mult mai eficientă din punct de vedere energetic și biologic. Această lucrare demonstrează că SNN-ul poate extrage trăsături relevante care depășesc performanța unui clasificator liniar aplicat pe date brute, deschizând calea către sisteme de recunoaștere a emoțiilor implementabile pe hardware neuromorfic.

Preprocesare și Augmentarea Datelor

Pentru modelul CNN-LSTM, strategia de pregătire a datelor a fost semnificativ mai complexă față de abordările SNN și SVM. Antrenarea s-a realizat exclusiv pe imagini focalizate strict pe zona feței (*cropped faces*), eliminând astfel elementele de fundal care ar fi putut introduce zgomot în procesul de extracție a trăsăturilor.

Spre deosebire de experimentele anterioare, unde s-au utilizat preprocesări minimale, aici s-a implementat o conductă de **augmentare a datelor** pentru a îmbunătăți generalizarea și a combate supra-antrenarea (*overfitting*). Deoarece setul CK+ este monocrom, cadrele au fost convertite la 3 canale (RGB) pentru a fi compatibile cu arhitectura ResNet18, aplicându-se următoarele transformări: Resize(224, 224), Grayscale(num_output_channels=3), RandomHorizontalFlip(), RandomRotation(10), Co-

lorJitter(brightness=0.2, contrast=0.2), Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]).

Analiza performanței pe Fold 5

Analiza detaliată a Fold-ului 5 relevă eficiența modelului CNN-LSTM, acesta obținând o acuratețe de 96.97%. În Figura 3.5 se poate observa că modelul a reușit să clasifice perfect 6 din cele 7 emoții.

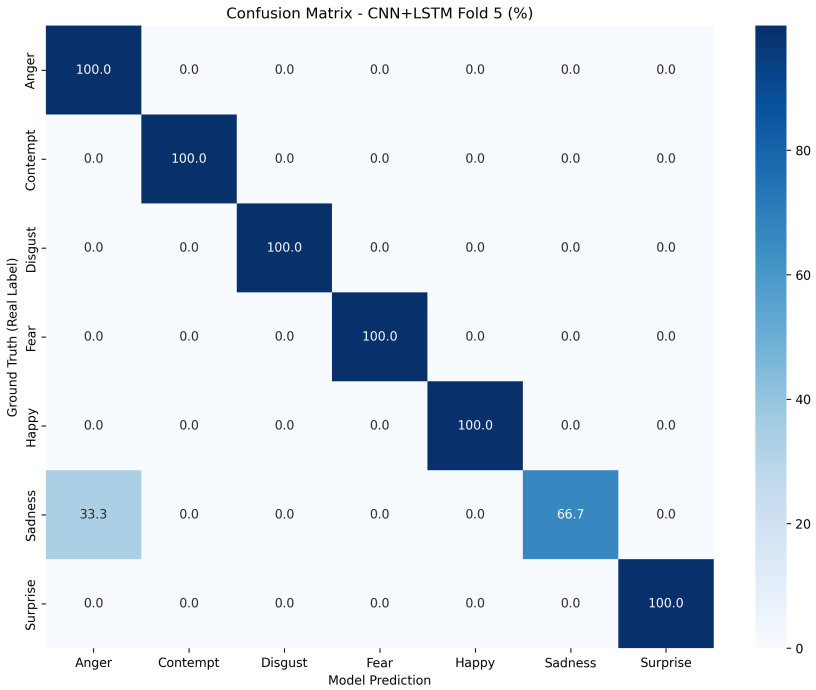
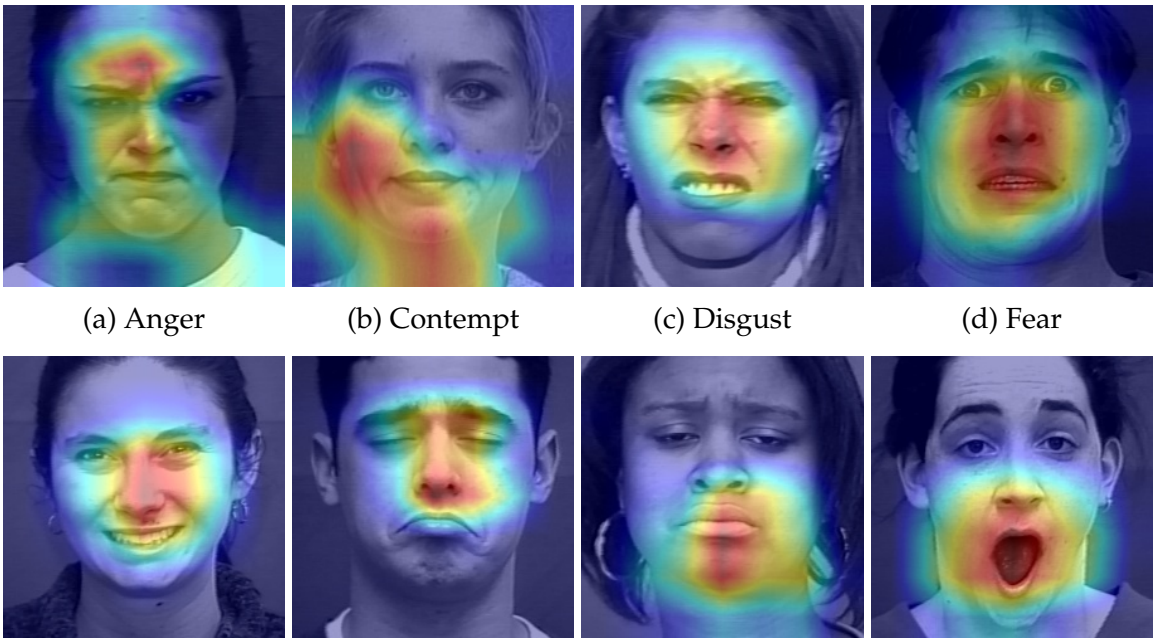


Figura 3.5: Matricea de confuzie pentru CNN-LSTM (Fold 5). Se observă o singură eroare de clasificare.

Singura eroare înregistrată a fost un eșantion din clasa **Sadness**, care a fost identificat eronat ca fiind **Anger**. Această confuzie este explicabilă prin tensiunea musculară similară în zona sprâncenelor (coborârea acestora) prezentă în ambele expresii în stadiul de apex.

Interpretarea deciziilor prin Grad-CAM

Pentru a valida calitativ modelul, s-a utilizat tehnica **Grad-CAM (Gradient-weighted Class Activation Mapping)**. Aceasta permite vizualizarea zonelor feței pe care rețeaua s-a focalizat pentru a stabili eticheta emoțională. În Figura 3.6 sunt prezentate hărțile de activare pentru eșantioanele din Fold 5.



(a) Anger (b) Contempt (c) Disgust (d) Fear

(e) Happy (f) Sad (Corect) (g) Sad → Anger (h) Surprise

Figura 3.6: Vizualizarea regiunilor de interes prin Grad-CAM (Fold 5).

Observații privind regiunile de interes

Analiza hărților de activare a scos în evidență câteva aspecte contra-intuitive și interesante:

- **Recunoașterea Fericirii (Happy):** Contrar așteptărilor, modelul nu se concentrează exclusiv pe zona gurii (zâmbet). Grad-CAM indică o activare intensă în **zona nasului** și a pliurilor nazolabiale. Acest lucru sugerează că modelul a învățat că lărgirea bazei nasului și ridicarea obrajilor sunt indicatori mai robuști pentru această clasă decât simpla deschidere a gurii.
- **Eroarea Sad vs. Anger:** În cazul eșantionului clasificat greșit, se observă că activarea Grad-CAM este localizată puternic între sprâncene. Modelul a interpretat contracția mușchiului *corrugator supercilii* ca fiind specifică furiei, ignorând indiciile mai subtile de tristețe din jurul ochilor.
- **Surpriza:** Modelul prezintă o focalizare clară pe zona ochilor și a frunții, detectând ridicarea sprâncenelor, trăsătură anatomică definitorie pentru această emoție.

3.8 Modelul ANN Echivalent: Izolarea Impactului Paradigmei de Spiking

Pentru a determina în ce măsură performanța modelului bio-inspirat este influențată de arhitectura sa convoluțională versus mecanismul de spiking, a fost implementat un model **ANN Echivalent (Equivalent Baseline ANN)**.

Arhitectură și Metodologie

Acest model păstrează topologia hărților de trăsături ale CSNN-ului, dar înlocuiește neuronii de tip *Leaky Integrate-and-Fire* cu unități de activare standard și învățare prin *backpropagation*.

- **Configurație:** Strat convoluțional 3D (64 filtre, kernel $3 \times 5 \times 5$), urmat de *Batch Normalization* și activare ReLU.
- **Pooling:** `AvgPool3d` ($2 \times 4 \times 4$), similar mecanismului de pooling din SNN.
- **Antrenare:** S-a utilizat optimizatorul Adam și funcția de cost *Cross-Entropy*, fără augmentare de date, pentru a menține condițiile experimentale ale CSNN-ului.
- **Date de intrare:** S-au utilizat fețe pre-procesate (*cropped*), eliminând fundalul pentru a permite modelului să conveargă rapid în absența augmentărilor.

Rezultate Experimentale

Rezultatele pe cele 10 fold-uri sunt prezentate în Tabelul 3.9. Media obținută este de **73.23%**, o valoare semnificativ superioară baseline-ului SVM, dar inferioară modelului complex CNN-LSTM.

Tabela 3.9: Performanța modelului ANN echivalent (3D Conv + ReLU)

Fold	Acuratețe (%)	Fold	Acuratețe (%)
Fold 1	75.76%	Fold 6	67.86%
Fold 2	80.00%	Fold 7	71.88%
Fold 3	72.73%	Fold 8	65.52%
Fold 4	84.38%	Fold 9	71.43%
Fold 5	69.70%	Fold 10	73.08%
Medie		**73.23%**	

Analiză Comparativă Finală

Prezența acestui model în studiu permite o interpretare critică a rezultatelor:

1. **Importanța Focalizării (Cropping):** Experimentele preliminare au arătat că utilizarea imaginilor complete (cu fundal) scade performanța ANN-ului la aproximativ 30-40%. Acest lucru subliniază un avantaj teoretic al CSNN-ului: prin filtrarea On/Off și Latency Coding, SNN-ul este inherent mai robust la zgomotul de fundal (pixelii statici nu generează spike-uri), în timp ce un ANN clasic are nevoie de decuparea feței sau de volume masive de date pentru a ignora fundalul.

2. **STDP vs. Backpropagation:** Diferența de la 48.02% (SNN) la 73.23% (ANN) reprezintă „costul" utilizării unei reguli de învățare locale și nesupervizate (STDP). Deși Backpropagation este mai precis, STDP oferă un avantaj major în termeni de eficiență energetică pe hardware neuromorfic și plauzibilitate biologică.

3. **Concluzie:** Modelul ANN echivalent demonstrează că topologia 3D Conv este extrem de potrivită pentru setul CK+, dar eficiența implementării spike-based depinde critic de rafinarea mecanismelor de homeostazie și a pragurilor de activare.

Capitolul 4

Neuro-Guardian

4.1 Colectarea cerințelor

În cadrul acestui capitol sunt detaliate funcționalitățile sistemului Neuro-Guardian, interacțiunile dintre utilizatori și platformă, precum și constrângerile tehnice sub care funcționează soluția propusă.

Actorii sistemului

Actorii reprezintă entitățile externe care interacționează cu sistemul. În cadrul platformei Neuro-Guardian, am identificat următoarele categorii:

- **Angajat Spital:** Actorul generic care include funcționalitățile de bază de autentificare (Înregistrare, Autentificare, Deconectare).
- **Administrator:** Moștenește trăsăturile angajatului și are responsabilitatea gestionării fluxului administrativ al pacienților (internări, externări și alocarea resurselor/paturilor).
- **Doctor:** Actorul principal în procesul de îngrijire, responsabil de preluarea pacienților și monitorizarea stării lor clinice și emoționale în timp real.
- **Camera Web (Sistem Extern):** Furnizează fluxul de date video necesar procesului de captare a cadrelor.
- **CSNN - Convolutional Spiking Neural Network (Sistem Extern):** Modelul de inteligență artificială care procesează cadrele recepționate și returnează predicțiile emoționale.

Diagrama cazurilor de utilizare

Diagrama cazurilor de utilizare, prezentată în Figura 4.1, oferă o imagine de ansamblu asupra modului în care funcționalitățile sunt distribuite între actori.

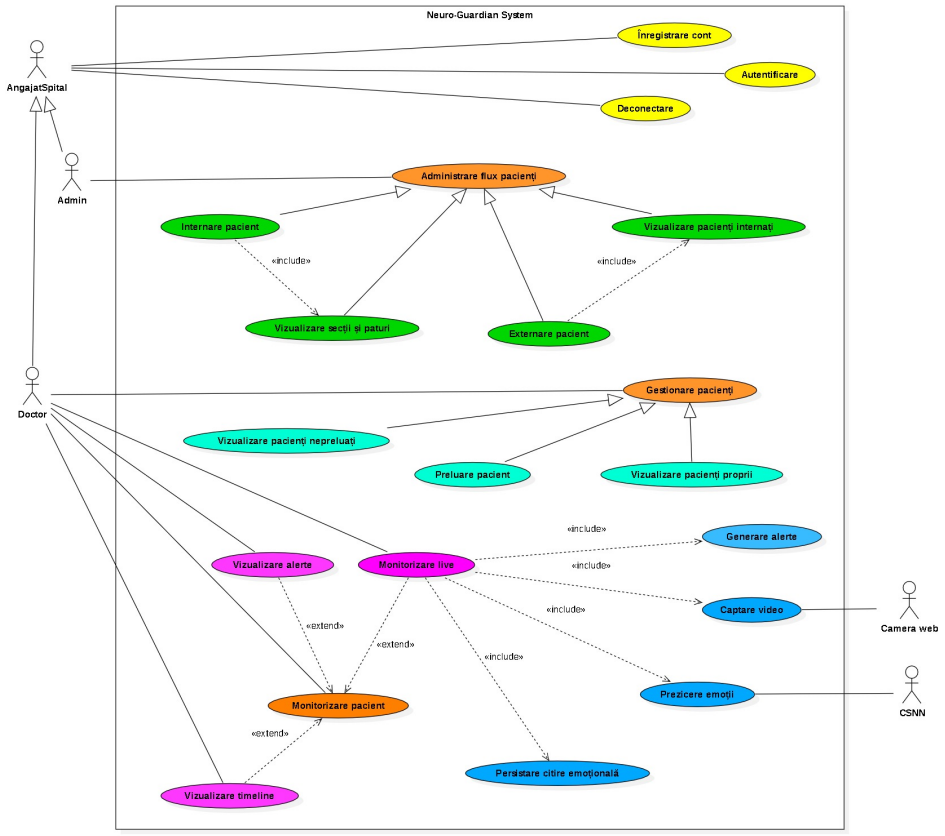


Figura 4.1: Diagrama cazurilor de utilizare - Sistemul Neuro-Guardian

Descrierea detaliată a unui caz de utilizare

În această secțiune este detaliat fluxul principal de interacțiune pentru cazul de utilizare **Gestionare Pacienți**, reprezentat grafic în Figura 4.1. Acest caz de utilizare este central pentru activitatea clinică a medicului, asigurând tranziția între selecția pacienților și monitorizarea activă.

Cerințe nefuncționale (NFR)

Cerințele nefuncționale definesc criteriile de calitate și constrângerile tehnice ale sistemului.

Securitate și control acces

- **NFR-S1:** Toate operațiile sensibile necesită autentificare pe bază de Bearer JWT.
- **NFR-S2:** Autorizare strictă pe rol (ADMIN vs DOCTOR); medicii pot vizualiza doar pacienții care le sunt asignați.
- **NFR-S3:** Parolele sunt stocate exclusiv în format hashed folosind algoritmul *bcrypt*.

Elemente de identificare	
Nume	Gestionare pacienți
Actori	Doctor
Descriere	Procesul prin care un medic vizualizează lista pacienților din secție, preia responsabilitatea pentru un pacient nou sau comută între pacienții deja asignați pentru a vizualiza datele biometrice și emoționale.
Condiții de context	
Precondiții	1. Medicul este autentificat în sistem. 2. Pacienții sunt deja internați în secție (via Admin). 3. Conexiunea WebSocket cu modelul AI este activă.
Postcondiții	Interfața reflectă starea în timp real a pacientului selectat, iar baza de date actualizează relația medic-pacient.
Scenariu Principal de Succes (MSS)	
Flux normal	1. Medicul accesează dashboard-ul; sistemul afișează implicit datele ultimului pacient preluat. 2. Medicul acționează butonul „Show panel" pentru a extinde panoul de selecție. 3. Sistemul interoghează baza de date și afișează două liste: „Available Patients" și „My Patients". 4. Medicul identifică un pacient nepreluat și selectează opțiunea „Take patient". 5. Sistemul realizează maparea medic-pacient și trimite medicul către pagina destinată pacientului respectiv 6. Medicul selectează un alt pacient din lista „My Patients" pentru a verifica evoluția acestuia. 7. Sistemul comută contextul de monitorizare, afișând informațiile specifice noului pacient selectat. 8. Medicul acționează „Hide panel" pentru a maximiza spațiul de vizualizare a detaliilor pacientului. 9. Sistemul ascunde panoul lateral și trece în modul de monitorizare imersivă.
Scenarii alternative	
Alt. 1	**Medicul nu are pacienți în responsabilitate:** 1a. Sistemul afișează un mesaj de informare și prezintă doar lista pacienților nepreluați. 1b. Medicul efectuează preluarea primului pacient (se revine la pasul 5 din MSS).
Alt. 2	**Inexistența pacienților nepreluați:** 3a. Lista „Available patients" apare goală; medicul poate gestiona doar portofoliul actual de pacienți (se revine la pasul 6 din MSS).
Excepții	
Excepție 1	**Pacientul este externat simultan:** Sistemul blochează preluarea și notifică medicul că datele nu mai sunt disponibile.
Cerințe de calitate	
Performanță	Timpul de comutare între doi pacienți nu trebuie să depășească 1 secundă pentru datele textuale.
Fiabilitate	Persistarea asocierii medic-pacient trebuie să fie confirmată tranzacțional în baza de date.

Tabela 4.1: Descrierea detaliată a scenariului de utilizare: Gestionare pacienți

- **NFR-S4:** Token-urile au timp de expirare implementat, iar refresh token-urile sunt rotite automat și invalidate la logout.
- **NFR-S5:** Validarea riguroasă a input-ului la frontierele API (câmpuri obligatorii, formate de timestamp valide).

Confidențialitate și protecția datelor

- **NFR-P1:** Datele pacienților sunt accesibile doar personalului medical autorizat direct.
- **NFR-P2:** Minimizarea expunerii datelor; sistemul nu afișează informații care nu sunt necesare contextului clinic curent.
- **NFR-P3:** Comunicația între client și server se realizează prin protocoale criptate HTTPS/WSS (TLS).

Performanță și timp real

- **NFR-F1:** Afișarea emoțiilor se realizează aproape în timp real prin WebSocket, cu o latență minimă, în timp ce persistarea în baza de date se face periodic.
- **NFR-F2:** Limitarea backlog-ului de cadre video procesate pentru a preveni consumul excesiv de memorie RAM pe client.
- **NFR-F3:** Dashboard-ul rămâne responsiv și funcțional chiar și în perioadele de indisponibilitate a motorului de analiză AI.

Fiabilitate și disponibilitate

- **NFR-R1:** Mecanism de *fallback*: dacă InfluxDB este indisponibil, sistemul utilizează baza de date principală (Postgres) sau date sintetice pentru continuitate.
- **NFR-R2:** Interfața semnalează vizual deconectarea modelului AI, permițând utilizatorului să continue alte activități administrative.
- **NFR-R3:** Operațiile critice (Check-in/Check-out) sunt tratate tranzacțional pentru a menține consistența între starea patului și cea a pacientului.

Uzabilitate și Mentenabilitate

- **NFR-U1:** Interfața oferă feedback vizual clar pentru stările de încărcare sau eroare (Loading/Error states).
- **NFR-M1:** Arhitectură modulară bazată pe TypeScript, cu separare clară între controller, rute și servicii.
- **NFR-M3:** Aplicația este cross-platform (Windows/Linux) utilizând stack-ul Node.js și Vite.

4.2 Modelul structural

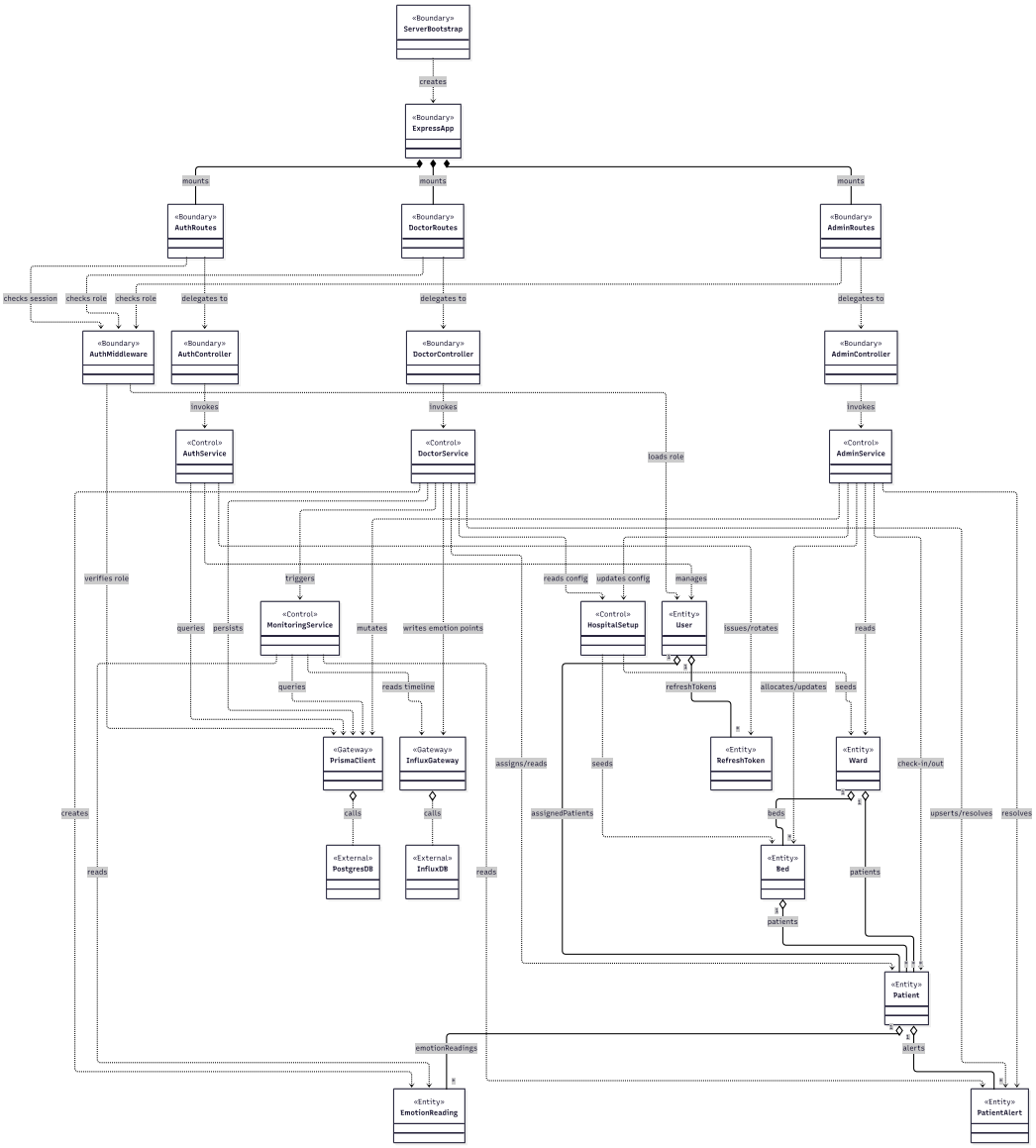
Backend

Figura 4.2a și Figura 4.2b ilustrează organizarea backend-ului bazată pe modelul arhitectural *Entity–Control–Boundary* (ECB), structură care asigură o separare clară a responsabilităților:

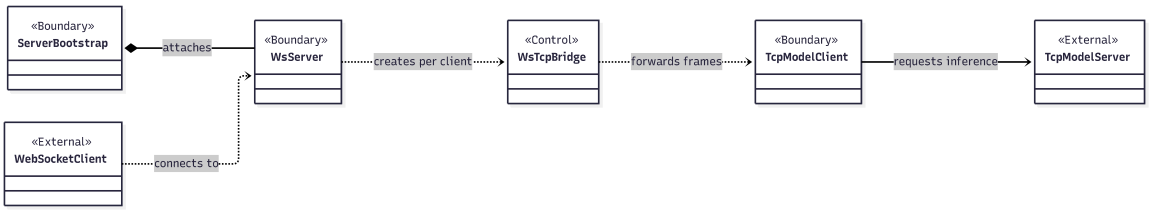
- *Entity* reprezintă nucleul domeniului, adică modelele persistente gestionate în baza de date (ex.: *User*, *Patient*, *EmotionReading*, *PatientAlert*).
- *Control* concentrează logica de business și orchestrează fluxurile de lucru (serviciile aplicației).
- *Boundary* delimitează interfața către exterior și gestionează comunicarea cu clienții (rute, controllere, middleware, servere de transport).

Pentru integrarea cu infrastructura, arhitectura este extinsă cu etichetele *Gateway* și *External*, care izolează accesul la servicii externe și decuplează logica internă de detaliile tehnice (de exemplu, accesul la baze de date). În Figura 4.2a, inițializarea aplicației este realizată de *ServerBootstrap* și *ExpressApp*, care montează rutele (*AuthRoutes*, *DoctorRoutes*, *AdminRoutes*) și aplică *AuthMiddleware* pentru validarea sesiunii și a rolurilor. La nivelul stratului de expunere HTTP, componentele de tip controller (AuthController, DoctorController, AdminController) au fost proiectate conform paradigmei de 'thin controllers' (asigurând o logică minimală de procesare) și deleagă operațiile către servicii (*AuthService*, *DoctorService*, *AdminService*), astfel încât regulile de business și validările specifice domeniului să fie centralizate și ușor de întreținut. Serviciile operează asupra entităților domeniului (de exemplu, *AuthService* gestionează *User*/*RefreshToken*, *DoctorService* gestionează *Patient*/*EmotionReading*/*PatientAlert*, iar *AdminService* gestionează *Ward*/*Bed*/*Patient*).

Persistența și integrarea sunt decuplate prin gateway-uri: *PrismaClient* mediază accesul la baza de date relațională (PostgreSQL), iar *InfluxGateway* asigură scrierea/citirea seriilor temporale în InfluxDB, pentru vizualizări și analiză istorică. Separat de fluxul HTTP, Figura 4.2b prezintă canalul de procesare în timp real pentru inferență: *WsServer* acceptă conexiuni de la clientul WebSocket al aplicației web, iar *WsTcpBridge* orchestrează fluxul dintre WebSocket și modelul extern, trimițând frame-urile prin *TcpModelClient* către *TcpModelServer*. Rezultatele inferenței (probabilități) sunt retransmise către client, păstrând responsabilitățile de transport separate de logica de business și de mecanismele de stocare.

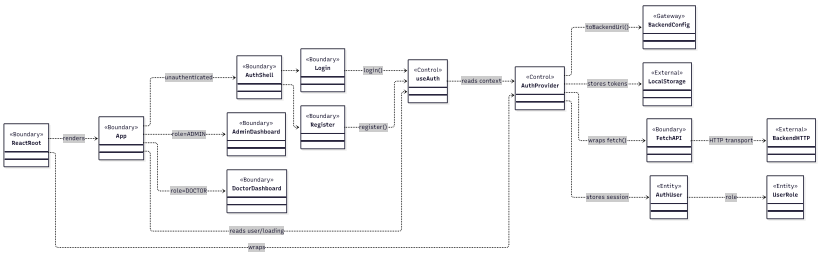


(a) API-ul HTTP: Boundary (Express/Routes/Controllers/Middleware), Control (Services) și persistența prin Gateway/External.

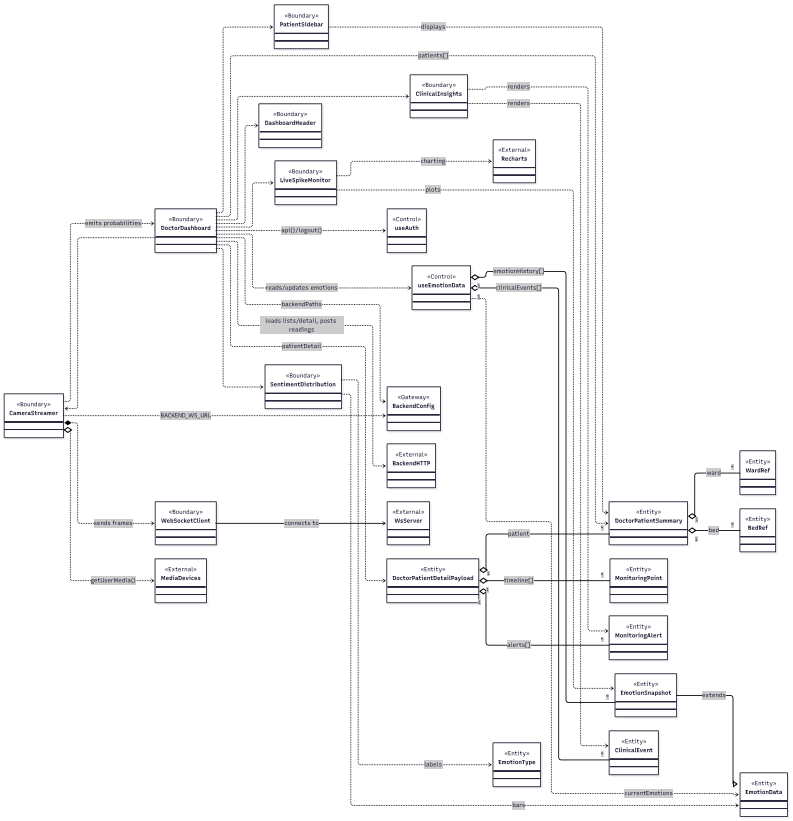


(b) Canalul de inferență în timp real: WebSocket ↔ TCP (model extern).

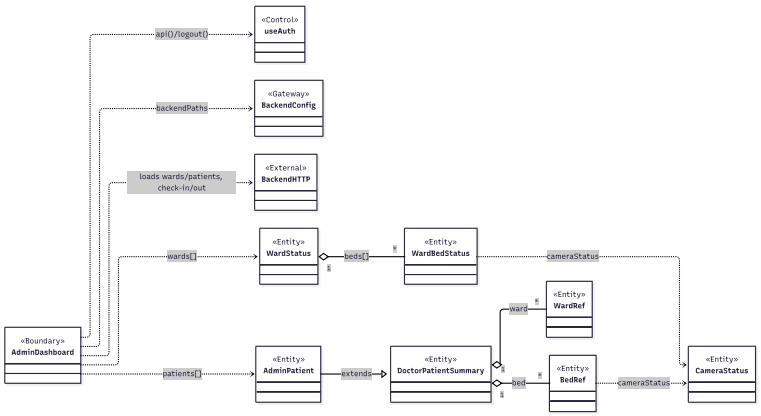
Figura 4.2: Modelul structural al backend-ului, conform ECB (Entity–Control–Boundary) extins cu Gateway și External.



(a) Bootstrap și autentificare (App Shell).



(b) Fluxul medicului: monitorizare, vizualizare și streaming live.



(c) Fluxul administrativ: management secții/paturi/pacienți.

Figura 4.3: Modelul structural al frontend-ului, conform ECB (Entity–Control–Boundary), extins cu Gateway și External.

Frontend

Figura 4.3 prezintă structura frontend-ului (React + TypeScript) folosind aceeași separare ECB ca în backend, însă cu o nuanță importantă: *Entity* reprezintă în principal **modele de date/DTO-uri și stări UI** (de ex. *AuthUser*, *DoctorPatientSummary*, *MonitoringPoint*), nu entități persistate direct în aplicația client. *Control* grupează logica de orchestrare (hook-uri precum *useAuth*, *useEmotionData* și provider-ul *AuthProvider*), iar *Boundary* include paginile și componentele UI (de ex. *App*, *DoctorDashboard*, *CameraStreamer*, *LiveSpikeMonitor*). Pentru integrare, sunt evidențiate și componente *Gateway/External*, care izolează accesul la resurse de infrastructură (configurația către backend, transport HTTP/WS, API-uri ale browserului).

În Figura 4.3a este surprinsă fundația aplicației: *ReactRoot* compune *AuthProvider* peste *App*, iar aplicația decide ruta principală în funcție de sesiune și rol. *AuthProvider* centralizează autentificarea și persistă token-urile în *LocalStorage*, oferind un API unificat pentru cereri HTTP (prin *FetchAPI* și configurarea adreselor prin *BackendConfig*). Această separare reduce duplicarea și menține componentele de UI independente de detaliile de transport.

Figura 4.3b descrie fluxul principal al medicului: *DoctorDashboard* încarcă lista de pacienți și detaliile de monitorizare din backend (DTO-uri precum *DoctorPatientDetailPayload*, *MonitoringPoint*, *MonitoringAlert*), apoi compune componente specializate pentru vizualizare și suport decizional. Starea „live" a sesiunii este gestionată de *useEmotionData* (entități UI precum *EmotionData*, *EmotionSnapshot*, *ClinicalEvent*), în timp ce *CameraStreamer* folosește *MediaDevices* pentru captură și un *WebSocketClient* dedicat pentru trimiterea frame-urilor către backend. Probabilitățile primite sunt propagate către dashboard pentru actualizarea UI și, periodic, pentru persistență prin endpoint-urile HTTP (ex. trimiterea de readings).

În Figura 4.3c este reprezentat fluxul administrativ, unde *AdminDashboard* gestionează operațiile de check-in/check-out și statusul secțiilor/paturilor folosind DTO-uri precum *WardStatus*, *WardBedStatus* și *AdminPatient*. Ca și în cazul medicului, comunicarea cu backend-ul este abstractizată prin *useAuth* și aceeași infrastructură de transport/configurare, menținând UI-ul concentrat pe logică de prezentare și pe interacțiunile specifice rolului.

Diagrama entitate–relație (ER)

Figura 4.4 prezintă modelul relațional al datelor persistente utilizate de aplicație, complementând diagramele ECB. Dacă în modelul structural *Entity* apare ca „nucleu al domeniului", diagrama ER detaliază modul în care acest nucleu este implementat în baza de date: chei primare, chei străine, constrângeri de unicitate și relațiile dintre entități (cardinalități).

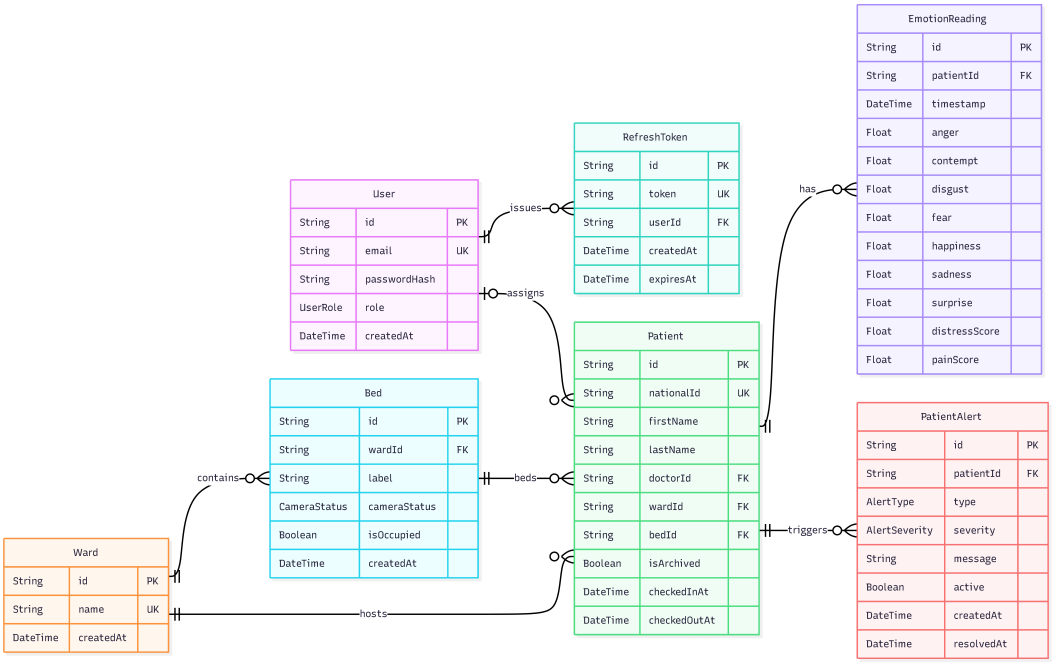


Figura 4.4: Diagrama entitate–relație (ER) a bazei de date.

Zona de autentificare este compusă din *User* și *RefreshToken*: fiecare utilizator poate emite mai multe refresh token-uri (relația *User–RefreshToken*), iar câmpul *role* (*UserRole*) diferențiază fluxurile și permisiunile din aplicație (medic vs. administrator). La nivel operațional, aceste entități susțin sesiuni persistente și controlul accesului pe roluri.

Modelarea spitalului și a monitorizării este structurată în jurul entității *Patient*, care este asociată cu o secție (*Ward*) și un pat (*Bed*), iar opțional poate fi asignată unui medic (*User*). Starea clinică în timp este capturată prin *EmotionReading* (seturi de valori pe emoții + scoruri agregate), legate de pacient, iar evenimentele de risc sunt reprezentate prin *PatientAlert* (tip, severitate, activ/rezolvat). În acest fel, baza de date relațională păstrează atât structura organizațională (secții/paturi/pacienți), cât și istoricul și semnalările necesare fluxurilor din interfața medicului și a administratorului.

4.3 Modelul dinamic al sistemului

Diagrama de secvență: Monitorizare live

Figura 4.5 descrie fluxul de monitorizare în timp real, declanșat de medic din interfața aplicației. La pornirea sesiunii, componenta de captură inițializează camera folosind API-ul browserului (*MediaDevices*) și stabilește o conexiune WebSocket către backend. Cadrele video sunt pre-procesate (redimensionare/compresie) și transmise ca mesaje binare către server, permițând un canal cu latență redusă față de un

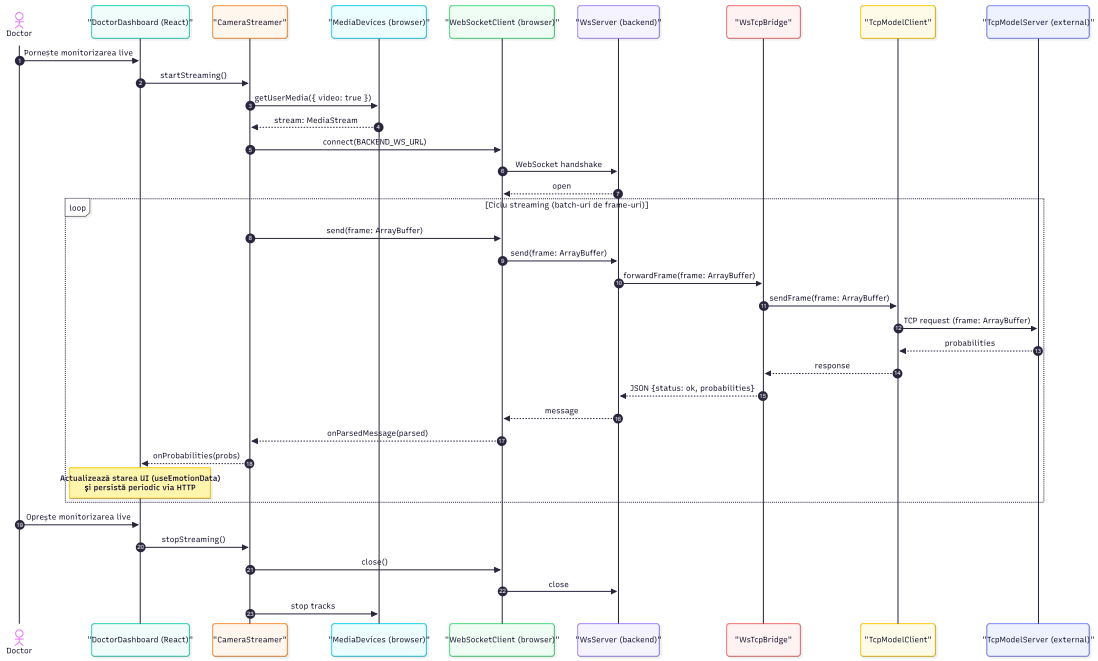


Figura 4.5: Diagrama de secvență pentru use case-ul *Monitorizare live*: captură video, transmitere prin WebSocket și inferență în model extern (TCP).

flux exclusiv HTTP.

Pe backend, serverul WebSocket primește cadrele și le direcționează către un modul de tip „bridge" care gestionează ordonarea și trimiterea lor către modelul extern. Bridge-ul asigură conectarea către serviciul de inferență (prin clientul TCP) și aplică un mecanism de coadă, astfel încât procesarea să rămână stabilă chiar și când rata de captură depășește temporar capacitatea modelului. Rezultatul inferenței (probabilități pe emoții) este apoi serializat și retransmis prin WebSocket către aplicația web.

La nivel de UI, probabilitățile primite sunt folosite pentru actualizarea stării de monitorizare (grafice, distribuții, evenimente clinice) fără a bloca interacțiunea utilizatorului. Oprirea monitorizării închide conexiunea WebSocket și eliberează resursele de captură (oprirea track-urilor camerei), închizând complet bucla de streaming.

Diagrama de comunicare: Vizualizare timeline

Figura 4.6 evidențiază participanții și dependențele dintre ei pentru afișarea timelineului clinic. Interfața medicului declanșează cererea `/detail`, autorizată prin token JWT, proces prin care backend-ul orchestrează încărcarea metadatelor pacientului și a istoricului de monitorizare. Datele istorice, denumite `timelineHistory`, sunt preluate preferențial din *InfluxDB* via *gateway*, aplicându-se un mecanism de *fallback* pe *PostgreSQL* în cazul indisponibilității bazei de date temporale. La fiecare

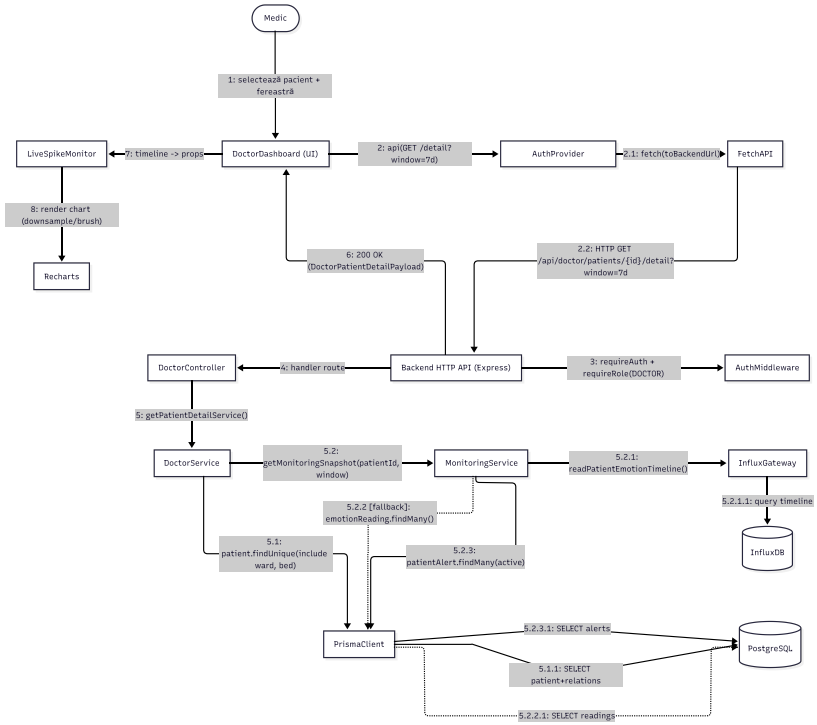


Figura 4.6: Diagrama de comunicare pentru use case-ul *Vizualizare timeline*.

nouă inferență de emoții aceste date istorice sunt actualizate prin adăugarea ultimelor probabilități primite. Pentru a asigura continuitatea vizuală a graficului și o experiență de utilizare fluidă, sistemul implementează un mecanism de eșantionare locală la nivelul interfeței. Astfel, la fiecare 500 ms, buffer-ul local de memorie, `emotionHistory` — care reține datele din ultimele 65 de minute — este actualizat prin adăugarea de noi puncte de date. În intervalele în care nu sunt recepționate cadre noi de la motorul de inferență AI, valorile emoțiilor sunt menținute constante, corespunzând ultimelor probabilități identificate. Matematic 4.1, acest comportament este echivalent cu un filtru de menținere de ordin zero (*Zero-Order Hold*), unde valoarea funcției de stare $E(t)$ rămâne invariantă între două momente de detecție succesive t_n și t_{n+1}.

$$E(t) = E(t_n), \quad \text{pentru} \quad t_n \leq t < t_{n+1} \tag{4.1}$$

În final, payload-ul vizualizat de medic reprezintă o concatenare (*mergedMonitorHistory*) a celor două fluxuri: `timelineHistory` (datele persistate) și `emotionHistory` (datele curente din memoria volatilă). Acest set de date unificat este mapat către componenta *LiveSpikeMonitor*, care utilizează biblioteca *Recharts* pentru randarea graficului în timp real, oferind o rezoluție de 0.5 secunde. Un exemplu vizual intuitiv este oferit în Figura 4.7. Alertele actualizate sunt folosite de mecanismul de notificări pentru a anunța doctorul schimbări importante legate de starea emoțională a pacientului.



Figura 4.7: Exemplu de vizualizare timeline.

Diagrama de deployment

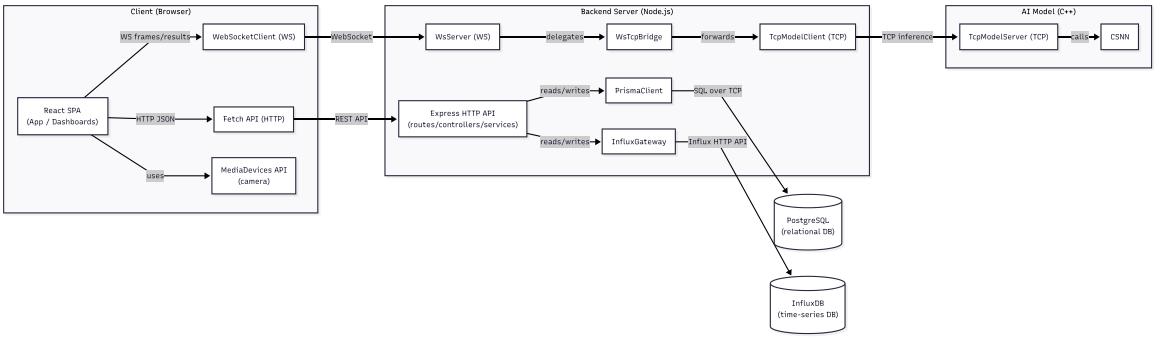


Figura 4.8: Diagrama de deployment: noduri de execuție și protocoale de comunicare (REST/HTTP, WebSocket, TCP).

Figura 4.8 evidențiază distribuția fizică a sistemului: aplicația web rulează în browser (interfața React) și comunică cu backend-ul Node.js atât sincron prin REST/HTTP (pentru operații CRUD și încărcarea datelor), cât și asincron prin WebSocket (pentru streaming de cadre și rezultate). Backend-ul accesează persistența relațională în PostgreSQL prin Prisma și utilizează InfluxDB ca depozit time-series pentru timeline-uri, iar inferența este externalizată către un server de model dedicat, accesat prin TCP. Între TCP și WS există o punte de legătură (*TsTcpBridge*) ce asigură comunicarea între cele 2 protocoale.

Motivația utilizării PostgreSQL + InfluxDB

Pentru persistență am ales o abordare hibridă, separând datele **tranzacționale** (structurate, cu relații) de datele **time-series** (flux continuu de puncte indexate temporal). Această separare reduce cuplarea dintre workload-ul de tip OLTP (operare zil-

nică, integritate, actualizări) și workload-ul de tip analiză temporală (range queries, agregări pe ferestre).

- **PostgreSQL (bază relațională, sursa de adevăr pentru domeniu clinic).** PostgreSQL este utilizat pentru entitățile cu relații clare și reguli de consistență: utilizatori și roluri, pacienți, alocarea în salon/pat, stări operaționale, respectiv alertele persistente. Acest tip de date beneficiază direct de: *ACID transactions*, constrângeri (chei primare/străine), modelare normalizată, interogări cu *JOIN*-uri și migrații controlate ale schemei. Într-un context clinic, aceste proprietăți sunt importante pentru a evita stări invalide (ex. pacient fără pat/ward valid sau alerte asociate greșit) și pentru a păstra un istoric coerent.

- **InfluxDB (TSDB pentru timeline-uri și interogări pe intervale de timp).** Pentru monitorizarea emoțiilor, aplicația produce un volum mare de puncte cu timestamp (emoții + scoruri derivate), interogate aproape exclusiv pe ferestre de timp (ex. ultimele 24h sau 7 zile) și afișate ca grafice. O bază de date time-series este optimizată pentru acest tip de acces: (i) scrieri frecvente și eficiente, (ii) compresie și stocare eficientă pentru date temporale, (iii) interogări rapide de tip *range scan* ("toate punctele dintre t_0 și t_1"), (iv) posibilitatea de a aplica politici de retenție și/sau downsampling fără a afecta modelul relațional.

Eficiență și izolare a performanței. Deși datele time-series pot fi păstrate și într-o bază relațională, pe măsură ce crește frecvența de eșantionare și retenția, tabelele mari pot introduce costuri suplimentare (indexuri mai voluminoase, operații de mentenanță, impact asupra latențelor OLTP). Prin utilizarea InfluxDB pentru timeline-uri, încărcarea asociată interogărilor temporale este mutată către un motor dedicat, menținând backend-ul și baza relațională responsive pentru operațiile critice (CRUD și tranzacții).
Robustețe operațională (fallback). Arhitectura permite ca timeline-ul să fie citit cu prioritate din InfluxDB (dacă este disponibil și are puncte), iar în caz contrar din persistența relațională; astfel, sistemul rămâne funcțional chiar și în situații de configurare incompletă sau indisponibilitate temporară a depozitului time-series.

Diagrama de stare: *PatientAlert*

Figura 4.9 descrie ciclul de viață al unei alerte clinice (*DISTRESS/PAIN*) asociate unui pacient. Pentru optimizarea mecanismului de declanșare a alertelor și eliminarea fenomenului de instabilitate (*jitter*) la pragurile de decizie, a fost implementat un algoritm de **histerezis** detaliat în Figura 4.10. Acesta utilizează praguri diferențiate pentru activarea ($T_{high} = 70$) și dezactivarea ($T_{low} = 55$) stării de alertă,

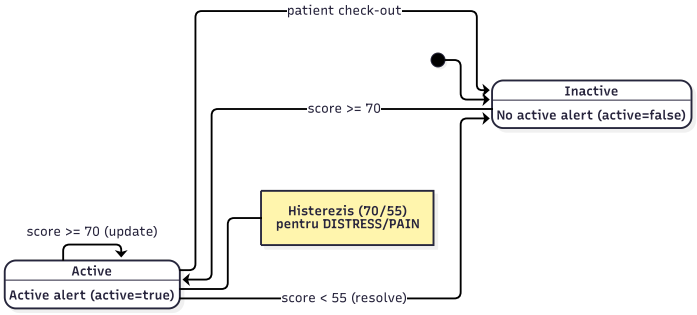


Figura 4.9: Diagrama de stare pentru *PatientAlert* (per pacient și tip de alertă).

asigurând astfel o stabilitate temporală a monitorizării și reducând oscilațiile frecvente ale stării.

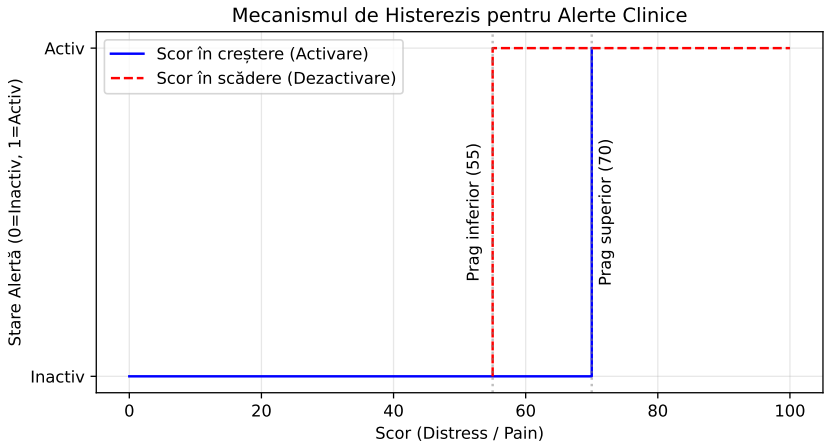


Figura 4.10: Reprezentarea grafică a mecanismului de histerezis aplicat sistemului de alerte. Activarea se produce la depășirea pragului de 70, în timp ce revenirea la starea inactivă necesită scăderea scorului sub 55.

Calculul severității se bazează pe două metrici fundamentale, derivate din vectorul stărilor emoționale detectate (V). Prima metrică, scorul de **Distress** (S_D), este calculată ca medie aritmetică a stărilor de furie, frică și tristețe, conform ecuației (4.2):

$$S_D = \text{clamp}\left(\frac{V_{\text{anger}} + V_{\text{fear}} + V_{\text{sadness}}}{3}, 0, 100\right) \tag{4.2}$$

Cea de-a doua metrică, scorul de **Pain** (S_P), reprezintă o medie ponderată ce prioritizează frica și tristețea ca indicatori non-verbali ai suferinței fizice, fiind definită în relația (4.3):

$$S_P = \text{clamp}\left(0.45 \cdot V_{\text{fear}} + 0.4 \cdot V_{\text{sadness}} + 0.15 \cdot V_{\text{disgust}}, 0, 100\right) \tag{4.3}$$

În scenariul în care ambele scoruri calculate prin formulele (4.2) și (4.3) depășesc pragul critic de activare (T_{high}), sistemul aplică o logică de arbitrare prin selecția

valorii maxime. Astfel, intensitatea și eticheta alertei sunt determinate de scorul final (S_{final}), determinat conform relației (4.4):

$$S_{final} = \max(S_D, S_P) \tag{4.4}$$

O alertă trece în starea *Active* atunci când valoarea rezultată din ecuația (4.4) respectă condiția $S_{final} \geq 70$. În timpul stării active, severitatea și mesajul sunt actualizate dinamic pe baza noilor citiri. Pentru a evita declanșarea repetitivă a notificărilor la fluctuații minore, rezolvarea alertei se produce doar atunci când $S_{final} < 55$, moment în care alertarea este dezactivată și se înregistrează timpul de rezolvare. Toate aceste actualizări au loc la persistarea unui nou reading de emoții, backendul calculând severitatea pe baza scorului:

- `HIGH` dacă score ≥ 85,
- `MEDIUM` dacă score ≥ 72,
- `LOW` în caz contrar.

În interfață, aceste alerte sunt afișate în *Clinical Insights* la secțiunea *Current Alerts* (Figura 4.11) și reprezintă starea clinică persistată pentru pacient (pe intervalele recente de funcționare).

Sistemul calculează și alerte **derivate** (nepersistate), pe baza unui *timeline* de emoții pe o fereastră de timp (ex. 24h sau 7d). Aceste alerte urmăresc episoade susținute, nu doar vârfuri scurte:

- Se generează o alertă de tip `DISTRESS` sau `PAIN` dacă scorul corespunzător rămâne peste **65%** timp de cel puțin **60 minute**.
- Mesajul include durata episodului și vârful (peak) atins în acea perioadă.
- Severitatea este calculată după aceeași logică (HIGH/MEDIUM/LOW) în funcție de peak.

În interfață, aceste alerte apar în *Clinical Insights* la secțiunea *Prolonged Alerts* (Figura 4.12) și sunt recalculte la fiecare obținere a unui nou *snapshot* de monitorizare din backend.

Diagrama de activități: Check-in pacient

Figura 4.13 prezintă fluxul operațional de internare (check-in) și deciziile principale care garantează consistența datelor: validarea câmpurilor obligatorii, verificarea existenței patului, compatibilitatea pat–secție (când se indică o secție), și prevenirea ocupării simultane. Operația este tratată tranzacțional, astfel încât înregistrarea/actualizarea pacientului și marcarea patului ca ocupat să fie aplicate atomic. În cazul unui pacient deja internat se returnează un răspuns de tip conflict, iar pentru erori neprevăzute se semnalizează eșecul operației.

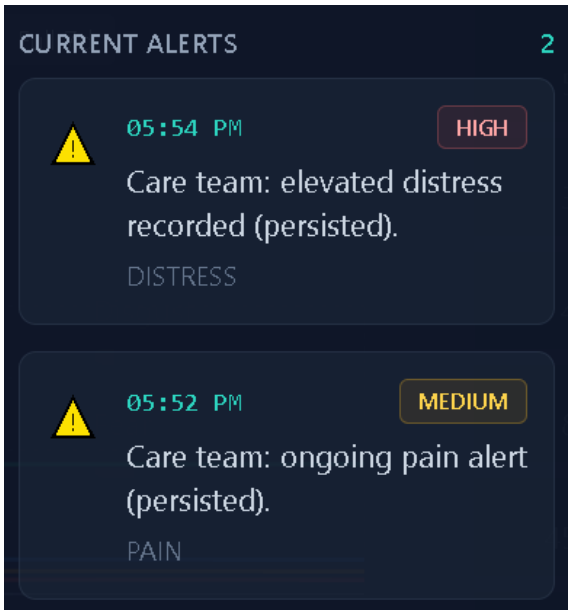


Figura 4.11: Secțiunea *Current Alerts* - vizualizarea alertelor persistente.

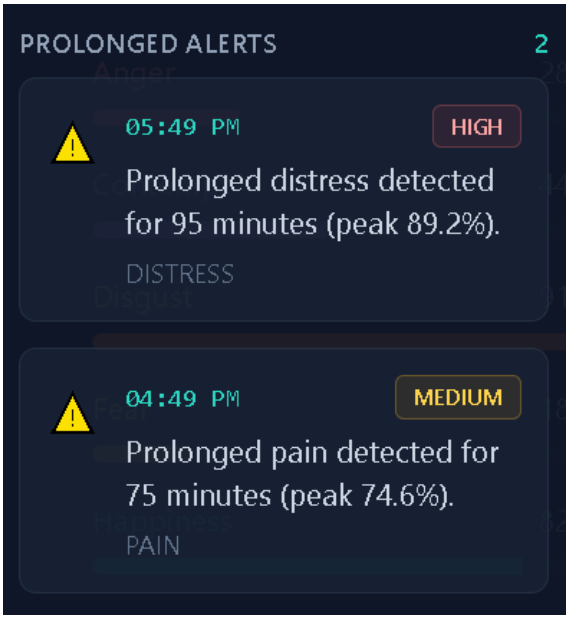


Figura 4.12: Secțiunea *Prolonged Alerts* - vizualizarea alertelor de durată.

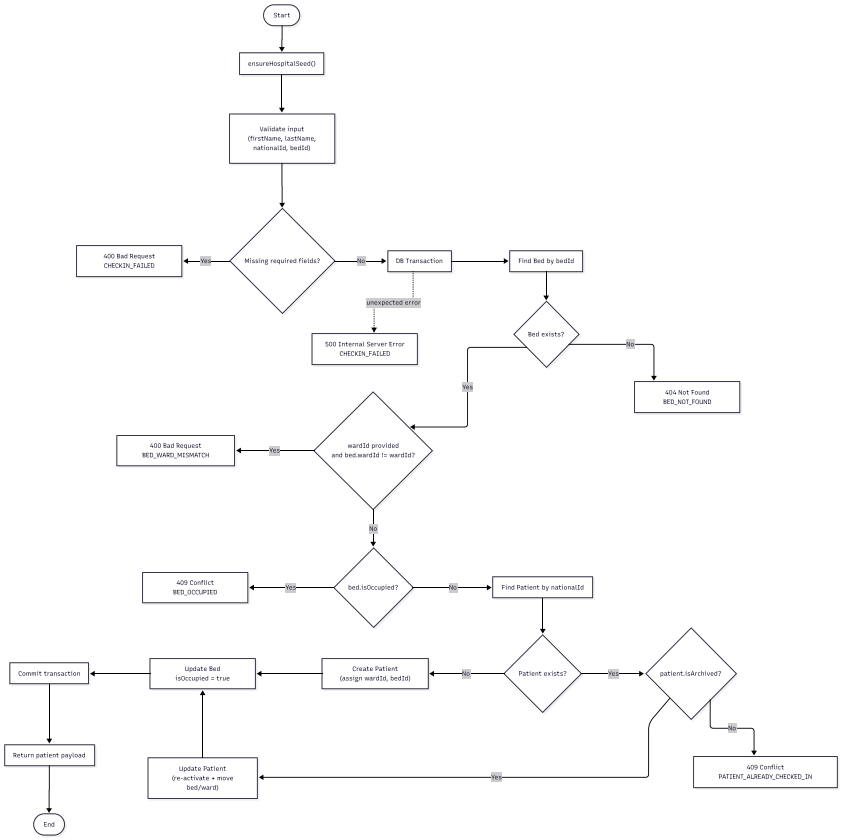


Figura 4.13: Diagrama de activități pentru use case-ul *Check-in pacient*.

4.4 Sistemul de alerte și notificări

Aplicația integrează un mecanism multi-nivel de alertare, destinat în principal utilizatorului cu rol de medic. Semnalele sunt generate atât **în timp real** (din probabilitățile inferate de model), cât și **la nivel backend** (din istoricul pacientului și alertele persistate). În plus, există notificări operaționale (erori/stări de funcționare).

Evenimente clinice în sesiune (Live Session Events)

La recepționarea fiecărui vector de probabilități de la modelul de inferență, valorile emoționale sunt supuse unui proces de normalizare în intervalul $[0, 100]$, fiind ulterior redate în interfața medicului sub formă de procente. Această vizualizare a emoțiilor curente, exemplificată în Figura 4.14, oferă o analiză granulară a profilului afectiv, permițând identificarea rapidă a stărilor dominante și a nuanțelor emoționale ale pacientului. Simultan cu afișarea brută a datelor, sistemul execută un mecanism de analiză euristică pentru identificarea episoadelor emoționale semnificative. Se determină emoția dominantă (E_{max}) prin identificarea valorii maxime din setul de probabilități curente. Dacă această valoare depășește pragul critic de activare, stabilit la un nivel de încredere de 60%, sistemul generează automat un eveniment clinic (*Clinical Event*), conform logicii descrise în relația (4.5):

$$\text{Trigger} = \begin{cases} \text{generare eveniment}, & \text{daca } \max(V_{\text{emotii}}) > 60 \\ \text{stare latenta}, & \text{altfel} \end{cases} \tag{4.5}$$

Structura și comportamentul acestor evenimente sunt definite prin următoarele caracteristici tehnice, așa cum reiese și din detaliile surprinse în figurile de mai jos:

- **Trigger**: $\max(\text{emotii}) > 60\%$.
- **Conținut**: emoția dominantă, intensitatea (în %), timestamp și un mesaj selectat aleator din 3 variante predefinite (pentru a evita repetitivitatea).
- **Afișare**: evenimentele apar în panoul *Clinical Insights* la secțiunea *Live Session Events*.
- **Feedback vizual**: emoția detectată este evidențiată temporar (efect de tip *flash/pulse*).

Indicatori rezumați în UI

Pentru a facilita monitorizarea rapidă și eficientă a stării pacientului, interfața grafică a aplicației integrează o serie de indicatori sintetici care agregă datele brute în metrici vizuale de nivel înalt. Aceștia au rolul de a traduce fluxurile complexe de date emoționale în informații intuitive și direct acționabile pentru cadrul clinic. În



Figura 4.14: Vizualizarea emoțiilor și a intensităților curente.

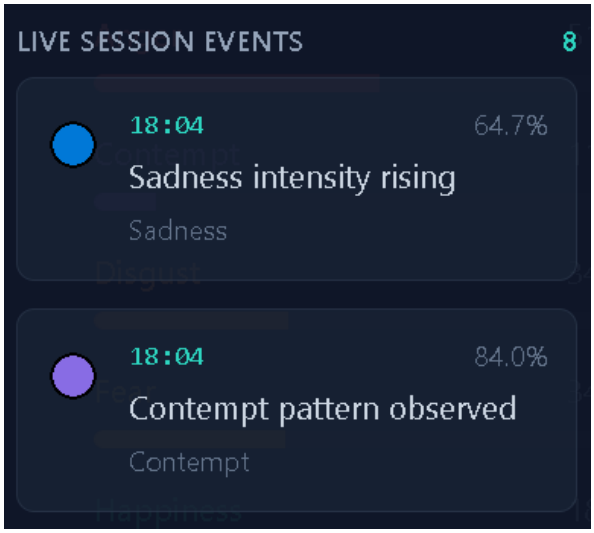


Figura 4.15: Detaliu al panoului *Clinical Insights* cu evenimentele generate.

continuare, sunt detaliate logica matematică de calcul și modalitățile de reprezentare în interfață pentru cele două componente principale: nivelul de risc clinic (*Clinical Risk Level*) și tipul sesiunii (*Session Type*).

Clinical Risk Level (Low / Medium / High)

În *Patient Sidebar* se afișează nivelul rezumat de risc clinic (*Clinical Risk Level*), derivat din media emoțiilor negative:

$$\text{avgNegative} = \frac{\text{Anger} + \text{Fear} + \text{Sadness}}{3} \tag{4.6}$$

$$\text{riskLevel} = \begin{cases} \text{High}, & \text{dacă avgNegative} > 50 \\ \text{Medium}, & \text{dacă avgNegative} > 25 \\ \text{Low}, & \text{altfel} \end{cases} \tag{4.7}$$

Exemplele vizuale pentru cele trei niveluri sunt prezentate în Figura 4.16.



(a) Low



(b) Medium



(c) High

Figura 4.16: Nivelul de risc clinic (Clinical Risk Level) în cele trei stări.

Session Type (normal / positive / high-risk)

În header-ul aplicației (lângă titlul platformei) este afișat *Session Type*, care sintetizează rapid starea sesiunii. Acesta este determinat combinând alertele clinice și emoțiile curente:

- **High-risk**: dacă există cel puțin o alertă cu severitate `HIGH` (din *Current* sau *Prolonged Alerts*) **sau** dacă Anger > 60 și Fear > 60.
- **Positive**: dacă Happiness > 50 și Surprise > 30.
- **Normal**: în celelalte situații.

Exemplele vizuale pentru cele trei tipuri sunt prezentate în Figura 4.17.



(a) Normal (b) Positive (c) High-risk

Figura 4.17: Tipul sesiunii (*Session Type*) afișat în header, în cele trei stări.

Notificare la schimbarea nivelului de risc

Sistemul urmărește dinamica lui *riskLevel* și notifică medicul **doar când există o schimbare**:

- **Risk Increased** (degradare): când nivelul trece de la Low → Medium/High sau Medium → High.
- **Risk Decreased** (îmbunătățire): când nivelul scade (High → Medium/Low sau Medium → Low).
- Dacă nivelul rămâne neschimbat, **nu se afișează notificare**.

Exemplele vizuale ce apar timp de 5 secunde în partea dreaptă de sus pentru cele două situații sunt prezentate în Figura 4.18.



(a) Notificare de degradare (*Risk Increased*). (b) Notificare de îmbunătățire (*Risk Decreased*).

Figura 4.18: Notificarea afișată la schimbarea nivelului de risc.

Notificări operaționale (erori și stări de funcționare)

Pe lângă alertarea clinică, aplicația comunică și stări/erori operaționale prin mesaje vizuale:

- **Autentificare**: erori la login/înregistrare (mesaj afișat în formular).
- **Operații de administrare/doctor**: erori la încărcarea listelor sau la acțiuni (ex. preluare pacient, check-in/check-out), afișate ca mesaj de eroare în UI.
- **Stare conexiune către model**: indicatori de tip *connected/disconnected*, respectiv stări de captură (*recording/idle*), pentru a confirma funcționarea pipeline-ului cameră–model.

4.5 Testarea aplicației

Testare white-box

Metoda analizata

Analiza white-box a fost realizată pe metoda `derivePrologedAlert` din serviciul de monitorizare, care se ocupă cu detectarea perioadelor de suferință prelungită (durere sau stres) ale unui pacient, analizând un istoric de date (timeline). Această funcție include ramificatii multiple, mai multe iterații și o funcție auxiliară (`flushStreak`) care influențează direct decizia de generare a alertelor.

Complexitate ciclomatrică

Pentru metoda analizată, au fost identificate 8 predicate de decizie (D), care determină ramificarea fluxului de execuție, după cum se poate remarca în Tabelul 4.2.

Determinarea complexității ciclomatrice (CC) se bazează pe teoria grafurilor aplicată fluxului de control al programului. Relația fundamentală utilizată este formula lui McCabe, introdusă de Thomas McCabe în lucrarea sa fundamentală din 1976 [McC76], fiind definită în ecuația 4.8:

$$CC = E - N + 2P, \tag{4.8}$$

unde variabilele reprezintă: E – numărul de muchii (edges) din graful de flux; N – numărul de noduri (nodes); P – numărul de componente conexe (pentru o singură metodă, $P = 1$).

Aplicând această metodologie asupra grafului de flux de control (CFG) generat - ilustrat în Figura 4.19, complexitatea a fost calculată prin trei metode convergente:

ID	Condiție (Predicat)	Descriere / Rol în fluxul de execuție
D_1	`timeline.length === 0`	Verificarea listei vide pentru *early return*.
D_2	`timeline.forEach`	Controlul iterațiilor prin punctele cronologiei.
D_3	`value >= threshold`	Condiția de depășire a pragului (început/continuare *streak*).
D_4	`streakStart < 0`	Identificarea inițierii unui segment nou de alertă.
D_5	`streakStart < 0`	Verificarea existenței unui *streak* activ la apelul `flushStreak`.
D_6	`streakStart < 0`	Verificarea *streak*-ului rămas deschis după finalizarea buclei.
D_7	`durationMinutes >= 60`	Validarea duratei minime necesare pentru generarea alertei.
D_8	`durationMinutes >= 60`	Validarea duratei pentru *streak*-ul procesat la finalul execuției.

Tabela 4.2: Predicatele de decizie identificate pentru funcția de alertare.

- CC = numărul de regiuni planare ale grafului = 9
- CC = D + 1 = 8 + 1 = 9
- Conform ecuației 4.8: CC = 37 - 30 + 2 * 1 = 9

Pentru CFG-ul metodei, complexitatea ciclomatrica este $CC = 9$, deci exista 9 drumuri independente teoretice care trebuie parcurse în faza de testare unitară pentru a asigura o acoperire completă la nivel de decizie. În Tabelul 4.3 este un set practic de drumuri independente descrise prin semnătura deciziilor $D_1 \dots D_8$.

Pentru funcția analizată, setul de teste white-box, ilustrat în Tabelul 4.4, a fost construit astfel încât fiecare drum independent din CFG să fie acoperit de cel puțin un test.

4.6 Testarea de integrare (Top-down, Depth-first)

Validarea sistemului a fost realizată printr-o abordare incrementală de tip *Top-down, Depth-first*. Această metodă a permis verificarea timpurie a fluxului critic de date, asigurându-se că integrarea componentelor respectă arhitectura definită și că datele circulă corect de la interfața HTTP până la nivelul de persistență.

Obiectivul principal a fost validarea completă a endpoint-ului `POST /api/doctor/patients/:patientId/readings`, care reprezintă nucleul funcțional al aplicației (colectarea de telemetrie și generarea de alerte). Strategia adoptată a presupus integrarea progresivă a componentelor conform următorului lanț de dependențe:

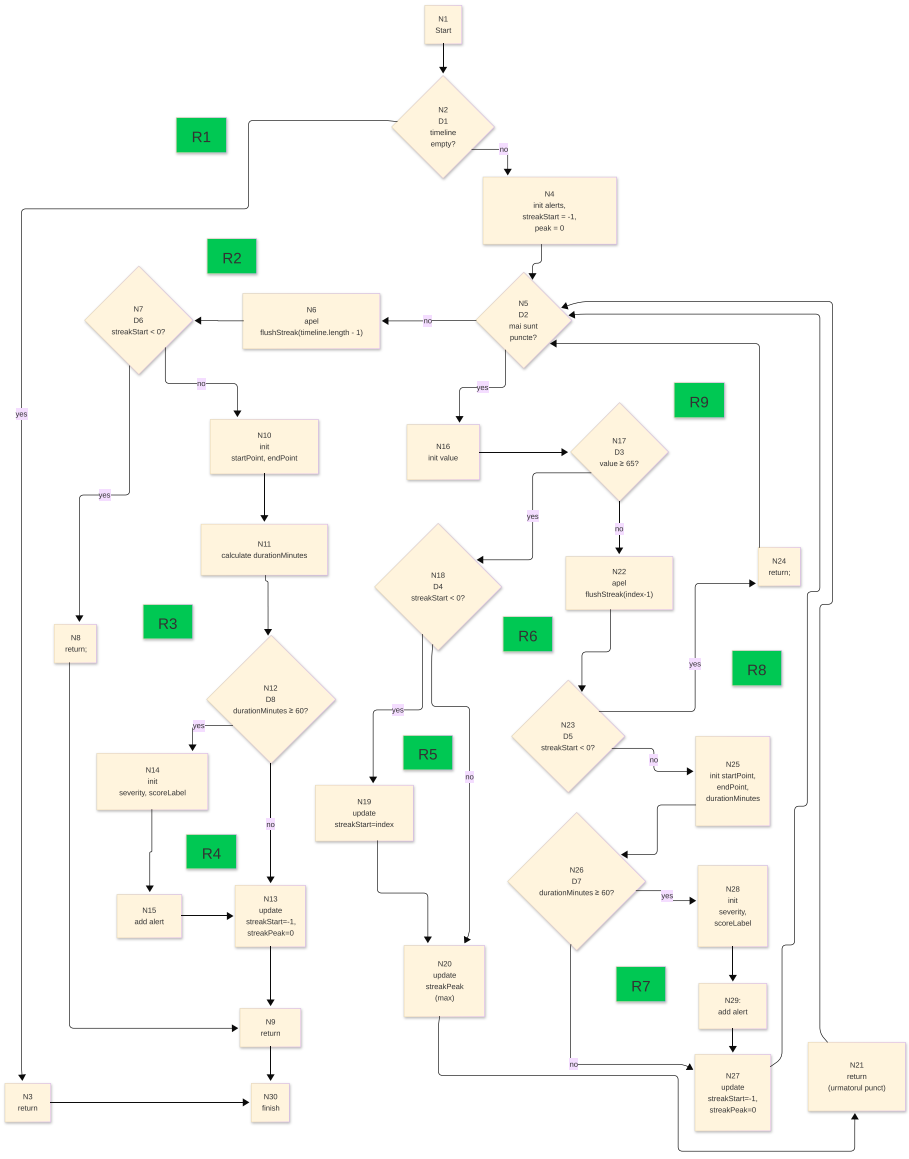


Figura 4.19: CFG pentru `derivateProlongedAlert`.

Tabela 4.3: Maparea corectată a cazurilor de testare (White-box)

Scenariu	**Descriere**	**Condiții Logice Corecte**
R1	Early return (Timeline gol)	$D_1 = T$
R2	Final cu streak lung (Alertă creată la flush-ul final)	$D_1 = F, D_2 = F, D_6 = F, D_8 = T$
R3	Nicio activitate (loop terminat, niciun streak găsit)	$D_1 = F, D_2 = F, D_6 = T$
R4	Final cu streak scurt (Fără alertă la flush-ul final)	$D_1 = F, D_2 = F, D_6 = F, D_8 = F$
R5	Start Streak Nou (Prima valoare ≥ 65)	$D_1 = F, D_2 = T, D_3 = T, D_4 = T, D_2 = F, D_6 = T$
R6	Ignorare valoare mică (Sub prag, dar nu eram în streak)	$D_1 = F, D_2 = T, D_3 = F, D_5 = T, D_2 = F, D_6 = T$
R7	Continuare Streak (Valoare ≥ 65, dar deja în streak)	$D_1 = F, D_2 = T, D_3 = T, D_4 = F, D_2 = F, D_6 = T$
R8	Închidere streak lung (Sub prag, alertă creată în buclă)	$D_1 = F, D_2 = T, D_3 = F, D_5 = F, D_7 = T, D_2 = F, D_6 = T$
R9	Închidere streak scurt (Sub prag, streak-ul nu are 60 min)	$D_1 = F, D_2 = T, D_3 = F, D_5 = F, D_7 = F, D_2 = F, D_6 = T$

Tabela 4.4: Corespondența testelor white-box pe drumuri (input / output așteptat)

Test	**Drum**	**Timeline (minute, scor)**	**Tip**	**Prag**	**Output**
P1	R1	[]	DISTR.	65	[]
P2	R6 → R3	$[(0, 20)]$	DISTR.	65	[]
P3	R5 → R4	$[(0, 70)]$	DISTR.	65	[]
P4	R5 → R7 → R2	$[(0, 70), (65, 80)]$	DISTR.	65	1 alertă; t_{65}
P5	R5 → R7 → R8 → R3	$[(0, 70), (75, 80), (80, 20)]$	DISTR.	65	1 alertă; t_{75}
P6	R5 → R7 → R9 → R3	$[(0, 70), (30, 80), (45, 20)]$	DISTR.	65	[]

Notă: În coloana „Drumuri asociate", am listat succesiunea de ramificări logice prin care trece execuția. De exemplu, R5 → R7 → R2 înseamnă: deschidere streak în buclă , actualizare peak la a doua iterație și generare alertă la ieșirea finală din funcție.

App → Route → Middleware → Controller → Service → {Prisma/Postgres, InfluxDB}

Procesul a fost structurat în cinci etape distincte, detaliate în **Tabelul 4.5**. Fiecare pas a vizat extinderea razei de testare prin înlocuirea progresivă a componentelor simulate (*stubs* și *mocks*) cu implementări reale, până la obținerea unui sistem complet funcțional.

Tabela 4.5: Strategie Depth-first: pași, focus și componente simulate

Step	**Nivel introdus**	**Ce testezi efectiv (Focus)**	**Componente simulate**
1	Entry Point & Security	App + Route + Middleware. Verificarea autentificării și autorizării la nivel de rută.	Controller (Stub)
2	Orchestrare API	Integrarea Controller-ului: extragerea parametrilor și delegarea către logica de business.	Service (Stub)
3	Business Logic	Integrarea Service-ului real: calculul scorului de distress și logica de declanșare a alertelor.	Prisma + Influx (Mocks)
4	Data Access	Integrarea cu Postgres (via Prisma): persistența reală a citirilor și alertelor.	Influx (Mock)
5	Time-series / Telemetry	Integrarea cu InfluxDB: flux complet de scriere și citire a metricilor pe axa temporală.	Fără (Sistem complet)

Pentru fiecare etapă definită anterior, au fost stabilite seturi riguroase de date de intrare și rezultate așteptate. Această trasabilitate asigură faptul că nicio componentă nouă introdusă în lanț nu alterează logica validată anterior.

Corelarea dintre input-ul simulat și răspunsul sistemului pentru fiecare pas de integrare este sintetizată în **Tabelul 4.6**. Se observă tranziția de la un simplu răspuns de tip *hook* (Step 1) către o validare complexă a stării bazei de date și a consistenței telemetriei (Step 5).

Mediu de Testare și Infrastructură

Pentru a garanta fidelitatea testelor față de mediul de producție, s-a renunțat la bazele de date *in-memory* în favoarea unor instanțe reale rulate în containere. Suita de teste utilizează următoarele instrumente:

- **Jest & Supertest:** Cadru de testare și bibliotecă pentru simularea cererilor HTTP fără pornirea manuală a serverului.

Tabela 4.6: Input și output așteptat pe pașii de integrare

Step	Input (rezumat)	Output așteptat (rezumat)
1	Request HTTP `POST` cu JWT valid.	Status `202` și confirmarea atingerii rutei.
2	Request HTTP cu payload valid.	Status `201` și apelarea service-ului cu parametrii corecți.
3	Payload ce induce scor de distress ridicat.	Status `201` și validarea logică a calculului de alertă.
4	Request cu Postgres activ în container.	Status `201` și persistența confirmată în tabelele SQL.
5	Secvență `POST` urmată de `GET`.	Status `200` și timeline consistent extras din InfluxDB.

- **Testcontainers:** Gestionarea automată a ciclului de viață pentru containerele Docker (`postgres:16-alpine` și `influxdb:2.7-alpine`).
- **Prisma Migrations:** Sincronizarea automată a schemei bazei de date înainte de execuția testelor prin `prisma migrate deploy`.

Această infrastructură permite izolarea completă a fiecărui test, asigurând un mediu curat (prin mecanisme de *cleanup* post-test) pentru fiecare iterație a strategiei *depth-first*.

4.7 Manual de utilizare al aplicației

Aplicația **Neuro-Guardian** este o platformă web de monitorizare clinică asistată de analiză emoțională, destinată utilizării în context spitalicesc. Utilizarea este organizată pe două roluri:

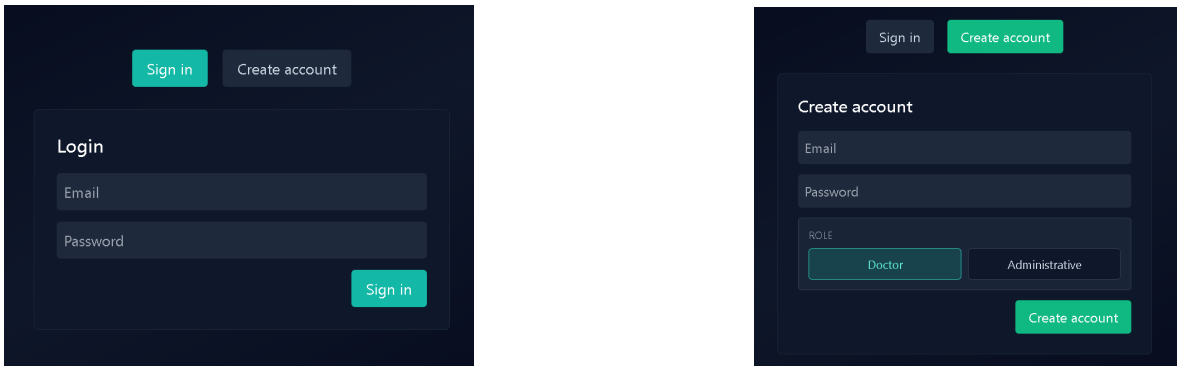
- **ADMIN** (personal administrativ): gestionează internarea/externarea pacienților și alocarea lor pe secție/pat.
- **DOCTOR** (personal medical): preia pacienți disponibili și urmărește tabloul de monitorizare (vizualizări, alerte, trenduri).

Aplicația se accesează dintr-un browser modern (Chrome/Edge/Firefox), prin adresa pusă la dispoziție în rulare locală). După autentificare, interfața este afișată automat în funcție de rol (ADMIN sau DOCTOR).

Autentificare și creare cont

Pentru autentificare se introduce adresa de e-mail și parola, apoi se apasă butonul *Sign in*. În cazul unor date invalide, aplicația afișează un mesaj de eroare.

Pentru înregistrare se introduc e-mailul și parola, apoi se selectează rolul (*Doctor* sau *Administrative*) și se apasă *Create account*. Într-un scenariu real, conturile cu rol administrativ pot fi restricționate conform politicilor instituției.



(a) Interfața de autentificare (b) Interfața de creare cont

Figura 4.20: Ecranele de autentificare și creare cont.

Modul ADMIN: Administrative Dashboard

După autentificare cu rolul ADMIN, utilizatorul accesează *Administrative Dashboard*, care include următoarele funcționalități principale:

Reîmprospătare și deconectare

În partea superioară există:

- **Refresh**: reîncarcă listele de secții/paturi și pacienții activi.
- **Logout**: închide sesiunea curentă.

Internare pacient (Patient Check-in)

Secțiunea *Patient Check-in* permite înregistrarea unui pacient activ în sistem:

1. Se completează *First name*, *Last name* și *National ID / CNP*.
2. Opțional, se completează *Diagnosis* și *Medical history*.
3. Se selectează *Ward* (secția) și *Bed* (patul disponibil).
4. Se apasă *Check in patient*.

Aplicația nu permite alocarea unui pat deja ocupat (paturile disponibile sunt filtrate automat).

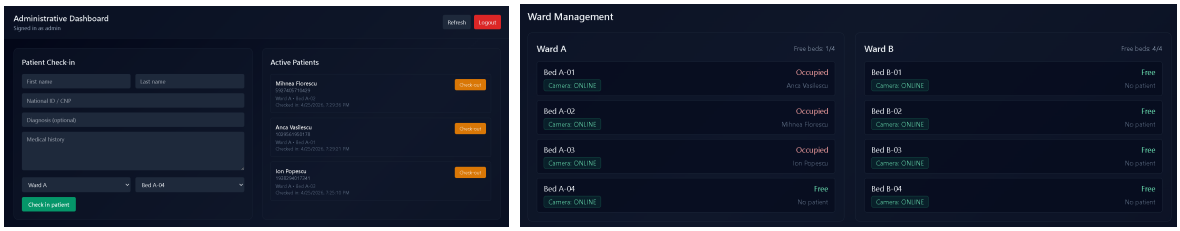
Pacienți activi (Active Patients) și externare (Check-out)

În *Active Patients* se afișează pacienții internați (activi), împreună cu secția/patul și momentul internării. Pentru externarea unui pacient se apasă butonul *Check-out* corespunzător.

Management secții/paturi (Ward Management)

Secțiunea *Ward Management* oferă o vedere de ansamblu asupra:

- gradului de ocupare (paturi libere/ocupate),
- stării camerei asociate patului (*ONLINE/OFFLINE/MAINTENANCE*),
- pacientului asociat (dacă patul este ocupat).



(a) Dashboard administrativ (b) Managementul secției și paturilor

Figura 4.21: Modul ADMIN: internare/externare și managementul secției.

Modul DOCTOR: Doctor Panel și Monitoring Dashboard

După autentificare cu rolul DOCTOR, utilizatorul poate gestiona pacienții disponibili și poate accesa tabloul de monitorizare.

Lista pacienților și preluare (Take patient)

În *Doctor Panel* sunt afișate două liste:

- **Available Patients**: pacienți nealocați; pot fi preluați prin *Take patient*.
- **My Patients**: pacienți alocați doctorului; pot fi selectați pentru monitorizare.

Butonul *Refresh patients* reîncarcă listele. Dacă nu există pacienți disponibili/ai doctorului, aplicația afișează mesaje de tip *No unassigned patients right now* sau *Pick a patient from the available list to start monitoring*.

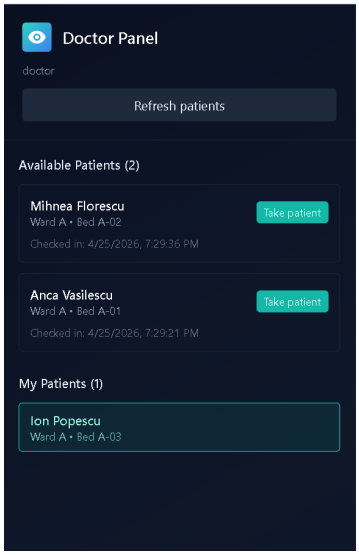


Figura 4.22: Modul DOCTOR: preluarea și selectarea pacienților.

Tabloul de monitorizare (Monitoring Dashboard)

După selectarea unui pacient, aplicația afișează tabloul de monitorizare, care include:

- **Header de sesiune**: status (*Normal Session*, *High Risk: Agitation*, *Positive Engagement*), butoanele *Live Monitor*, *Insights*, afișare/ascundere panou și *Logout*.
- **Patient Sidebar**: informații despre pacient și nivelul de risc curent (Low/Medium/High).
- **Zona principală**: flux camera (stream) și distribuția emoțiilor curente.

Live Monitor (vizualizare istoric și spike-uri)

Prin butonul *Live Monitor* se deschide o fereastră peste ecran (overlay) care prezintă un grafic cu istoricul emoțiilor, o figură reprezentativă fiind 4.7. Fereastra se închide cu butonul *Close* sau prin click în afara zonei.

Insights (alerte și interpretare clinică)

Prin butonul *Insights* se deschide un panou lateral cu:

- alerte curente și alerte prelungite (prolonged),
- evenimente clinice derivate (listate cronologic).

Scopul este de a sintetiza situațiile relevante (ex. creșteri susținute ale indicatorilor negativi).

Notificări de risc

La schimbarea nivelului de risc, aplicația afișează o notificare scurtă (*Risk Increased / Risk Decreased*) cu tranziția dintre niveluri. Notificarea dispare automat după câteva secunde.

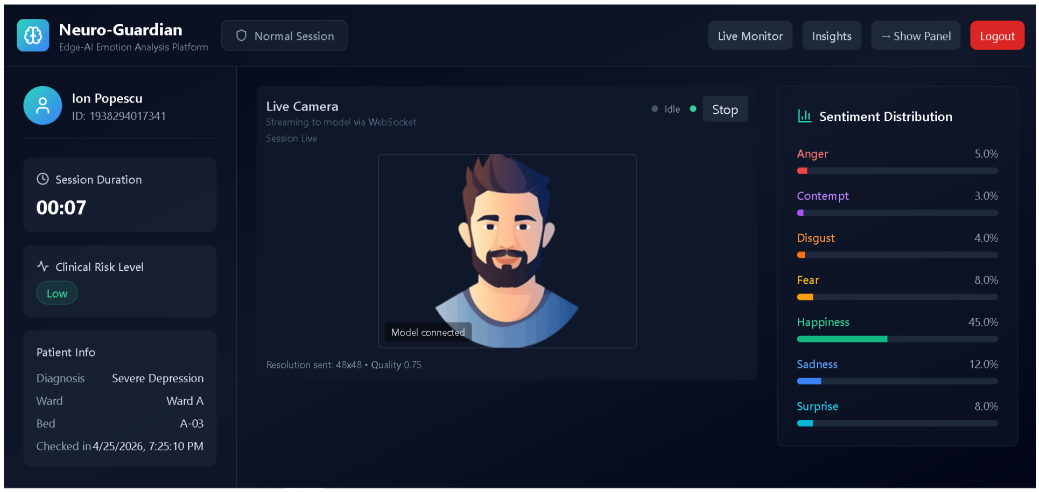


Figura 4.23: Modul DOCTOR: tabloul de monitorizare și instrumentele de analiză.

Capitolul 5

Concluzii

Prezenta lucrare a abordat optimizarea sistemelor de recunoaștere a emoțiilor faciale bazate pe rețele neuronale cu impulsuri prin preprocesare adaptivă, urmărind atât validarea potențialului SNN-urilor ca extractoare de trăsături spațio-temporale, cât și integrarea practică a unui motor de inferență neuromorfic într-o aplicație clinică funcțională.

5.1 Sinteza concluziilor experimentale

Evaluarea pe setul de date CK+ prin *10-fold cross-validation* a confirmat că modelul CSNN 3D antrenat nesupervizat prin STDP extrage trăsături temporale relevante pentru clasificarea emoțiilor faciale. Rezultatele comparative obținute sunt sintetizate după cum urmează:

- **Modelul CSNN 3D (propus):** Atinge o acuratețe medie de **48.02%**, depășind clasificatorul de referință (SVM pe pixeli bruți, 41.61%) cu un câștig net de **6.41%**. Acest fapt demonstrează că nucleele convoluționale specializate prin STDP codifică corelații spațio-temporale discriminative.
- **Modelul ANN echivalent:** A obținut **73.23%**, cuantificând costul utilizării unei reguli de învățare locale și nesupervizate față de optimizarea globală prin gradient (*backpropagation*).
- **Arhitectura CNN-LSTM:** Proiectată ca referință de tip *State-of-the-Art*, a atins **88.37%** $\pm$ **5.65%**, confirmând superioritatea transferului de cunoștințe (ResNet18) și a modelării explicite a dinamicii temporale.

Analiza hărților de trăsături a evidențiat că modelul SNN funcționează ca un extractor de trăsături fiziologice: emoțiile cu activare musculară intensă (*Happiness, Disgust*) au obținut un *recall* de **100%** pe anumite fold-uri, în timp ce emoțiile subtile (*Contempt, Sadness*) au condus la confuzii sistematice.

5.2 Impactul preprocesării adaptive

Experimentele au subliniat un avantaj structural al arhitecturii SNN: filtrul On/Off și codificarea prin latență oferă o robustețe nativă la zgomotul vizual. Pixelii statici din fundal nu generează impulsuri, spre deosebire de modelele ANN unde fundalul poate scădea acuratețea la aproximativ 30–40% în absența decupării. Pipelineul propus (Viola-Jones, KLT și eșantionarea centrată pe reperele faciale 17–67 din standardul 300-W) asigură concentrarea activității neuronale pe zonele cu relevanță semantică maximă (ochi, sprâncene, gură).

5.3 Sistemul Neuro-Guardian

Aplicația dezvoltată demonstrează viabilitatea integrării unui motor neuromorfic într-un flux clinic real. Elementele cheie ale implementării includ:

- **Arhitectura hibridă:** WebSocket pentru latență redusă, TCP pentru comunicarea cu serverul C++, PostgreSQL pentru date clinice și **InfluxDB** pentru monitorizarea seriilor temporale ale emoțiilor.
- **Mecanismul de histerezis:** Utilizarea pragurilor *70/55* pentru alertele de tip *Distress* și *Pain* elimină oscilațiile la pragurile de decizie, oferind notificări stabile.
- **Validare:** Testarea *white-box* a confirmat acoperirea completă a celor 9 drumuri independente (Complexitate Ciclomatică = 9).

5.4 Limitări identificate

Lucrarea prezintă câteva limitări inerente:

1. **Setul de date:** CK+ este colectat în condiții controlate, ceea ce poate limita generalizarea la emoții spontane.
2. **Dezechilibrul claselor:** Numărul redus de exemple pentru *Contempt* și *Fear* afectează negativ performanța, STDP prioritizând trăsăturile frecvente.
3. **Hardware:** Modelul rulează pe hardware convențional, beneficiile energetice ale SNN fiind maximizate doar pe cipuri precum Intel Loihi.

5.5 Direcții viitoare de cercetare

Pe baza rezultatelor obținute, se propun următoarele direcții de extindere:

- **Optimizarea hiperparametrilor:** Rafinarea pragului de activare (θ) și a ratei de învățare (μ_{th}) prin optimizare bayesiană.
- **Codificare temporală:** Înlocuirea *latency coding* alte metode de encodaare la nivelul întregii arhitecturi.
- **Antrenare End-to-End:** Implementarea unui strat *Fully Connected Spiking* antrenat fie prin *surrogate gradients*, fie prin STDP pentru a elimina necesitatea unui SVM extern.

În concluzie, lucrarea demonstrează că rețelele neuronale cu impulsuri sunt capabile să extragă trăsături relevante chiar și în regim nesupervizat, deschizând perspective reale pentru sisteme de recunoaștere a emoțiilor eficiente energetic, de tip *Edge AI*.

Bibliografie

[BAA+23] Sami Barchid, Benjamin Allaert, Amel Aissaoui, José Mennesson, and Chaabane C Djeraba. Spiking-fer: Spiking neural network for facial expression recognition with event cameras. In *Proceedings of the 20th International Conference on Content-Based Multimedia Indexing (CBMI '23)*, pages 1–7, 2023.

[BBJ+19] Zhenshan Bing, Ivan Baumann, Zhuangyi Jiang, Kai Huang, Caixia Cai, and Alois Knoll. Supervised learning in snn via reward-modulated spike-timing-dependent plasticity for a target reaching vehicle. *Frontiers in Neurorobotics*, 13:10, 2019.

[BL73] Timothy VP Bliss and Terje Lømo. Long-lasting potentiation of synaptic transmission in the dentate area of the anaesthetized rabbit following stimulation of the perforant path. *The Journal of physiology*, 232(2):331–356, 1973.

[BP98] Guo-qiang Bi and Mu-ming Poo. Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type. *Journal of Neuroscience*, 18(24):10464–10472, 1998.

[Bur06] Anthony N Burkitt. A review of the integrate-and-fire neuron model: I. homogeneous synaptic input. *Biological Cybernetics*, 95(1):1–19, 2006.

[Che17] Yanqing Chen. Mechanisms of winner-take-all and group selection in neuronal spiking networks. *Frontiers in computational neuroscience*, 11:20, 2017.

[DB92] Serena M Dudek and Mark F Bear. Homosynaptic long-term depression in hippocampus induced by low-frequency stimulation. *Proceedings of the National Academy of Sciences*, 89(10):4363–4367, 1992.

[DC15a] Peter U. Diehl and Matthew Cook. Unsupervised learning of digit recognition using spike-timing-dependent plasticity. *Frontiers in Computational Neuroscience*, 9:99, 2015.

[DC15b] Peter U Diehl and Matthew Cook. Unsupervised learning of digit recognition using spike-timing-dependent plasticity. *Frontiers in computational neuroscience*, 9:99, 2015.

[DPT01] Arnaud Delorme, Laurent Perrinet, and Simon J Thorpe. Networks of integrate-and-fire neurons using rank order coding b: Spike timing dependent plasticity and emergence of orientation selectivity. *Neurocomputing*, 38:539–545, 2001.

[EA24] Mireille El-Assal. *Unsupervised STDP-based Feature Learning for Video Analysis with Spiking Neural Networks*. Doctoral dissertation, Université de Lille, 2024.

[EF86] Paul Ekman and Wallace V. Friesen. A new pan-cultural facial expression of emotion. *Motivation and Emotion*, 10(2):159–168, 1986.

[Ekm92] Paul Ekman. An argument for basic emotions. *Cognition & Emotion*, 6(3-4):169–200, 1992.

[Fal19] Pierre Falez. *Improving Spiking Neural Networks Trained with Spike Timing Dependent Plasticity for Image Recognition*. PhD thesis, Université de Lille, 2019.

[FMT18] Paul Ferré, Franck Mamalet, and Simon J Thorpe. Unsupervised feature learning with winner-takes-all based stdp. *Frontiers in computational neuroscience*, 12:24, 2018.

[FTB+19] Pierre Falez, Pierre Tirilly, Ioan Marius Bilasco, Philippe Devienne, and Pierre Boulet. Multi-layered spiking neural network with target timestamp threshold adaptation and stdp. In *International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2019.

[GFES21] Wenzhe Guo, Mohammed E. Fouda, Ahmed M. Eltawil, and Khaled Nabil Salama. Neural coding in spiking neural networks: A comparative study for robust neuromorphic systems. *Frontiers in Neuroscience*, 15, 2021.

[GK02] Wulfram Gerstner and Werner M Kistler. *Spiking neuron models: Single neurons, populations, plasticity*. Cambridge university press, 2002.

[GM08] Tim Gollisch and Markus Meister. Rapid neural coding in the retina with relative spike latencies. *Science*, 319(5866):1108–1111, 2008.

[Gru23] Marcus Grum. Learning representations by crystallized back-propagating errors. In *Artificial Intelligence and Soft Computing - 22nd International Conference, ICAISC 2023, Zakopane, Poland, June 18-22, 2023, Proceedings, PartI*, Lecture Notes in Computer Science, pages 78–100, 2023.

[Heb02] Donald O Hebb. *The Organization of Behavior: A Neuropsychological Theory*. 2002.

[HH52] Alan L Hodgkin and Andrew F Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of Physiology*, 117(4):500–544, 1952.

[HLD21] Yuhuang Hu, Shih-Chii Liu, and Tobi Delbruck. v2e: From video frames to realistic dvs events. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1312–1321, 2021.

[HS81] Berthold KP Horn and Brian G Schunck. Determining optical flow. *Artificial intelligence*, 17(1-3):185–203, 1981.

[HS97] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, November 1997.

[HZ23] Manu Srinath Halvagal and Friedemann Zenke. The combination of hebbian and predictive plasticity learns invariant object representations in deep sensory networks. *bioRxiv*, pages 2023–03, 2023.

[HZRS16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778. IEEE, June 2016.

[Izh04] Eugene M Izhikevich. Which model to use for cortical spiking neurons? *IEEE Transactions on Neural Networks*, 15(5):1063–1070, 2004.

[Kas14] Nikola K. Kasabov. Neucube: A spiking neural network architecture for mapping, learning and understanding of spatio-temporal brain data. *Neural Networks*, 52:62–76, 2014.

[KGTM18] Saeed Reza Kheradpisheh, Mohammad Ganjtabesh, Simon J. Thorpe, and Timothée Masquelier. STDP-based spiking deep convolutional neural networks for object recognition. *Neural Networks*, 99:56–67, 2018.

[KPPC20] Alexander Kugele, Thomas Pfeil, Michael Pfeiffer, and Elisabetta Chicca. Efficient processing of spatio-temporal data streams with spiking neural networks. *Frontiers in Neuroscience*, 14:439, 2020.

[LBY20] D. Liu, N. Bellotto, and S. Yue. Deep spiking neural network for video-based disguise face recognition based on dynamic facial movements. *IEEE Transactions on Neural Networks and Learning Systems*, 31(6):1843–1855, 2020.

[LDP16] Jun Haeng Lee, Tobi Delbruck, and Michael Pfeiffer. Training deep spiking neural networks using backpropagation. *Frontiers in Neuroscience*, 10:508, 2016.

[LK81] Bruce D Lucas and Takeo Kanade. An iterative image registration technique with an application to stereo vision. *IJCAI*, 1981.

[LPSR18] Chankyu Lee, Priyadarshini Panda, Gopalakrishnan Srinivasan, and Kaushik Roy. Training deep spiking convolutional neural networks with stdp-based unsupervised pre-training followed by supervised fine-tuning. *Frontiers in Neuroscience*, 12:435, 2018.

[LWG+19] Dahua Li, Zhe Wang, Qiang Gao, Yu Song, Xiao Yu, and Chuhan Wang. Facial expression recognition based on electroencephalogram and facial landmark localization. *Technology and Health Care*, 27:1–15, 01 2019.

[LXJL22] Mengting Lan, Xiaogang Xiong, Zixuan Jiang, and Yunjiang Lou. PC-SNN: Supervised learning with local hebbian synaptic plasticity based on predictive coding in spiking neural networks. *CoRR*, abs/2211.15386, 2022.

[MBY21] E. Mansouri-Benssassi and J. Ye. Generalisation and robustness investigation for facial and speech emotion recognition using bio-inspired spiking neural networks. *Soft Computing*, 25:1717–1730, 2021.

[McC76] T. J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320, 1976.

[Meh71] Albert Mehrabian. *Silent Messages*. Wadsworth, Belmont, CA, 1971.

[MF67] Albert Mehrabian and Susan R. Ferris. Inference of attitudes from nonverbal communication in two channels. *Journal of Consulting Psychology*, 31(3):248–252, 1967.

[MW67] Albert Mehrabian and Morton Wiener. Decoding of inconsistent communications. *Journal of Personality and Social Psychology*, 6(1):109–114, 1967.

[QBG11] Damien Querlioz, Olivier Bichler, and Christian Gamrat. Simulation of a memristor-based spiking neural network immune to device variations. In *International Joint Conference on Neural Networks (IJCNN)*, pages 1775–1781, 2011.

[SM07] Gilad Silberberg and Henry Markram. Disynaptic inhibition between neocortical pyramidal cells mediated by martinotti cells. *Neuron*, 53(5):735–746, 2007.

[SMA00] Sen Song, Kenneth D Miller, and Laurence F Abbott. Competitive hebbian learning through spike-timing-dependent synaptic plasticity. *Nature Neuroscience*, 3(9):919–926, Sep 2000.

[ST94] Jianbo Shi and Carlo Tomasi. Good features to track. In *1994 Proceedings of IEEE Conference on Computer Vision and Pattern Recognition*, pages 593–600, 1994.

[STZP13] Christos health Sagonas, Georgios Tzimiropoulos, Stefanos Zafeiriou, and Maja Pantic. 300 faces in-the-wild challenge: The first facial landmark localization challenge. In *Proceedings of the IEEE international conference on computer vision workshops*, pages 397–403, 2013.

[SZ15] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015.

[TCKPS20] C. Tan, G. Ceballos, N. Kasabov, and N. Puthanmadam Subramaniyam. Fusionsense: Emotion classification using feature fusion of multimodal data and deep learning in a brain-inspired spiking neural network. *Sensors*, 20(18):5328, 2020.

[TG98] Simon Thorpe and Jacques Gautrais. Rank order coding. In *Computational Neuroscience*, pages 113–118. 1998.

[TGK+19] Amirhossein Tavanaei, Masoud Ghodrati, Saeed Reza Kheradpisheh, Timothée Masquelier, and Anthony Maida. Deep learning in spiking neural networks. *Neural Networks*, 111:47–63, 2019.

[Vap95] Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer Tracks in Natural Computing. Springer-Verlag, New York, NY, USA, 1995.

[VJ01] Paul Viola and Michael Jones. Robust real-time face detection. In *Proceedings eighth IEEE international conference on computer vision*, volume 2, pages 747–747, 2001.

[vRBT00] M. C. W. van Rossum, G. Q. Bi, and G. G. Turrigiano. Stable hebbian learning from spike timing-dependent plasticity. 20(23):8812–8821, 2000.

[WDZ+22] Binqiang Wang, Gang Dong, Yaqian Zhao, Rengang Li, Hongbin Yang, Wenfeng Yin, and Lingyan Liang. Spiking emotions: Dynamic vision emotion recognition using spiking neural networks. In *2022 International Conference on Algorithms, High Performance Computing and Artificial Intelligence (AHPCAI)*, pages 1–6, 2022.

[ZZL22] Dongcheng Zhao, Yi Zeng, and Yang Li. Backeisnn: A deep spiking neural network with adaptive self-feedback and balanced excitatory–inhibitory neurons. *Neural Networks*, 154:68–77, 2022.

[ZZPM16] Amir Zadeh, Rowan Zellers, Eli Pincus, and Louis-Philippe Morency. Mosi: Multimodal corpus of sentiment intensity and subjectivity analysis in online opinion videos. *ArXiv*, abs/1606.06259, 2016.